



Nuevo Modelo de XAI basado en Análisis Formal de Conceptos

Andrés Rubio Ramos



Nuevo Modelo de XAI basado en Análisis Formal de Conceptos

Andrés Rubio Ramos

Memoria presentada como parte de los requisitos para la obtención del título de Doble Grado en Ingeniería Informática. Tecnologías Informáticas y Matemáticas por la Universidad de Sevilla.

Tutorizada por

Joaquín Borrego Díaz

Agradecimientos

Querría agradecer a todas las personas que han hecho posible la realización de este trabajo y que me han acompañado a lo largo de estos últimos 5 años de mi formación.

En primer lugar, a mis padres y a mi abuela, por su amor incondicional, su apoyo constante y su confianza en mí en cada etapa de este camino. Gracias por no juzgarme aunque la mayoría de días estuviese sin aprovechar el tiempo en mis cosas, ya que sabíais que al final lo acabaría sacando todo.

A mi novia Inés, por este increíble año a mi lado, por su paciencia, por su ánimo y por creer siempre en mí en las situaciones más imposibles. Su compañía y cariño han sido mi pilar fundamental en estos meses de trabajo.

A mis compañeros del doble grado, con quienes he compartido no solo clases y proyectos, sino también experiencias, retos y aprendizajes que recordaré siempre. Espero que estas amistades nunca se pierdan y que nos sigamos viendo todo lo que se pueda.

A todos mis profesores y profesoras de la carrera, por transmitirme su conocimiento y por contribuir a mi crecimiento académico y personal.

Y, por último, al profesor Joaquín, tutor de este Trabajo de Fin de Grado, por su guía, su disponibilidad para las reuniones y su confianza a lo largo del desarrollo de esta investigación. Su orientación y ayuda sobre como orientar el trabajo ha sido fundamental para llevarlo a cabo.

A todos, gracias.

Resumen

El auge del aprendizaje automático ha causado un creciente interés por comprender sus decisiones, situando este trabajo en el ámbito de la *Inteligencia Artificial Explicable* (XAI) y sugiere un modelo que utiliza el Análisis Formal de Conceptos (AFC) como una herramienta para crear conocimiento que se pueda interpretar a partir de datos organizados.

La metodología propuesta se concentra en las implicaciones lógicas entre atributos, obtenidas de contextos formales contruidos con datos del mundo real. Estas implicaciones, que se organizan en bases como Stem y Luxenburger, representan conocimiento jerárquico y simbólico, y pueden aclarar o prever resultados de modelos predictivos tradicionales.

A lo largo del trabajo, se lleva a cabo un análisis práctico utilizando el famoso conjunto de datos *Titanic*, junto con un segundo caso basado en *Churn Modelling*, que trata de predecir la cancelación de cuentas bancarias. En ambos casos, se aplican técnicas de discretización en atributos no booleanos, se estudian distintos umbrales de soporte y confianza, y se crean contextos formales que facilitan la extracción de implicaciones.

Después, se evalúa cómo estas relaciones son útiles en tareas explicativas, como el análisis de verdaderos y falsos positivos. También se analizan parámetros como el número de objetos, el filtrado de reglas o el tratamiento de atributos objetivo.

Este trabajo marca un avance hacia la creación de modelos de IA más comprensibles y justificables desde una perspectiva lógica y simbólica, estableciendo las bases para futuras mejoras y aplicaciones en contextos reales.

Palabras clave: Análisis Formal de Conceptos (AFC), Inteligencia Artificial Explicable (XAI), Bases de implicaciones, Discretización, Razonamiento simbólico, Interpretabilidad y Extracción de conocimiento.

Abstract

The rise of machine learning has sparked a growing interest in understanding machine learning decisions, placing this work within the realm of Explainable Artificial Intelligence (XAI). It suggests a model that uses Formal Concept Analysis (CFA) as a tool to create interpretable knowledge from organized data.

The proposed methodology focuses on the logical implications between attributes, obtained from formal contexts constructed with real-world data. These implications, organized in bases such as Stem and Luxenburger, represent hierarchical and symbolic knowledge and can clarify or predict the results of traditional predictive models.

Throughout the work, a practical analysis is carried out using the famous Titanic dataset, along with a second case based on Churn Modeling, which attempts to predict bank account cancellations. In both cases, discretization techniques are applied to non-Boolean attributes, different support and confidence thresholds are studied, and formal contexts are created to facilitate implication extraction.

The usefulness of these relationships in explanatory tasks, such as the analysis of true and false positives, is then evaluated. Parameters such as the number of objects, rule filtering, and the processing of target attributes are also analyzed.

This work marks a step toward creating more understandable and justifiable AI models from a logical and symbolic perspective, laying the groundwork for future improvements and applications in real-world contexts.

Keywords: Formal Concept Analysis (FCA), Explainable Artificial Intelligence (XAI), Implication Bases, Discretization, Symbolic Reasoning, Interpretability and Knowledge Extraction.

Índice general

Introducción	3
1. Análisis Formal de Conceptos	7
1.1. Principios del Análisis de Conceptos Formales	8
1.2. Elementos del Análisis de Conceptos Formales	10
1.3. Algoritmos de cálculo de conceptos	14
1.4. Atributos No Booleanos y Escalas	23
2. Razonamiento en AFC	29
2.1. Implicaciones entre atributos	29
2.2. Bases de implicaciones	31
2.3. Razonamiento con la base: cálculo lógico	32
2.4. Aplicación: cierre implicacional	33
2.5. Pseudo-intenciones y la base Stem	33
2.6. Bases de Luxenburger	38
3. AFC como Herramienta en Machine Learning	47
3.1. Generación de conocimiento a partir de implicaciones	48

3.2. Integración de implicaciones en modelos de Machine Learning	54
3.3. Limitaciones y desafíos del enfoque con AFC	56
3.4. Impacto del Número de Objetos	60
4. Caso de estudio: Titanic	65
4.1. Descripción del conjunto de datos	65
4.2. Preparación y construcción del contexto formal	67
4.3. Extracción de implicaciones	71
4.4. Primeros resultados y análisis	77
5. Ajustes y mejoras en la extracción de conocimiento	87
5.1. Modificaciones en las bases de implicaciones	87
5.2. Ajuste de hiperparámetros: confianza, soporte y discretización	92
5.3. Validación externa del modelo	96
5.4. Caso de estudio adicional: <i>Churn Modelling</i>	98
6. FCA como Herramienta para la Explicabilidad en IA	107
6.1. Análisis Explicable de Resultados: VP y FP	107
6.2. Análisis Explicable de Resultados: VN y FN	112
7. Conclusiones y Trabajo Futuro	117
Referencias	119

Introducción

Objetivo del trabajo

El propósito fundamental de esta memoria es proponer y validar un modelo de Inteligencia Artificial Explicable (XAI) basado en el Análisis Formal de Conceptos (AFC). El modelo se apoya en las bases de implicaciones (Stem y Luxenburger) para extraer conocimiento simbólico a partir de datos reales y utilizarlo, tanto para **explicar** como para **predecir** resultados de clasificación. La validez y la generalidad del enfoque se comprueban en dos conjuntos de datos de referencia:

- *Titanic: Machine Learning from Disaster* (Kaggle) – problema de supervivencia de pasajeros [6].
- *Churn Modelling* (Kaggle) – problema de fuga de clientes bancarios [16].

Con ello se busca:

1. Demostrar que las implicaciones lógicas obtenidas con AFC son útiles para interpretar predicciones y métricas clásicas (VP, FP, FN, VN).
2. Evaluar el impacto de los parámetros de soporte y confianza y de la discretización sobre la precisión del modelo.
3. Analizar la escalabilidad del algoritmo y la necesidad de herramientas externas (p. ej. ConExp) para contextos de gran tamaño.

Estructura de la memoria

El documento se organiza en siete capítulos, precedidos por esta introducción y seguidos de las conclusiones:

1. **Capítulo 1 – Análisis Formal de Conceptos.** Fundamentos matemáticos y algoritmos de construcción de retículos.
2. **Capítulo 2 – Razonamiento en AFC.** Implicaciones, bases de conocimiento y cálculo lógico.
3. **Capítulo 3 – AFC como Herramienta en Machine Learning.** Integración del conocimiento implicacional en tareas predictivas.
4. **Capítulo 4 – Caso de estudio: *Titanic*.** Preparación de datos, generación de contextos y evaluación.
5. **Capítulo 5 – Ajustes y mejoras.** Optimización de hiperparámetros y **caso adicional: *Churn Modelling*.**
6. **Capítulo 6 – AFC para la Explicabilidad en IA.** Análisis de VP–FP y VN–FN con reglas simbólicas.
7. **Capítulo 7 – Conclusiones y trabajo futuro.**

Flujo de trabajo

1. **Preprocesamiento** – discretización de variables numéricas y *one-hot encoding* de categóricas.
2. **Construcción del contexto formal** con concepts (Titanic) o ConExp (Churn), en función del tamaño.
3. **Extracción de bases Stem y Luxenburger** (algoritmo *Next Closure* optimizado).
4. **Evaluación** – aplicación de cierres implicacionales para predecir atributos objetivos y cálculo de ratio de acierto, FP y FN sobre un conjunto de prueba estratificado.
5. **Visualización y análisis** – gráficos y tablas generados con matplotlib para estudiar la influencia de soporte, confianza u otros parámetros.

Materiales usados

Recurso	Función principal	Detalles
Python 3.12	Lenguaje de programación	Único lenguaje de programación usado en el trabajo.
pandas	Carga y limpieza de datos tabulares	Lectura de CSV, tratamiento de valores nulos y discretización.
numpy	Cálculo numérico	Operaciones vectorizadas para soporte y confianza.
matplotlib	Visualización	Histogramas de soporte/confianza y curvas de precisión.
concepts	Construcción de contextos y retículos	Objeto Context y jerarquía de conceptos.
ConExp	Cálculo de bases implícacionales en grandes contextos	Usado en el caso Churn para generar más de 117 000 implicaciones en pocos minutos.
Datasets	Datos de entrada	Titanic (891 instancias) y Churn Modelling (10 000 instancias), extraídos de Kaggle (27 mayo 2025).
Herramientas de IA	Asistencia instrumental	Se emplearon modelos de lenguaje para <i>formatear tablas, detectar inconsistencias menores, sugerir sinónimos y pulir redacción</i> . No tomaron decisiones metodológicas ni definieron estructura.

Cuadro 1: Recursos y herramientas empleados en el desarrollo del trabajo.

El repositorio completo con todos los notebooks, scripts .py, conjuntos de datos discretizados y resultados generados está disponible públicamente en GitHub en la siguiente dirección:

<https://github.com/Andros5/TFG-AndresRubio.git>

1 | **Análisis Formal de Conceptos**

Para la redacción de este capítulo, se han utilizado como referencias [4] y [9]. En este capítulo se introduce el concepto principal del trabajo, Análisis de Conceptos Formales o Formal Concept Analysis (FCA).

El Análisis Formal de Conceptos (AFC), introducido en 1982 por Rudolf Wille, es una metodología matemática diseñada para estructurar y representar el conocimiento de manera jerárquica. Su aplicación abarca diversos campos como la inteligencia artificial, la minería de datos y la organización del conocimiento, facilitando la identificación de relaciones entre conceptos dentro de un dominio específico.

Este enfoque se basa en la formalización de conceptos a partir de datos, estableciendo conexiones entre objetos y atributos mediante contextos formales. A partir de estos, se generan retículos de conceptos que permiten visualizar de manera clara la estructura del conocimiento dentro de un sistema.

El AFC se fundamenta en la teoría de la conexión de Galois, un marco matemático que, aplicado a AFC, formaliza la correspondencia entre objetos y atributos. Gracias a esto, es posible organizar la información de manera estructurada, identificando patrones y relaciones implícitas en los datos.

En este capítulo, se abordarán los fundamentos del Análisis Formal de Conceptos, incluyendo la definición de contextos formales, la construcción de retículos de conceptos y los algoritmos utilizados para la extracción de conceptos. Además, se explorará la representación de atributos no booleanos mediante escalas, así como su impacto en la estructuración del conocimiento.

A lo largo de este análisis, se verá cómo el AFC contribuye a la conceptualización automática y la generación de ontologías, aspectos clave para la representación del conocimiento en sistemas inteligentes. Que después se usarán para crear relaciones

entre conceptos que ayudarán a hacer predicciones sobre otros atributos con mayor importancia.

1.1 Principios del Análisis de Conceptos Formales

La representación del conocimiento a través del AFC se basa en dos principios fundamentales:

- Extracción automática de una ontología a partir de datos (conceptualización).
- Generación automática de un sistema experto basado en el dominio analizado.

| Definición 1.1. *Una **ontología**, en filosofía, es la rama de la metafísica que estudia la naturaleza de la existencia.*

| Definición 1.2. *En ciencias de la computación e inteligencia artificial, una **ontología** es una estructura de datos que define las relaciones entre los conceptos de un dominio o área del conocimiento.*

El concepto de ontología no tiene una definición universalmente aceptada, pero en términos generales representa una **conceptualización** de un dominio, es decir, una forma estructurada de entender y organizar la información dentro de un ámbito específico.

Una ontología está compuesta por un conjunto de conceptos, definiciones y relaciones que establecen restricciones sobre la estructura de una parte del mundo real. Su objetivo es proporcionar un marco claro y compartido que facilite la comunicación y el intercambio de información entre distintos sistemas.

Según Gruber [10], una ontología es una representación explícita de una conceptualización. Esto implica que no solo define los elementos de un dominio, sino también las reglas y conexiones que rigen su interacción.

El desarrollo de ontologías debe basarse en el consenso, evitando la subjetividad individual. Para garantizar una interpretación uniforme de la información, las ontologías suelen expresarse mediante estructuras formales, como lógica de primer orden, eliminando ambigüedades y facilitando su integración en sistemas computacionales y bases de conocimiento compartidas.

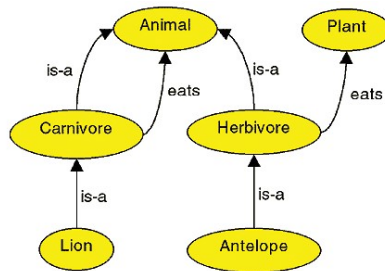


Figura 1.1: Grafo de una ontología sobre animales.

A continuación, se presenta un ejemplo de una ontología sobre animales (véase Figura 1.1):

Se puede ver que se estructura en torno a tres elementos principales:

- **Clases:** Carnívoro, Herbívoro, Planta y Animal
- **Relaciones:** "es un" y "se alimenta de"
- **Individuos:** León y Antílope

El propósito de esta ontología es organizar y representar de manera estructurada las características y relaciones entre diferentes especies, facilitando su análisis y estudio.

La relación "is-a" es un principio fundamental en la organización jerárquica de una ontología, estableciendo una estructura de herencia donde los conceptos más específicos heredan las propiedades de los más generales.

Para verificar la validez de esta jerarquía, se aplica el siguiente criterio: *Si un elemento pertenece a una subclase, ¿también pertenece a la clase superior?*

Por ejemplo:

- Perro *is-a* Mamífero → Todos los perros son mamíferos.
- Mamífero *is-a* Animal → Todos los mamíferos son animales.

Dentro de una ontología, los individuos representan instancias concretas de una clase. Por ejemplo, León y Antílope son individuos de la clase Mamífero. Las relaciones establecen conexiones entre estos individuos y otros elementos de la ontología, ya sean:

- Jerárquicas ("is-a"): Definen la pertenencia a una clase superior.
- No jerárquicas: Expresan interacciones entre individuos o clases, como "se alimenta de".

A partir de estas relaciones, se pueden inferir conocimientos adicionales. Por ejemplo: *"Todo carnívoro come (al menos) un animal", "Los herbívoros solo comen plantas"*.

1.2 Elementos del Análisis de Conceptos Formales

El Análisis de Conceptos Formales, como su propio nombre indica es un Análisis de datos que trabaja con una abstracción de los Conceptos que manejamos los humanos y Formaliza la idea de dicho concepto.

El estudio comienza con la representación de la información como contextos formales, es decir, objetos con sus respectivas propiedades (que llamaremos atributos).

| Definición 1.3 (Contexto formal). En AFC, un **Contexto formal** K es una terna (O, A, I) , donde:

- O es el conjunto de objetos.
- A es el conjunto de atributos.
- $I \subseteq O \times A$ es una relación binaria entre objetos y atributos. También llamada *relación de incidencia*.

Formalmente, la **relación binaria** I se puede considerar como un grafo dirigido desde los nodos pertenecientes a el conjunto de objetos (O) hacia los nodos pertenecientes a el conjunto de atributos (A). O también como una tabla donde los objetos ocupan las filas, los atributos las columnas y si una celda está marcada es que el objeto de esa fila posee el atributo de esa columna. La forma en tabla es la mas cómoda y práctica para visualizar y procesar.

	Selva	Agua	Bosque	Pez	Planta	Mamífero
León	x					x
Carpa		x		x		
Delfín		x				x
Oso			x			x
Cebra	x					x
Pino			x		x	

Cuadro 1.1: Clasificación de seres vivos.

Ejemplos 1.1. Supongamos el siguiente contexto, que viene dado en forma de la Tabla 1.1 de pares (objeto, atributo):

La tabla representa un conjunto de seres vivos al que se le asocian una serie de características para su estudio.

Los objetos en nuestro caso son los animales que se considerarán en el estudio:

- León (L)
- Carpa (Ca)
- Delfín (D)
- Oso (Os)
- Cebra (Ce)
- Pino (P)

Los atributos serían las características que se han considerado para el estudio:

- Habitan en la selva (Selva)
- Habitan en el agua (Agua)
- Habitan en el bosque (Bosque)
- Es un tipo de pez (Pez)
- Es un tipo de planta (Planta)
- Es un tipo de mamífero (Mamífero)

En resumen en nuestro ejemplo:

- $O = \{L, Ca, D, Os, Ce, P\}$

- $A = \{\text{Selva, Agua, Bosque, Pez, Planta, Mamifero}\}$
- I está representada en la Tabla 1.1, es decir, dos elementos están relacionados por I si tienen una X en la casilla correspondiente.

Derivación

Sea (O, A, I) un contexto formal. La operación básica sobre Contextos Formales es la denominada **derivación**. Aunque en realidad son dos según si el dominio de partida es un subconjunto de objetos o de atributos.

Definición 1.4. Sean $X \subseteq O$ e $Y \subseteq A$. La operación de **derivación** se define como sigue:

- La derivada sobre X se denomina **intención**, y es el conjunto de atributos comunes a todos los objetos de X :

$$X' = \{y \in A : \forall x \in X, xIy\}$$

- La derivada sobre Y se denomina **extensión**, y es el conjunto de todos los objetos que poseen todos los atributos de Y :

$$Y' = \{x \in O : \forall y \in Y, xIy\}$$

Ejemplos 1.2. En el ejemplo anterior, se verifica que:

- Intención: $\{L, Os\}' = \{\text{Mamifero}\}$
- Extensión: $\{\text{Agua, Mamifero}\}' = \{D\}$

Propiedades 1.1. Sean $X, X_1, X_2 \subseteq O$. Entonces:

1. $X_1 \subseteq X_2 \Rightarrow X_2' \subseteq X_1'$
2. $X \subseteq (X')'$
3. $X' = X'''$

Demostración. Las demostraciones se pueden encontrar en [9].

Propiedades 1.2. Sean $Y, Y_1, Y_2 \subseteq A$. Entonces:

1. $Y_1 \subseteq Y_2 \Rightarrow Y_2' \subseteq Y_1'$

2. $Y \subseteq (Y')'$
3. $Y' = Y'''$

Demostración. Las demostraciones se pueden encontrar en [9].

Proposición 1.1. Si T es un conjunto de índices, para cada $t \in T$, $X_t \subseteq O$ conjunto de objetos, entonces

$$\left(\bigcup_{t \in T} X_t \right)' = \bigcap_{t \in T} X_t'.$$

Lo mismo se aplica para conjuntos de atributos.

Demostración. Sea $a \in A$ un atributo. Entonces:

$$\begin{aligned} a \in \left(\bigcup_{t \in T} X_t \right)' &\Leftrightarrow x I a \text{ para todo } x \in \bigcup_{t \in T} X_t \\ &\Leftrightarrow x I a \text{ para todo } x \in X_t, \text{ para todo } t \in T \\ &\Leftrightarrow a \in X_t' \text{ para todo } t \in T \\ &\Leftrightarrow a \in \bigcap_{t \in T} X_t'. \end{aligned}$$

Conceptos formales

Una vez introducida la operación de derivación ya podemos definir el objeto fundamental en el que se basa esta teoría.

Definición 1.5 (Concepto formal). Es un par (X, Y) del contexto $M = (O, A, I)$ que debe cumplir lo siguiente:

- $X \subseteq O$ (la extensión del concepto)
- $Y \subseteq A$ (la intención del concepto)
- $X' = Y$ y $Y' = X$

El concepto formal (X, Y) posee muy buenas características dentro del Análisis Formal de Conceptos, ya que representa una correspondencia completa entre un conjunto de objetos X y un conjunto de atributos Y . Esta correspondencia garantiza que todos los objetos de X comparten exactamente los atributos de Y , y viceversa, lo que permite una representación precisa y no ambigua del conocimiento contenido en la

relación I . Adicionalmente, al cumplir que $X' = Y$ y $Y' = X$, el concepto formal refleja una simetría lógica que lo hace especialmente útil para la estructuración jerárquica de datos en una jerarquía de conceptos, facilitando así el análisis, la comparación y la inferencia dentro del dominio estudiado.

Ejemplos 1.3. Siguiendo con el ejemplo:

1. $(\{L, Ce\}, \{Selva, Mamifero\})$ es un concepto.
2. $(\{Ca\}, \{Agua, Pez\})$ es un concepto.
3. $(\emptyset, \emptyset)$ no es un concepto ya que $\emptyset' = A \neq \emptyset$
4. $(\emptyset, A)$ es un concepto ya que $\emptyset' = A$ y $A' = \emptyset$

Schütt [20] da una estimación del número de conceptos dependiendo de $|I|$:

Proposición 1.2 (Cota superior del número de conceptos). Sea (O, A, I) un contexto formal, donde O es el conjunto de objetos, A es el conjunto de atributos, e $I \subseteq O \times A$ es la relación de incidencia. Sea $|I|$ el número de pares en la relación I . Si $|I| > 2$, entonces el número M de conceptos formales cumple la siguiente desigualdad:

$$M \leq \frac{3}{2} \sqrt{2^{|I|+1}} - 1$$

Otras acotaciones son discutidas en [1]. Encontrar el número de conceptos formales se sabe que es un problema #P-completo (Kuznetsov [14]).

1.3 Algoritmos de cálculo de conceptos

Cálculo de conceptos

Ejemplos 1.4. Calculemos el menor concepto que contiene a León y a Deflín:

- $X = \{L, D\}'' = \{\text{Mamifero}\}' = \{L, D, Os, Ce\}$
- $Y = \{L, D\}' = \{\text{Mamifero}\}$

Por tanto es $(\{L, D, Os, Ce\}, \{\text{Mamifero}\})$

| Definición 1.6 (Relación de orden). Una relación binaria R sobre un conjunto M se llama una relación de orden (parcial) (o simplemente un orden), si satisface las siguientes condiciones para todos los elementos $x, y, z \in M$:

1. $x R x$ (reflexividad)
2. $x R y \wedge x \neq y$ implican que no se cumple $y R x$ (antisimetría)
3. $x R y \wedge y R z$ implican $x R z$ (transitividad)

Un conjunto ordenado¹ es un par $(M, \leq)$, donde M es un conjunto y $\leq$ es una relación de orden sobre M .

Dentro del Análisis Formal de Conceptos, la noción de *subconcepto* permite establecer relaciones de orden entre conceptos formales.

| Definición 1.7 (Subconcepto). Dado un contexto formal (O, A, I) y dos conceptos formales (X_1, Y_1) y (X_2, Y_2) , se dice que el primero es un **subconcepto** del segundo si se cumple al menos una —y por tanto ambas— de las siguientes condiciones:

- $X_1 \subseteq X_2$. Es decir, todos los objetos que pertenecen al subconcepto también están presentes en el concepto más general.
- $Y_2 \subseteq Y_1$. Es decir, que el subconcepto contiene una intención más específica que el concepto del que parte.

Esta relación se representa mediante la notación:

$$(X_1, Y_1) \leq (X_2, Y_2)$$

Si C_1 es subconcepto de C_2 y $C_1 \neq C_2$ su notación sería:

$$C_1 \preceq C_2$$

Ejemplos 1.5. $(\{L, Ce\}, \{\text{Selva, Mamifero}\}) \leq (\{L, D, O, Ce\}, \{\text{Mamifero}\})$

Proposición 1.3. Dado un conjunto de objetos $X \subseteq O$, el **menor concepto que lo contiene** es:

$$C_1 = (X_1, Y_1) = (X'', X''')$$

¹También llamado conjunto parcialmente ordenado o poset.

(recuerda que $X''' = X'$).

Dado un conjunto de atributos $Y \subseteq A$, el **mayor concepto que los contiene** es:

$$C_2 = (X_2, Y_2) = (Y', Y'').$$

Proposición 1.4 (Mayor subconcepto común). Dado un contexto formal (O, A, I) y dos conceptos formales $C_1 = (X_1, Y_1)$ y $C_2 = (X_2, Y_2)$, existe el mayor subconcepto común que es:

$$C_1 \wedge C_2 = ((X_1 \cap X_2)'', Y_1 \cup Y_2)$$

Proposición 1.5 (Menor subconcepto común). Dado un contexto formal (O, A, I) y dos conceptos formales $C_1 = (X_1, Y_1)$ y $C_2 = (X_2, Y_2)$, existe el menor subconcepto común que es:

$$C_1 \vee C_2 = (X_1 \cup X_2, (Y_1 \cap Y_2)'')$$

Tras establecer la relación de subconcepto entre los conceptos formales y observar cómo se relacionan entre sí a través de la extensión y la intención, surge una pregunta natural: ¿podemos usar esta ordenación para algo más allá de organizar los conceptos de manera jerárquica?

Es en este punto donde entra en juego una herramienta fundamental del Análisis Formal de Conceptos: el **retículo de conceptos**.

| Definición 1.8. Un conjunto parcialmente ordenado $L := (L, \leq)$ es un **retículo** si, para cada par de elementos x e y en L , existen tanto el supremo $x \vee y$ como el ínfimo $x \wedge y$.

Se dice que L es un **retículo completo** si el supremo $\bigvee X$ y el ínfimo $\bigwedge X$ existen para todo subconjunto X de L .

El retículo es una estructura algebraica que organiza los conceptos de acuerdo con la relación de subconcepto, proporcionando un marco más formal y estructurado para analizar las relaciones entre los conceptos de un dominio.

| Definición 1.9 (Retículo de conceptos). El conjunto de todos los conceptos formales de (O, A, I) ordenado según $\leq$ se denota por $\underline{\mathfrak{B}}(O, A, I)$ y se llama el **retículo de conceptos del contexto formal** (O, A, I) .

| Teorema 1.1 (Teorema Básico sobre Retículos de Conceptos). El conjunto $\underline{\mathfrak{B}}(O, A, I)$ es un **retículo completo**, en el cual el ínfimo y el supremo vienen dados por:

$$\bigwedge_{t \in T} (X_t, Y_t) = \left(\bigcap_{t \in T} X_t, \left(\bigcup_{t \in T} Y_t \right)'' \right),$$

$$\bigvee_{t \in T} (X_t, Y_t) = \left(\left(\bigcup_{t \in T} X_t \right)'' , \bigcap_{t \in T} Y_t \right).$$

Demostración. Primero explicamos la fórmula del ínfimo. Como $X_t = Y_t'$ para cada $t \in T$, el par

$$\left(\bigcap_{t \in T} X_t, \left(\bigcup_{t \in T} Y_t \right)'' \right)$$

puede reescribirse, por la Proposición 1.1, como

$$\left(\left(\bigcup_{t \in T} Y_t \right)' , \left(\bigcup_{t \in T} Y_t \right)'' \right),$$

es decir, tiene la forma (Z', Z'') y, por lo tanto, es ciertamente un concepto. Que este sea el ínfimo —es decir, el mayor subconcepto común de los conceptos (X_t, Y_t) — se deduce inmediatamente del hecho de que la extensión de este concepto es exactamente la intersección de las extensiones de (X_t, Y_t) .

La fórmula para el supremo se justifica de forma análoga. Por tanto, hemos demostrado que $\mathfrak{B}(O, A, I)$ es un retículo completo. |

Hemos concluido con que el conjunto de todos los conceptos de un contexto es un *retículo completo* con respecto al orden $\leq$, es decir:

1. $\leq$ es un orden (parcial).
2. Para cualesquiera número de conjunto de conceptos, existe un único ínfimo que es el menor superconcepto común y un único supremo que es el mayor subconcepto común.

Una de las maneras más efectivas de representar visualmente un retículo de conceptos es mediante el **diagrama de Hasse**.

| **Definición 1.10 (Diagrama de Hasse).** Dado un contexto $M = (O, A, I)$. Es un grafo dirigido en la que los conceptos se colocan como vértices del grafo, y las relaciones de subconcepto entre ellos se muestran mediante aristas dirigidas en sentido ascendente. Donde dados C_1 y C_2 conceptos del contexto, un arco $C_1 \rightarrow C_2$ representa:

- C_1 es subconcepto de C_2 .
- $\nexists C$ concepto $| C_1 \leq C \leq C_2$ (ejemplo de la condición en la Figura 1.2)

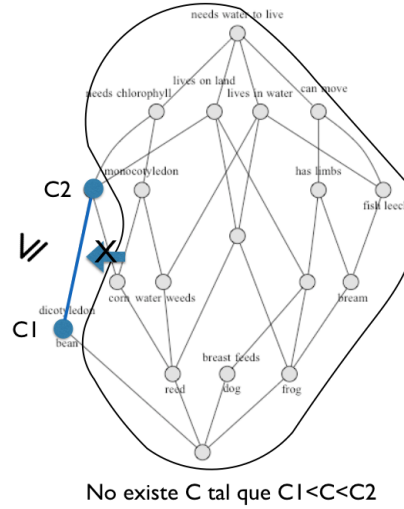


Figura 1.2: Ejemplo visual de la segunda condición sobre los arcos entre conceptos.

Nota 1.1. ConExp² implementa una herramienta para la visualización de **diagramas de Hasse**.

Si disponemos del **diagrama de Hasse** es sencillo determinar $C_1 \wedge C_2$ y $C_1 \vee C_2$. Para determinar $C_\wedge = C_1 \wedge C_2$ solo tenemos que encontrar en el diagrama el que tenga un camino mínimo desde $C_\wedge$ hasta C_1 y C_2 , o sea, el primer concepto por debajo de ambos. Y por contraparte para determinar $C_\vee = C_1 \vee C_2$ tenemos que encontrar el que tenga un camino mínimo desde C_1 y C_2 hasta $C_\vee$, o sea, el primer concepto por encima de ambos. $C_\wedge$ y $C_\vee$ siempre existen ya que (A', A) es subconcepto de todos los demás, y también cada concepto es subconcepto de (O, O') (ambas conclusiones son evidentes viendo la definición 1.7). La siguiente imagen muestra un ejemplo de cómo determinarlos ayudándonos del diagrama de la Figura 1.3:

Ejemplos 1.6. Para el ejemplo de los seres vivos se tendría el siguiente diagrama de Hasse (Fig. 1.4), donde:

- Si un nodo tiene escrito en la parte *inferior* un objeto significa que todos los nodos en sentido *ascendente* lo poseen.
- Si un nodo tiene escrito en la parte *superior* un atributo significa que todos los nodos en sentido *descendente* lo poseen.

²<http://conexp.sourceforge.net>

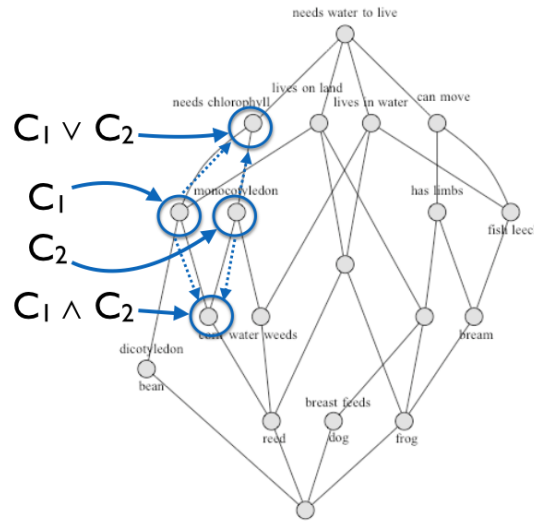


Figura 1.3: Ejemplo de la determinación de $C_{\wedge}$ y $C_{\vee}$ para C_1 y C_2 .

Extracción de todos los conceptos

El Análisis Formal de Conceptos no solo busca identificar algunos conceptos específicos, sino que también persigue extraer la totalidad de los conceptos que salen de un contexto formal dado. Esta tarea es fundamental para construir el retículo completo de conceptos, lo cual es esencial para representar las jerarquías y relaciones existentes en un dominio.

Problema principal

Dado un contexto formal $K = (O, A, I)$, una estrategia básica para extraer todos los conceptos consiste en calcular todos los pares (Y', Y'') para cada subconjunto de atributos $Y \subseteq A$. Cada uno de estos pares constituye un concepto formal si cumple que $Y'' = (Y')' = Y$.

Sin embargo, este método presenta una desventaja significativa: el número de subconjuntos posibles de atributos es $2^{|A|}$, lo cual hace que la complejidad del algoritmo sea exponencial. En la práctica, este enfoque se vuelve inviable para contextos con un número elevado de atributos.

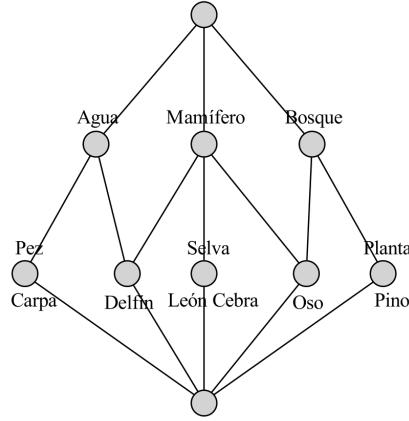


Figura 1.4: Diagrama de Hasse de los seres vivos.

Propiedades que permiten optimización

Afortunadamente, existen propiedades matemáticas que permiten mejorar la eficiencia en la extracción de conceptos deducidas de la Proposición 1.1 para el caso particular de conjuntos de objetos o atributos con solo un elemento. Por lo que las expresiones quedarían de esta manera:

- **La extensión de un conjunto de atributos** es la intersección de las extensiones individuales de cada atributo:

$$\{a_1, a_2, \dots, a_k\}' = \bigcap_{i=1}^k \{a_i\}'$$

- **La intención de un conjunto de objetos** también puede obtenerse como la intersección de las intenciones individuales de cada objeto:

$$\{o_1, o_2, \dots, o_m\}' = \bigcap_{j=1}^m \{o_j\}'$$

Estas propiedades permiten evitar la generación redundante de subconjuntos, reduciendo así significativamente la cantidad de operaciones necesarias.

Algoritmo optimizado de extracción

Una mejora sustancial sobre el enfoque básico se logra con el siguiente algoritmo iterativo, que construye las extensiones de todos los conceptos de manera progresiva:

Algorithm 1 Extracción de conceptos mediante extensiones

```

1: Entrada: Contexto formal  $(O, A, I)$ 
2: Salida: Conjunto de extensiones de conceptos
3: Inicializar  $L \leftarrow \{O\}$ 
4: for all  $a \in A$  do
5:   for all  $D \in L$  do
6:      $L \leftarrow L \cup \{D \cap \{a\}'\}$ 
7:   end for
8: end for
  
```

El Algoritmo 1 aprovecha que todas las extensiones de conceptos son cerradas bajo intersección, y genera de forma progresiva nuevas extensiones tomando la intersección de las ya conocidas con la extensión de cada nuevo atributo.

Ejemplos 1.7 (Aplicación del algoritmo). Supongamos el contexto formal dado en la Tabla 1.1, con los seres vivos y sus atributos. Aplicando el algoritmo, se inicializa la lista L con el conjunto total de objetos:

$$L = \{\{L, Ca, D, Os, Ce, P\}\}$$

Luego se recorren los atributos (Selva, Agua, Bosque, Pez, Planta, Mamifero), y se va intersecando con las extensiones individuales para cada atributo:

- $Selva' = \{L, Ce\}$

$$L = \{\{L, Ca, D, Os, Ce, P\}, \{L, Ce\}\}$$

- $Agua' = \{Ca, D\}$

$$L = \{\{L, Ca, D, Os, Ce, P\}, \{L, Ce\}, \{Ca, D\}, \emptyset\}$$

- $Bosque' = \{Os, P\}$

$$L = \{\{L, Ca, D, Os, Ce, P\}, \{L, Ce\}, \{Ca, D\}, \emptyset, \{Os, P\}\}$$

$$\blacksquare \text{Pez}' = \{Ca\}$$

$$L = \{\{L, Ca, D, Os, Ce, P\}, \{L, Ce\}, \{Ca, D\}, \emptyset, \{Os, P\}, \{Ca\}\}$$

$$\blacksquare \text{Planta}' = \{P\}$$

$$L = \{\{L, Ca, D, Os, Ce, P\}, \{L, Ce\}, \{Ca, D\}, \emptyset, \{Os, P\}, \{Ca\}, \{P\}\}$$

$$\blacksquare \text{Mamifero}' = \{L, D, Os, Ce\}$$

$$L = \{\{L, Ca, D, Os, Ce, P\}, \{L, Ce\}, \{Ca, D\}, \emptyset, \{Os, P\}, \{Ca\}, \{P\}, \{L, D, Os, Ce\}, \{D\}, \{Os\}\}$$

Nuestro contexto tendría un total de diez conceptos cuyas extensiones son las anteriormente calculadas, como también podemos observar en la Figura del diagrama de Hasse del retículo 1.4.

Para calcular todos los conceptos bastaría con calcular la intención de cada una para tener el par (extensión, intención) de cada concepto.

Complejidad y mejora práctica

Aunque la complejidad teórica sigue siendo exponencial en el peor de los casos, este algoritmo es significativamente más eficiente que el método básico, ya que evita muchas repeticiones y construcciones innecesarias. En la práctica, la mayoría de contextos generan muchos menos conceptos de los $2^{|A|}$ posibles, y esta técnica permite su extracción efectiva.

Además, herramientas como ConExp³ implementan versiones optimizadas de estos algoritmos, permitiendo su uso en contextos reales con cientos de objetos y atributos.

Importancia de la extracción completa

La extracción de todos los conceptos de un contexto no es un simple ejercicio técnico. Proporciona una visión completa y estructurada del conocimiento presente

³<http://conexp.sourceforge.net>

en los datos, permite generar jerarquías de conceptos, descubrir patrones ocultos y, especialmente, establecer las bases para la inferencia de reglas y la construcción de sistemas expertos.

Esta técnica es fundamental en la minería de conceptos, la construcción de ontologías formales y la representación de conocimiento en sistemas inteligentes.

1.4 Atributos No Booleanos y Escalas

Hasta ahora, en el Análisis Formal de Conceptos (AFC), hemos trabajado exclusivamente con contextos formales en los que los atributos eran *booleanos*, es decir, representaban la presencia o ausencia de una propiedad determinada en los objetos. No obstante, muchos datos reales contienen atributos que no se pueden representar de forma binaria de manera natural: edad, color, nivel educativo, tamaño, entre otros.

Para poder aplicar AFC a estos casos, es necesario transformar estos atributos en otros que sí se ajusten al esquema binario del contexto formal. Esta transformación se realiza mediante lo que se conoce como **escalado de atributos**, utilizando **escalas** específicas según el tipo de dato.

Atributos no booleanos

Un atributo no booleano es aquel que puede tomar más de dos valores o valores numéricos continuos, como por ejemplo:

- *Género*: masculino, femenino
- *Edad*: 18, 23, 45, etc.
- *Color*: rojo, verde, azul
- *Tamaño*: pequeño, mediano, grande

En estos casos, no basta con decir si un objeto tiene o no el atributo. Es necesario representar distintos valores, categorías o rangos de manera compatible con el marco binario del AFC. Para ello se aplican diferentes **tipos de escalas**.

Escalas

Una escala es un contexto formal auxiliar que permite transformar un atributo no booleano en varios atributos booleanos. Esta operación se conoce como *escalado*. Dependiendo de la naturaleza del atributo, se elige una escala adecuada que refleje su estructura semántica.

Existen distintos tipos de escalas utilizadas en AFC. A continuación se describen las más comunes:

Escala dicotómica

Se utiliza cuando el atributo puede tomar solo dos valores opuestos y mutuamente excluyentes. Por ejemplo:

- Atributo: *Aprobado examen*
- Valores: Sí / No
- Escala resultante: *examen:aprobado, examen:suspendido*

Cada valor se representa como un nuevo atributo booleano.

Escala nominal

Se utiliza cuando el atributo puede tomar múltiples valores no ordenados, como categorías. Por ejemplo:

- Atributo: *Color*
- Valores: rojo, azul, verde
- Escala resultante: *color:rojo, color:azul, color:verde*

No existe un orden natural entre los valores.

Escala ordinal

Se aplica cuando los valores del atributo tienen un orden natural, como niveles o rangos. Por ejemplo:

- Atributo: *Tamaño*
- Valores: pequeño < mediano < grande
- Escala resultante: *tamaño*: $\leq$ pequeño, *tamaño*: $\leq$ mediano, *tamaño*: $\leq$ grande

Estas escalas permiten hacer inferencias más precisas, ya que conservan la relación de orden.

Escala biordinal

Es una combinación de la escala ordinal y la escala inversa, útil para datos numéricos. Permite representar un valor tanto como “mayor o igual que” como “menor o igual que”.

Por ejemplo, para el atributo *edad*, podríamos usar:

$$\text{edad} \in \{\leq 18, \leq 25, \leq 40, \geq 18, \geq 25, \geq 40\}$$

Esto genera una estructura más rica que facilita la comparación entre conceptos.

Escalado de un contexto

Para escalar un atributo no booleano, se crea un nuevo contexto formal —llamado *contexto escalado*— donde se reescriben los valores originales en forma de nuevos atributos booleanos, siguiendo la escala seleccionada.

Ejemplos 1.8 (Escalado de género y edad). Supongamos un contexto con los siguientes objetos (personas) y atributos:

- Género: masculino, femenino
- Edad: 17, 22, 30, 45

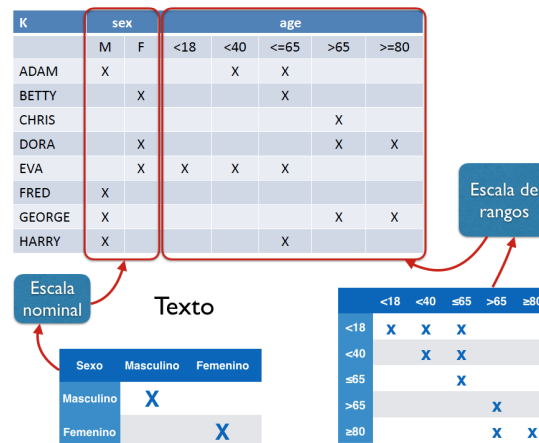


Figura 1.5: Ejemplo de escalado de género y edad.

Podemos aplicar:

- Una **escala dicotómica** para género:

género:masculino (M), género:femenino (F)

- Una **escala biordinal** para edad:

edad $\in \{< 18, < 40, \leq 65, > 65, \geq 80\}$

Cada persona se representará como una fila del contexto, con una marca en los atributos booleanos que correspondan a su perfil (véase la Figura 1.5).

Importancia del escalado correcto

Elegir la escala adecuada no es trivial. Si se utiliza una escala que no refleja correctamente la semántica del atributo, se pueden obtener conceptos poco informativos o incluso inconsistentes. Por eso, el especialista debe tener en cuenta:

- El tipo de valores que toma el atributo.
- El objetivo del análisis.
- La granularidad deseada en los resultados.

Por ejemplo, si el atributo es *tamaño* con valores pequeños, medianos y grandes, usar una escala nominal perdería la información de orden. En cambio, una escala ordinal permite mantener la relación entre los valores.

Retículos generados por contextos escalados

Una vez escalado el contexto, se puede aplicar todo el aparato teórico del Análisis Formal de Conceptos: cálculo de conceptos, construcción de retículos, extracción de implicaciones, etc.

El retículo resultante refleja ahora relaciones más complejas entre los objetos, gracias a la información adicional capturada por los atributos derivados. Esto permite:

- Una mejor clasificación de los datos.
- La detección de patrones basados en rangos o categorías.
- El análisis multivaluado con las mismas herramientas del AFC.

Esta técnica es fundamental para aplicar el AFC en entornos reales como bases de datos, sistemas expertos, minería de datos, sistemas de recomendación y en nuestro caso en inteligencia artificial, donde los atributos cualitativos y cuantitativos son la norma.

2 | Razonamiento en AFC

Para la redacción de este capítulo, se han utilizado como referencias [5] y [9].

Una vez que se han extraído todos los conceptos formales de un contexto, el siguiente paso es utilizar esta información para obtener conocimiento explícito e implícito. En esta sección se desarrollan los mecanismos lógicos que permiten formular implicaciones, calcular bases de conocimiento y aplicar esas reglas de forma sistemática para representar y explotar el conocimiento contenido en un contexto formal.

2.1 Implicaciones entre atributos

En AFC, una implicación entre atributos tiene la forma:

$$Y_1 \rightarrow Y_2$$

siendo $Y_1, Y_2 \subseteq A$ (atributos). Esta expresión puede interpretarse como una regla del tipo: "todo objeto que posea todos los atributos de Y_1 , también posee los de Y_2 ".

Ejemplos 2.1. En el ejemplo de los seres vivos, algunas implicaciones son:

- $\{\text{Selva}\} \rightarrow \{\text{Mamifero}\}$
- $\{\text{Pez}\} \rightarrow \{\text{Agua}\}$
- $\{\text{Planta}\} \rightarrow \{\text{Bosque}\}$

Nota 2.1. Para simplificar la notación también podríamos llegar a escribir los conjuntos sin corchetes.

| Definición 2.1. Un conjunto $T \subseteq A$ **respeto la implicación** $Y_1 \rightarrow Y_2$ si:

$$Y_1 \not\subseteq T \vee Y_2 \subseteq T$$

es decir, si $Y_1 \subseteq T$, entonces también $Y_2 \subseteq T$.

Nota 2.2. Si un conjunto $T \subseteq A$ respeta cada implicación $Y_1 \rightarrow Y_2$ contenida en $\mathcal{L}$ también se dirá que T respeta $\mathcal{L}$

| Definición 2.2. Una implicación $Y_1 \rightarrow Y_2$ es **válida** en un contexto (O, A, I) si para todo $o \in O$, o' respeta la implicación;

$$\forall o \in O, o' \text{ respeta } Y_1 \rightarrow Y_2$$

Se denotará como: $(O, A, I) \models Y_1 \rightarrow Y_2$

Nota 2.3. Esto es equivalente a decir que la intención o' de cada objeto o respeta la implicación, según la semántica de lógica proposicional: $Y_1 \not\subseteq o' \vee Y_2 \subseteq o'$.

Ejemplos 2.2. En el Ejemplo 1.1:

- $(O, A, I) \models \{\text{Selva}\} \rightarrow \{\text{Mamifero}\}$
- $(O, A, I) \not\models \{\text{Pez}\} \rightarrow \{\text{Mamifero}\}$. Ya que Carpa tiene el atributo Pez pero no Mamifero.

| Definición 2.3 (Cláusula de Horn de primer orden). Una **cláusula de Horn de primer orden** es una fórmula lógica en la forma:

$$P_1(x) \wedge P_2(x) \wedge \dots \wedge P_n(x) \Rightarrow Q(x)$$

donde $P_1, \dots, P_n, Q$ son predicados unarios, y x es una variable. Estas cláusulas son fundamentales en lógica computacional y corresponden a las implicaciones utilizadas en el AFC, ya que modelan reglas que relacionan atributos.

Ejemplos 2.3. Para la implicación $\{\text{Selva}\} \rightarrow \{\text{Mamifero}\}$ se escribiría en primer orden como:

$$\forall x(\text{Selva}(x) \rightarrow \text{Mamifero}(x))$$

o en cláusulas de Horn

$$\text{Selva}(x) \rightarrow \text{Mamifero}(x)$$

Proposición 2.1. Sea $Y_2 = \{a_1, a_2, \dots, a_n\} \subseteq A$. Entonces:

$$(O, A, I) \models Y_1 \rightarrow Y_2 \quad \Leftrightarrow \quad (O, A, I) \models Y_1 \rightarrow \{a_i\} \quad \text{para todo } 1 \leq i \leq n$$

Demostración. $(\Rightarrow)$ Supongamos que $(O, A, I) \models Y_1 \rightarrow Y_2$. Entonces, por definición de implicación válida, para todo $o \in O$, si $Y_1 \subseteq o'$, entonces $Y_2 \subseteq o'$. En particular, cada $a_i \in Y_2$ también estará en o' . Por tanto, $(O, A, I) \models Y_1 \rightarrow \{a_i\}$ para todo i .

($\Leftarrow$) Supongamos ahora que $(O, A, I) \models Y_1 \rightarrow \{a_i\}$ para todo $i = 1, \dots, n$. Entonces, para todo objeto $o \in O$, si $Y_1 \subseteq o'$, se cumple que $a_i \in o'$ para todo i , es decir, $Y_2 \subseteq o'$. Por tanto, $(O, A, I) \models Y_1 \rightarrow Y_2$.

Proposición 2.2. La implicación $Y_1 \rightarrow Y_2$ es válida en (O, A, I) si y solo si:

$$Y_2 \subseteq Y_1''$$

Demostración. ($\Rightarrow$) Supongamos que $(O, A, I) \models Y_1 \rightarrow Y_2$. Entonces, para todo $o \in O$, si $Y_1 \subseteq o'$, se cumple que $Y_2 \subseteq o'$. Esto implica que todo objeto que contiene Y_1 también contiene Y_2 , es decir:

$$Y_1' \subseteq a' \quad \text{para todo } a \in Y_2$$

Aplicando doble derivación:

$$Y_2 \subseteq \bigcap_{o \in Y_1'} o' = Y_1''$$

($\Leftarrow$) Supongamos ahora que $Y_2 \subseteq Y_1''$. Entonces, para todo $o \in Y_1'$, se cumple $Y_2 \subseteq o'$. Como $Y_1' = \{o \in O : Y_1 \subseteq o'\}$, esto implica que para cualquier $o \in O$ tal que $Y_1 \subseteq o'$, también se cumple $Y_2 \subseteq o'$. Por tanto, $(O, A, I) \models Y_1 \rightarrow Y_2$.

2.2 Bases de implicaciones

En general, un contexto formal puede implicar muchas relaciones válidas entre atributos. Nuestro objetivo es encontrar un conjunto (**finito y manejable**) de implicaciones que nos permita deducir todas las válidas. A este conjunto lo llamamos **base de implicaciones**.

Definición 2.4 (Consecuencia lógica). Dado un conjunto de implicaciones $\mathcal{L}$ y una implicación $Y \rightarrow Z$, se dice que $Y \rightarrow Z$ es **consecuencia lógica** de $\mathcal{L}$ y se escribe:

$$\mathcal{L} \models Y \rightarrow Z$$

si y solo si, para todo $T \subseteq A$, si T respeta $\mathcal{L}$, entonces también respeta $Y \rightarrow Z$.

| Definición 2.5 (Base). Sea $\mathcal{L}$ un conjunto de implicaciones. Decimos que $\mathcal{L}$ es una **base de implicaciones** del contexto (O, A, I) si cumple:

- **Completitud:** Toda implicación válida en (O, A, I) es consecuencia lógica de $\mathcal{L}$. Es decir, si $(O, A, I) \models Y \rightarrow Z$, entonces $\mathcal{L} \models Y \rightarrow Z$.
- **Irredundancia:** Ninguna implicación en $\mathcal{L}$ puede deducirse a partir del resto. Es decir, para toda $Y \rightarrow Z \in \mathcal{L}$, se cumple que $\mathcal{L} \setminus \{Y \rightarrow Z\} \not\models Y \rightarrow Z$.

Una base completa e irredundante proporciona una representación óptima del conocimiento extraíble desde un contexto. Es decir, todas las implicaciones válidas pueden obtenerse a partir de la base, y ninguna de sus reglas es prescindible.

2.3 Razonamiento con la base: cálculo lógico

Para deducir nuevas implicaciones a partir de una base, se utilizan las **reglas de Armstrong**, conocidas también en bases de datos.

| Definición 2.6 (Reglas de Armstrong). Sea $X, Y, Z, W \subseteq A$ conjuntos de atributos. Las siguientes reglas permiten deducir implicaciones:

1. **Reflexividad:** $X \rightarrow X$
2. **Aumento:** Si $X \rightarrow Y$, entonces $X \cup Z \rightarrow Y$
3. **Transitividad:** Si $X \rightarrow Y$ y $Y \cup Z \rightarrow W$, entonces $X \cup Z \rightarrow W$

| Teorema 2.1. El cálculo de Armstrong es **completo y correcto** respecto al razonamiento con bases:

$$\mathcal{L} \models Y \rightarrow Z \quad \Leftrightarrow \quad \mathcal{L} \vdash Y \rightarrow Z$$

donde $\mathcal{L} \vdash Y \rightarrow Z$ significa que $Y \rightarrow Z$ puede deducirse formalmente mediante reglas de Armstrong a partir de $\mathcal{L}$.

| Teorema 2.2. Si $\mathcal{L}$ es una base para un contexto formal M , entonces $\mathcal{L}$ es una teoría implicacionalmente completa para el contexto, es decir,

$$\mathcal{L} \vdash Y \rightarrow Z \quad \Leftrightarrow \quad M \models Y \rightarrow Z$$

2.4 Aplicación: cierre implicacional

Una herramienta esencial en el razonamiento con bases es el **cierre implicacional**.

| Definición 2.7 (Cierre implicacional). Sea $L = \{Y_i \rightarrow Z_i\}_{i=1}^n$ un conjunto de implicaciones. Dado $P \subseteq A$, se define su **cierre implicacional** respecto de L , denotado $L(P)$, como el menor conjunto $Q \supseteq P$ tal que:

- Para todo i , si $Y_i \subseteq Q$ entonces $Z_i \subseteq Q$

| Teorema 2.3. Una implicación $I \rightarrow J$ se deduce de una base L si y solo si:

$$J \subseteq L(I)$$

Ejemplos 2.4. Sea la base:

$$L = \{\emptyset \rightarrow \{N\}, \{N, A\} \rightarrow \{M\}, \{N, P\} \rightarrow \{M\}\}$$

Entonces:

$$L(\{A\}) = \{A, N, M\} \Rightarrow L \models \{A\} \rightarrow \{M\}$$

pero

$$P \notin L(\{A\}) \Rightarrow L \not\models \{A\} \rightarrow \{P\}$$

2.5 Pseudo-intenciones y la base Stem

El número total de implicaciones válidas en un contexto puede ser muy elevado. La base Stem ofrece una manera eficiente de representar todas estas implicaciones mediante un subconjunto mínimo e irredundante, que puede calcularse a partir de las llamadas **pseudo-intenciones**.

| Definición 2.8 (Pseudo-intención). Un conjunto $P \subseteq A$ se llama **pseudo-intención** si cumple:

- $P \neq P''$.
- Para toda pseudo-intención $Q \subsetneq P$, se tiene que $Q'' \subseteq P$.

Ejemplos 2.5. Dado el contexto de los seres vivos del Ejemplo 1.1:

- $\emptyset'' = A' = \emptyset$, por lo que $\emptyset$ es una pseudo-intención.
- $\{\text{Selva}\}'' = \{L, Ce\}' = \{\text{Selva}, \text{Mamifero}\}$, por lo que $\{\text{Selva}\}$ no es una pseudo-intención.

| Teorema 2.4. *El conjunto de implicaciones*

$$\mathcal{L} := \{P \rightarrow P'' \mid P \text{ es una pseudo-intención}\}$$

es completo e irredundante, o sea, que es una base.

Demostración. Cada implicación $P \rightarrow P'' \in \mathcal{L}$ es válida en (O, A, I) por la Proposición 2.2. Ya que si un conjunto de objetos posee los atributos de la pseudo-intención P , entonces al calcular P'' estamos obteniendo todos los atributos en su totalidad que cumplen dichos objetos, por lo que se cumpliría dicha implicación $P \rightarrow P''$.

Para demostrar que $\mathcal{L}$ es completo, debemos probar que todo conjunto $T \subseteq A$ que respeta $\mathcal{L}$ es una intención. Cada conjunto T de este tipo, en particular, respeta todas las implicaciones $Q \rightarrow Q''$ donde Q es una pseudo-intención y $Q \subseteq T$. Si asumimos que $T \neq T''$, entonces T cumple con la definición de pseudo-intención, y por tanto la implicación $T \rightarrow T''$ está en $\mathcal{L}$, pero no es respetada por T , lo cual es una contradicción.

Para demostrar que $\mathcal{L}$ es irredundante, consideramos una pseudo-intención arbitraria P y veamos que P respeta $\mathcal{L} \setminus \{P \rightarrow P''\}$. De hecho, si $Q \rightarrow Q''$ es una implicación en $\mathcal{L} \setminus \{P \rightarrow P''\}$ tal que $Q \subseteq P$, entonces debe cumplirse que $Q'' \subseteq P$, ya que P es una pseudo-intención. Pero P no respeta $\{P \rightarrow P''\}$ ya que $P \subseteq P$ y $P'' \not\subseteq P$. Por lo que $\mathcal{L} \setminus \{P \rightarrow P''\} \not\models P \rightarrow P''$. Como para cada implicación hay una pseudo-intención asociada, entonces $\mathcal{L}$ es irredundante. **|**

Estas implicaciones constituyen la **base Stem** del contexto. El conjunto de todas ellas es una base completa e irredundante de implicaciones válidas.

| Definición 2.9 (Base Stem (Duquenne-Guigues)). *Sea (O, A, I) un contexto formal. El conjunto de todas las implicaciones:*

$$\{P \rightarrow P'' : P \text{ es pseudo-intención}\}$$

*se denomina **base Stem**.*

Nota 2.4. En la práctica, se toman las implicaciones:

$$P \rightarrow P'' \setminus P$$

Pues $(O, A, I) \models P \rightarrow P$.

Nota 2.5. A las implicaciones de la base Stem se las denomina **reglas de asociación estrictas**.

| Teorema 2.5. A partir de la base de Stem para las reglas de asociación estrictas uno puede obtener todas la reglas de asociación exactas válidas en nuestro contexto, usando las Reglas de Armstrong definidas en 2.6.

Demostración. La demostración se obtiene de resultados de [9] y [8]. **|**

Ejemplos 2.6. Supongamos que en un contexto se tiene la pseudo-intención $P = \{N, A\}$ y que $P'' = \{N, A, M\}$. Entonces se genera la implicación:

$$\{N, A\} \rightarrow \{M\}$$

que pertenece a la base Stem del contexto.

Para calcular la base Stem tenemos que calcular todas las pseudo-intenciones del contexto. Las pseudo-intenciones tienen una definición recursiva, ya que para ver que un conjunto de atributos P es una pseudo-intención hay que comprobar también que para toda pseudo-intención $Q \subsetneq P$, se tiene que $Q'' \subseteq P$, o sea, que habría que tener en cuenta las posibles pseudo-intenciones estrictamente contenidas en P . Por eso nuestro algoritmo deberá ir calculando las pseudo-intenciones de menor a mayor número de elementos, ya que así nos aseguraremos de poder comprobar esta segunda propiedad de la definición de pseudo-intención (2.8). El algoritmo sería el descrito en **Algorithm 2 Cálculo de la base Stem (Usando la definición) (2)**.

Algorithm 2 Cálculo de la base Stem (Usando la definición)

```

1: Entrada: Contexto formal  $(O, A, I)$ 
2: Salida: Base Stem (conjunto de implicaciones  $P \rightarrow P'' \setminus P$ )
3: Base  $\leftarrow \emptyset$ 
4:  $\mathcal{P} \leftarrow \emptyset$  ▷ Conjunto de pseudo-intenciones
5: for all  $P \subseteq A$  en orden creciente de número de elementos do
6:   Calcular el cierre  $P''$ 
7:   if  $P \neq P''$  then
8:     if para todo  $Q \in \mathcal{P}$  con  $Q \subsetneq P$  se cumple que  $Q'' \subseteq P$  then
9:       Añadir implicación  $P \rightarrow P'' \setminus P$  a Base
10:      Añadir  $P$  a  $\mathcal{P}$ 
11:     end if
12:   end if
13: end for

```

El anterior Algoritmo 2 es muy ineficiente cuando trabajamos con contextos con un alto número de atributos, ya que estamos recorriendo el espacio de los subconjuntos de A . Esto conlleva una complejidad de orden exponencial con respecto al número de atributos ($2^{|A|}$). Por lo que hay que buscar **mejores optimizaciones**. Los siguientes algoritmos que mostraré pueden ser encontrados en [9].

Alg 3. **Next Closure**($C, A, \mathcal{L}$): se utiliza para enumerar todos los conjuntos cerrados (o clausuras) de un conjunto finito ordenado, en orden lexico-gráfico, como es el caso de A . Es clave para el Análisis Formal de Conceptos ya que entre otras utilidades permite la construcción de **bases canónicas de implicaciones**, como nuestra base Stem.

Algorithm 3 Next Closure($C, A, \mathcal{L}$)

```

1: Entrada: Un conjunto de implicaciones  $\mathcal{L}$  sobre un conjunto finito linealmente
   ordenado  $A$  y un subconjunto  $C \subseteq A$ 
2: Salida: El siguiente conjunto cerrado lexico-gráficamente después de  $C$ , si existe;
    $\perp$  en caso contrario
3: for all  $a \in A$  en orden inverso do
4:   if  $a \in C$  then
5:      $C \leftarrow C \setminus \{a\}$ 
6:   else
7:      $D \leftarrow \mathcal{L}(C \cup \{a\})$  ▷ Cierre implicacional
8:     if  $D \setminus C$  no contiene ningún elemento  $< a$  then
9:       return  $D$ 
10:    end if
11:  end if
12: end for
13: return  $\perp$ 

```

Alg 4. **Base Stem (optimizado)**: calcula de manera eficiente la base mínima de implicaciones (Duquenne-Guigues). Utiliza el algoritmo **Next Closure** para recorrer todos los subconjuntos cerrados en orden lexico-gráfico sin repeticiones, y solo añade implicaciones cuando realmente son necesarias. Es un método óptimo y sistemático, ampliamente usado en Análisis Formal de Conceptos por su capacidad de representar el conocimiento de forma que no contenga redundancias y sea compacto.

Algorithm 4 Cálculo de la base Stem (optimizado)

```

1: Entrada: Un conjunto de implicaciones  $\mathcal{L}$  sobre un conjunto finito  $A$ , dado por
   ejemplo a partir de un contexto formal  $(O, A, I)$ .
2: Salida: La base canónica del operador de clausura.
3:  $P \leftarrow \emptyset$ 
4:  $\text{Base} \leftarrow \emptyset$ 
5: while  $P \neq A$  do
6:   if  $P \neq P''$  then
7:      $\text{Base} \leftarrow \text{Base} \cup \{P \rightarrow P''\}$   $\triangleright$  O también  $\{P \rightarrow P'' \setminus P\}$ 
8:   end if
9:    $P \leftarrow \text{NextClosure}(P, A, \mathcal{L})$ 
10: end while
11: return  $\mathcal{L}$ 

```

	Nec. agua	Acuático	Movilidad	Patas
Gato	x		x	x
Sanguijuela	x	x	x	
Rana	x	x	x	x
Maiz	x			
Pez	x	x	x	

Cuadro 2.1: Tabla de otro ejemplo de animales.

Ejemplos 2.7. Sea el contexto $M = (O, A, I)$ dado por el Cuadro 2.1, vamos a calcular su base Stem usando la definición. Primero veamos cuales son todas las pseudo-intenciones siguiendo el algoritmo:

- $\emptyset'' = O' = \{N\}$, luego $\emptyset \rightarrow \{N\}$.
- $\{N\}'' = O' = \{N\}$ no lo es.
- $\{A\}'' = \{N, A, M\}$, pero $\emptyset \subsetneq \{A\}$ y $\emptyset'' \not\subseteq \{A\}$. De aquí podemos observar que el conjunto que no contenga a N no podrá ser pseudo-intención.
- $\{N, A\}'' = \{N, A, M\}$, luego $\{N, A\} \rightarrow \{M\}$.
- $\{N, M\}'' = \{N, M\}$ no lo es.
- $\{N, P\}'' = \{N, P, M\}$, luego $\{N, P\} \rightarrow \{M\}$.
- $\{N, A, M\}'' = \{N, A, M\}$ no lo es.
- $\{N, A, P\}'' = \{N, A, M, P\}$. No contiene a $\{N, P\}''$.
- $\{N, M, P\}'' = \{N, M, P\}$ no lo es.
- $\{N, A, M, P\}'' = \{N, A, M, P\}$ no lo es.

La **base Stem** sería:

$$\{\emptyset \rightarrow \{N\}, \{N, A\} \rightarrow \{M\}, \{N, P\} \rightarrow \{M\}\}$$

Y considerándolas como cláusulas de Horn de primer orden:

$$\Delta = \begin{cases} N(x) \\ N(x) \wedge A(x) \rightarrow M(x) \\ N(x) \wedge P(x) \rightarrow M(x) \end{cases}$$

2.6 Bases de Luxenburger

En esta sección desarrollaremos el concepto clave para este trabajo, el que usaremos como principal herramienta para implementar nuestro algoritmo de IA usando estas bases.

Las **bases de Luxenburger** permiten describir conjunto de implicaciones "casi" válidas en un contexto formal, considerando medidas como el soporte y la confianza, como las reglas de asociación clásicas en minería de datos. Esta noción extiende el análisis formal de conceptos hacia marcos de trabajo más flexibles, donde no todas las implicaciones se cumplen estrictamente.

En toda la sección estaremos trabajando con un contexto formal $M = (O, A, I)$, donde O es el conjunto de objetos, A el conjunto de atributos e $I \subseteq O \times A$ la relación de incidencia.

Conjuntos cerrados y soporte de atributos

| Definición 2.10. Un conjunto de atributos $Y \subseteq A$ se dice **cerrado** si $Y = Y''$.

Proposición 2.3. Un conjunto de atributos $Y \subseteq A$ es **cerrado** si y sólo si es la **intención** de un concepto formal.

Demostración. Recordemos que un conjunto de atributos Y es **cerrado** si $Y = Y''$, y que es la **intención** de un concepto si existe un conjunto de objetos $X \subseteq O$ tal que (X, Y) es un concepto formal, es decir, $X' = Y$ y $Y' = X$.

($\Rightarrow$) Supongamos que Y es cerrado. Entonces $Y = Y''$. Tomamos $X := Y'$. Aplicando la propiedad de doble derivación, tenemos:

$$X' = (Y')' = Y'' = Y.$$

Así, $(X, Y) = (Y', Y)$ es un concepto formal, por lo tanto Y es la intención de un concepto.

($\Leftarrow$) Supongamos que Y es la intención de un concepto. Entonces existe $X \subseteq O$ tal que (X, Y) es un concepto formal, es decir, $X' = Y$ y $Y' = X$. Aplicamos doble derivación:

$$Y'' = (Y')' = X' = Y.$$

Luego, $Y = Y''$, por lo que Y es cerrado.

Por lo tanto, Y es cerrado si y sólo si es la intención de un concepto formal del contexto. |

Ejemplos 2.8. Según el contexto del Ejemplo 1.1:

- $\{\text{Pez}\}$ es cerrado ya que $\{\text{Pez}\}'' = \{Ca\} = \{\text{Pez}\}$.
- $\{\text{Selva, Planta}\}$ no es cerrado ya que $\{\text{Selva, Planta}\}'' = \emptyset' = A$.

| **Definición 2.11.** El **soporte** de un conjunto de atributos Y se define como:

$$\text{supp}(Y) = \frac{|Y'|}{|O|}$$

Es decir, la proporción de objetos que poseen todos los atributos de Y .

Ejemplos 2.9. Según el contexto del Ejemplo 1.1:

- $\text{supp}(\{\text{Agua, Pez}\}) = \frac{|\{\text{Agua, Pez}\}'|}{|O|} = \frac{|\{Ca\}|}{|O|} = \frac{1}{6}.$
- $\text{supp}(\{\text{Mamifero}\}) = \frac{|\{\text{Mamifero}\}'|}{|O|} = \frac{|\{L, D, Os, Ce\}|}{|O|} = \frac{4}{6}.$

| **Definición 2.12.** Fijado un umbral $\delta \in [0, 1]$, un concepto $C = (X, Y)$ es δ -frecuente si:

$$\frac{|X|}{|O|} \geq \delta$$

Es decir, si la proporción de objetos.

Ejemplos 2.10. Veamos si algunos de los conceptos del Ejemplo 1.1 calculados en el Ejemplo 1.7 son δ -frecuente para $\delta = \frac{3}{6}$:

- Para $(\emptyset, A)$, $\frac{|\emptyset|}{|O|} = \frac{0}{6} \not\geq \frac{3}{6}$. No es δ -frecuente.
- Para $(\{L, D, Os, Ce\}, \{\text{Mamifero}\})$, $\frac{|\{L, D, Os, Ce\}|}{|O|} = \frac{4}{6} \geq \frac{3}{6}$. Es δ -frecuente.
- Para $(\{L, Ce\}, \{\text{Selva}, \text{Mamifero}\})$, $\frac{|\{L, Ce\}|}{|O|} = \frac{2}{6} \not\geq \frac{3}{6}$. No es δ -frecuente.
- Para $(\{Os\}, \{\text{Bosque}, \text{Mamifero}\})$, $\frac{|\{Os\}|}{|O|} = \frac{1}{6} \not\geq \frac{3}{6}$. No es δ -frecuente.

Definición 2.13. Dado un par de conjuntos de atributos **cerrados** Y_1, Y_2 , escribimos $Y_1 < Y_2$ si $Y_1 \subset Y_2$ y no existe ningún conjunto cerrado Y tal que $Y_1 \subset Y \subset Y_2$.

A esta relación se le denomina **orden inmediato** o **relación de cobertura** entre conjuntos cerrados.

Nota 2.6. Es sencillo encontrar dos conjuntos de atributos cerrados $Y_1, Y_2 \subseteq A$ que estén en relación de cobertura $Y_1 < Y_2$ por medio del diagrama de Hasse.

Como sabemos que un conjunto de atributos cerrado coincide con la intención de algún concepto, entonces sea $C_1 = (Y'_1, Y_1)$ y $C_2 = (Y'_2, Y_2)$. Si miramos el diagrama de Hasse del retículo podemos ver que estarán en relación de cobertura si existe una arista entre los conceptos asociados que vaya del concepto C_2 al C_1 .

Confianza y soporte de implicaciones

Sea una implicación $L \equiv Y_1 \rightarrow Y_2$ que, como hemos visto en secciones anteriores, representa que todo objeto que posee los atributos de Y_1 también posee los de Y_2 .

Definición 2.14 (Confianza de una implicación). La **confianza** de la implicación es:

$$\text{conf}(L) = \frac{|(Y_1 \cup Y_2)'|}{|Y'_1|}$$

Es decir, la proporción de objetos que cumplen el antecedente y también el consecuente.

Definición 2.15 (Soporte de una implicación). El **soporte** de la implicación es:

$$\text{supp}(L) = \frac{|(Y_1 \cup Y_2)'|}{|O|}$$

Es decir, la proporción de objetos que satisfacen efectivamente la implicación.

Base de Luxenburger

| Definición 2.16. *Dados unos umbrales γ y δ de confianza y soporte respectivamente, la **base de Luxenburger** del contexto M se define como:*

$$\mathcal{L}(M, \gamma, \delta) = \left\{ Y_1 \rightarrow Y_2 \mid \begin{array}{l} Y_1, Y_2 \text{ cerrados, } Y_1 < Y_2, \\ \text{conf}(Y_1 \rightarrow Y_2) \geq \gamma, \text{ supp}(Y_1 \rightarrow Y_2) \geq \delta \end{array} \right\}$$

Algorithm 5 Cálculo de la base de Luxenburger

```

1: Entrada: Contexto formal  $(O, A, I)$  con su retículo de conceptos, umbrales de
   soporte y confianza.
2: Salida: La base de Luxenburger asociada al contexto formal con dichos umbrales.
3:  $\text{Base}_{\text{Lux}} \leftarrow \emptyset$ 
4:  $\text{Visitados} \leftarrow \emptyset$ 
5:  $n \leftarrow |O|$ 
6: procedure RECORRER(concepto)
7:   if concepto  $\in$  Visitados then
8:     return
9:   end if
10:  Añadir concepto a Visitados
11:  for all vecino  $\in$  concepto.lower_neighbors do
12:     $A \leftarrow$  intención de concepto
13:     $B \leftarrow$  intención de vecino
14:    implicación  $\leftarrow A \rightarrow B \setminus A$ 
15:     $\text{conf} \leftarrow \frac{|\text{extensión de vecino}|}{|\text{extensión de concepto}|}$ 
16:     $\text{sop} \leftarrow \frac{|\text{extensión de vecino}|}{n}$ 
17:    if  $\text{sop} \geq$  soporte y  $\text{conf} \geq$  confianza then
18:      Añadir (implicación, conf, sop) a  $\text{Base}_{\text{Lux}}$ 
19:    else if  $\text{sop} <$  soporte then
20:      return
21:    end if
22:    RECORRER(vecino)
23:  end for
24: end procedure
25: RECORRER(concepto superior del retículo)
26: return  $\text{Base}_{\text{Lux}}$ 

```

El Algoritmo 5 calcula la **base de Luxenburger**. Este recorre el retículo desde el concepto superior hacia abajo. Genera una implicación para cada par de conceptos vecinos, siempre que la implicación sea lo suficientemente representativa según los umbrales dados.

Nota 2.7. En realidad, basta con implicaciones del tipo $Y_1 \rightarrow Y_2 \setminus Y_1$.

Nota 2.8. A las implicaciones de la base de Luxenburger se las denomina **reglas de asociación aproximadas**.

Las **reglas de asociación aproximadas** de la base Luxenburger se complementan con las **reglas de asociación estrictas** (base Stem) para formar la **base completa de reglas de asociación**:

Definición 2.17 (Base completa de reglas de asociación). *Fijados los umbrales de confianza γ y de soporte δ , la base completa de reglas de asociación de un contexto M está formada por la **base Stem** junto con la **base de Luxenburger** correspondiente.*

Se denotará como $B_{asoc} = (M, \gamma, \delta)$.

Ejemplos 2.11. Ejemplo de cálculo de la base completa de reglas de asociación con confianza 0,67 y soporte 0,4 del contexto M , dado por el Cuadro 2.2:

	A	B	C	D	E
1	x		x	x	
2		x	x		x
3	x	x	x		x
4		x			x
5	x	x	x		x

Cuadro 2.2: Contexto de ejemplo para la base de reglas de asociación.

El diagrama de Hasse del retículo quedaría como se muestra en la Figura 2.1

■ **Implicaciones de la base Stem:**

1. $L_1 : \{A\} \rightarrow \{B\}$.
2. $L_2 : \{B\} \rightarrow \{E\}$.
3. $L_3 : \{D\} \rightarrow \{A, C\}$.
4. $L_4 : \{E\} \rightarrow \{B\}$.

■ **Implicaciones de la base de Luxenburger con umbrales mínimos:**

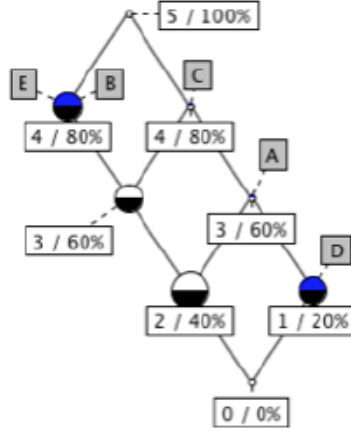


Figura 2.1: Diagrama Hasse del retículo del Ejemplo 2.11.

5. $L_5 : \emptyset \rightarrow \{B, E\}$, donde $\text{conf}(L_5) = 0.80$ y $\text{supp}(L_5) = 0.80$.
6. $L_6 : \emptyset \rightarrow \{C\}$, donde $\text{conf}(L_6) = 0.80$ y $\text{supp}(L_6) = 0.80$.
7. $L_7 : \{B, E\} \rightarrow \{C\}$, donde $\text{conf}(L_7) = 0.75$ y $\text{supp}(L_7) = 0.60$.
8. $L_8 : \{C\} \rightarrow \{B, E\}$, donde $\text{conf}(L_8) = 0.75$ y $\text{supp}(L_8) = 0.60$.
9. $L_9 : \{C\} \rightarrow \{A\}$, donde $\text{conf}(L_9) = 0.75$ y $\text{supp}(L_9) = 0.60$.
10. $L_{10} : \{B, C, E\} \rightarrow \{A\}$, donde $\text{conf}(L_{10}) = 0.67$ y $\text{supp}(L_{10}) = 0.40$.
11. $L_{11} : \{A, C\} \rightarrow \{B, E\}$, donde $\text{conf}(L_{11}) = 0.67$ y $\text{supp}(L_{11}) = 0.40$.
12. $L_{12} : \{A, C\} \rightarrow \{D\}$, donde $\text{conf}(L_{12}) = 0.33$ y $\text{supp}(L_{12}) = 0.20$.
13. $L_{13} : \{A, B, C, E\} \rightarrow \{D\}$, donde $\text{conf}(L_{13}) = 0$ y $\text{supp}(L_{13}) = 0$.
14. $L_{14} : \{A, C, D\} \rightarrow \{B, E\}$, donde $\text{conf}(L_{14}) = 0$ y $\text{supp}(L_{14}) = 0$.

Si queremos que cumpla la confianza y el soporte mínimos, entonces la base completa de reglas de asociación sería:

$$\mathcal{B}_{\text{asoc}}(M, 0.67, 0.4) = \{L_1, L_2, L_3, L_4, L_5, L_6, L_7, L_8, L_9, L_{10}, L_{11}\}$$

Se quedarían fuera las implicaciones L_{12} , L_{13} y L_{14} por tener la confianza o el soporte por debajo de los mínimos.

Razonamiento sobre reglas de asociación

Según el artículo [22], existen propiedades que permiten inferir nuevas reglas de asociación a partir de una base dada, respetando sus valores de soporte y confianza:

Lema 2.1. Las reglas 1 y 2 son válidas para $\varphi \in \{\text{conf}, \text{supp}\}$.

1. $\varphi(X \rightarrow Y) = \varphi(X \rightarrow Y \setminus Z)$, para todo $Z \subseteq X \subseteq A, Y \subseteq A$.
2. $\varphi(X'' \rightarrow Y'') = \varphi(X \rightarrow Y)$, para todo $X, Y \subseteq A$.
3. Si $\text{conf}(X \rightarrow Y) = p$ y $\text{conf}(Y \rightarrow Z) = q$, entonces:

$$\text{conf}(X \rightarrow Z) = p \cdot q,$$

para todos los conjuntos de intenciones de conceptos frecuentes $X \subset Y \subset Z$.

- 3'. $\text{supp}(X \rightarrow Z) = \text{supp}(Y \rightarrow Z)$, para todo $X, Y \subseteq Z$.
4. $\text{conf}(X \rightarrow X) = 1$, para todo $X \subseteq A$.

Demostración. Las demostraciones para la confianza están dadas en [15].

1. $\text{supp}(X \rightarrow Y) = \text{supp}(X \rightarrow Y \setminus Z)$ se obtiene ya que $X \cup Y = X \cup (Y \setminus Z)$ y por la definición de soporte 2.15.
2. $\text{supp}(X'' \rightarrow Y'') = \text{supp}(X \rightarrow Y)$ se obtiene porque $(X'' \cup Y'')' = X''' \cap Y''' = X' \cap Y' = (X \cup Y)'$.
- 3' $\text{supp}(X \rightarrow Z) = \frac{|(X \cup Z)'|}{|O|} = \frac{|Z'|}{|O|} = \frac{|(Y \cup Z)'|}{|O|} = \text{supp}(Y \rightarrow Z)$

Nota 2.9. Estas se pueden usar como reglas de inferencia sobre las reglas de asociación (implicación, confianza, soporte).

Teorema de completitud de Luxenburger

La demostración la obtenemos a partir del artículo [22].

Teorema 2.6. A partir de la base de Luxenburger para reglas de asociación aproximadas uno puede derivar todas las reglas de asociación no válidas en el contexto junto a sus soportes y sus confianzas usando las reglas dadas en el Lema 2.1. Además, $\mathcal{L}(M, \gamma, \delta)$ es minimal (con respecto a la inclusión de conjuntos) con esta propiedad.

Demostración. Para determinar si una regla de asociación $L := I \rightarrow J$ se cumple en un contexto (y para determinar su soporte y su confianza) se puede considerar la regla $I'' \rightarrow J''$ que tiene (por las Reglas 1 y 2) el mismo soporte y la misma confianza.

- Si $I'' = J''$, entonces $\text{conf}(L) = 1$ y $\text{supp}(L) = \text{supp}(I'')$.

- Si $I'' \neq J''$, entonces existe un camino de reglas aproximadas, es decir, existen conjuntos de atributos cerrados $I_1, \dots, I_n$ con $I_i \rightarrow I_{i+1} \in \mathcal{L}(M, \gamma, \delta)$ y $I'' = I_1$ y $I_n = J''$. El soporte y la confianza de L se pueden determinar ahora por $\text{supp}(L) = \text{supp}(I_n)$ (Regla 3') y $\text{conf}(L) = \prod_{i=1}^{n-1} \text{conf}(I_i \rightarrow I_{i+1})$ (Regla 3).

Ahora mostramos la minimalidad de $\mathcal{L}(M, \gamma, \delta)$. Sea $L := I \rightarrow J \in \mathcal{L}(M, \gamma, \delta)$. Mostramos que la confianza de L no se puede derivar de $\mathcal{L}(M, \gamma, \delta) \setminus \{L\}$ aplicando las reglas del Lema 2.1.

- La Regla 1 no se puede aplicar hacia adelante ya que J ya contiene I . No se puede aplicar hacia atrás debido a que $I < J$.
- La Regla 2 no se puede aplicar hacia adelante ya que $I = I''$ y $J = J''$. No se puede aplicar hacia atrás ya que $\mathcal{L}(M, \gamma, \delta)$ contiene solo reglas con antecedente cerrado y consecuente cerrado.
- La Regla 3 no se puede aplicar ya que no hay $K \subsetneq A$ con $I \rightarrow K \in \mathcal{L}(M, \gamma, \delta) \setminus \{L\}$ y $K \rightarrow J \in \mathcal{L}(M, \gamma, \delta) \setminus \{L\}$ (debido a $I < J$).
- La Regla 4 no se puede aplicar ya que $I \neq J$.

I

3 | AFC como Herramienta en Machine Learning

Para la redacción de este capítulo, se han utilizado como base ideas de las referencias [11] y [18].

En este capítulo nos centraremos en explicar las virtudes del Análisis Formal de Conceptos en las que nos basaremos para usarlas a nuestro favor como un modelo de **Machine Learning**, entendida como la rama de la IA que se centra en el desarrollo de algoritmos y modelos matemáticos que permiten a las máquinas aprender de los datos, mejorar su rendimiento con el tiempo y (ayudar a) tomar decisiones sin ser programadas explícitamente para hacerlo. En lugar de seguir instrucciones fijas, un sistema de Machine Learning ajusta su comportamiento en función de patrones y tendencias observadas en los datos.

En términos más simples, lo que se busca es la capacidad de una máquina o sistema para adaptarse y mejorar a partir de la experiencia, lo que en nuestro caso se traduce en utilizar estructuras formales derivadas del contexto de los datos para descubrir relaciones. A partir de estas relaciones, que se manifestarán en forma de implicaciones entre atributos, podremos aprender sobre el comportamiento subyacente del sistema, identificar patrones recurrentes, detectar atributos irrelevantes o redundantes, y, en algunos casos, realizar predicciones sobre nuevos datos.

De este modo, el AFC nos ofrece un enfoque lógico y estructurado que puede integrarse como modelo de aprendizaje automático. A través del uso de implicaciones formales, el objetivo es aprovechar el conocimiento implícito en los datos para facilitar tareas propias del Machine Learning, como la predicción de atributos o la generación de reglas interpretables.

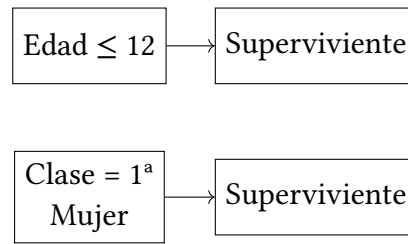


Figura 3.1: Posibles implicaciones extraídas del contexto del Titanic.

3.1 Generación de conocimiento a partir de implicaciones

El Análisis Formal de Conceptos permite obtener reglas de dependencia entre atributos que pueden considerarse como una forma de conocimiento explícito obtenido directamente de los datos. Estas reglas son expresadas como implicaciones lógicas del tipo $A \rightarrow B$, donde A y B son conjuntos de atributos, y se interpretan como que todo objeto que posee los atributos de A , también posee los de B . En la Figura 3.1 se pueden observar dos ejemplos de posibles implicaciones. La generación de conocimiento a partir de estas reglas es lo que trataremos de usar como herramienta en este trabajo para ver cómo de útil y eficaz pueden llegar a ser.

En el *Capítulo 2* nos hemos centrado en la definición lógica y matemáticas de las implicaciones con su estructura y propiedades. Lo más interesante ha sido la definición, existencia y creación de ciertas bases de implicaciones que cubren, en parte, el conocimiento extraíble de nuestro contexto formal.

Bases de implicaciones

El conjunto de todas las implicaciones que podrían existir en un contexto formal serían $2^{2^{|A|}}$, siendo $|A|$ el número total de atributos. Al ser orden exponencial, en la práctica, para un escaso conjunto de atributos el número de implicaciones se convierte en un conjunto intratable. Por ejemplo:

- Para $|A| = 10$, el conjunto de implicaciones tendría en torno al millón de elementos.

- Para $|A| = 15$, el conjunto de implicaciones ya pasa a tener mil millones de elementos.

Además, hay que tener en cuenta que nuestros atributos son siempre booleanos, ya que los objetos pueden poseer o no cierto atributo. Por lo que complica y empeora este espacio de implicaciones.

Todo esto hace necesaria la búsqueda de bases de dicho espacio de implicaciones, para quedarnos con las más relevantes y que nos den la máxima información posible, reduciendo así considerablemente el tamaño de los conjuntos de trabajo. Esto da pie al estudio de las bases.

La base Stem: conocimiento determinista

La base Stem es lo que se denomina **canónica**, lo que significa que representa de forma única y completa todas las implicaciones válidas del contexto sin contener información redundante. Además, es **minimal** en tamaño, ya que contiene el menor número posible de implicaciones necesarias para describir completamente el conjunto de dependencias del sistema. Esto la convierte en una base muy útil desde el punto de vista computacional.

La base contiene aquellas implicaciones de la forma

$$P \rightarrow P''$$

donde P es un conjunto de atributos que no es cerrado, es decir, $P \neq P''$, y además, P cumple la condición de ser un **pseudo-cierre**. Un pseudo-cierre es un conjunto que no es cerrado pero cuya clausura P'' no se puede obtener como clausura de ningún subconjunto propio de P que tenga la misma clausura. En otras palabras, P es el conjunto más pequeño posible que genera P'' , sin que otro subconjunto más pequeño lo haga.

Ejemplos 3.1. Ejemplos de implicaciones de la base Stem del contexto del Titanic

- Superviviente, Grupo hermanos/padres 0-1, Grupo padres/hijos 1-3, Grupo edad 27-31 $\rightarrow$ Mujer | confianza = 1.00 | soporte = 0.006
- Grupo edad 0-4, Grupo padres/hijos 1-3, Clase 1º $\rightarrow$ Superviviente, Embarque S, Hombre | confianza = 1.00 | soporte = 0.003

- Grupo edad 4-9, Grupo tarifa 12-25 $\rightarrow$ Clase 3° | confianza = 1.00 | soporte = 0.007
- Clase 2°, Grupo edad 40-45 $\rightarrow$ Embarque S | confianza = 1.00 | soporte = 0.015

La utilidad de esta base es que permite obtener cualquier implicación válida del contexto de manera **no ambigua** y con una **alta eficiencia computacional**. Gracias a su estructura compacta y sin redundancias, se convierte en una muy buena herramienta para la realización de tareas como la simplificación del conocimiento, la generación de reglas interpretables, y la reducción del espacio de búsqueda en algoritmos de razonamiento lógico o aprendizaje automático.

La base de Luxenburger: conocimiento aproximado

A diferencia de la base Stem, que proporciona un conjunto canónico y exacto de implicaciones válidas en un contexto formal, la base de Luxenburger introduce una noción de **conocimiento aproximado**, basada en medidas cuantitativas. Esta base se construye a partir de una relación de confianza (confidence) y soporte (support), conceptos que provienen de la minería de datos y que permiten evaluar cuán frecuentemente se cumple una implicación en el conjunto de datos.

Ejemplos 3.2. Ejemplos de implicaciones de la base Luxenburger del contexto del Titanic que se expondrá en el *Capítulo 4*

- Mujer, Clase 2°, Embarque S $\rightarrow$ Superviviente | confianza = 0.92 | soporte = 0.034
- Mujer, Grupo edad 27-31, Embarque C $\rightarrow$ Superviviente | confianza = 0.92 | soporte = 0.017
- Clase 1°, Embarque S, Grupo edad 22-27 $\rightarrow$ Hombre | confianza = 0.94 | soporte = 0.022
- Grupo edad 13-18, Grupo tarifa 0-12, Hombre $\rightarrow$ Embarque S | confianza = 0.91 | soporte = 0.014

El **soporte** de una implicación $A \rightarrow B$ representa la proporción de objetos del contexto que contienen tanto A como B , es decir, en los que la implicación se verifica tanto antecedente como consecuente.

Mientras tanto, la **confianza** mide la proporción de objetos que contienen A y en los que además también aparece B , es decir, cuán frecuentemente se cumple la implicación entre los casos en los que aparece el antecedente.

Más concretamente, una implicación $A \rightarrow B$ se incluye en la base de Luxenburger si se cumple en una proporción suficientemente alta de los objetos del contexto, según un umbral de confianza predeterminado (por ejemplo, 90 %). Esto implica que, aunque la implicación no se cumpla para todos los objetos (como se exige en el Análisis Formal de Conceptos clásico), sí lo hace en la mayoría de los casos, lo que la convierte en una regla empíricamente válida, parcial o probabilística.

Este enfoque tiene una utilidad especial en escenarios donde los datos pueden ser **parciales, inconsistentes o ruidosos**, situaciones frecuentes en contextos reales. En lugar de desechar relaciones que no son lógicamente necesarias pero que tienen una alta frecuencia, la base de Luxenburger permite capturarlas y utilizarlas como indicios útiles de comportamiento, lo que resulta muy valioso en fases de análisis exploratorio, generación de hipótesis o en sistemas de recomendación.

Además, al priorizar implicaciones frecuentes, esta base al emplearse como herramienta de aprendizaje automático, ayuda a descubrir reglas generales y patrones relevantes que podrían no ser evidentes desde un enfoque puramente determinista. De esta forma, el análisis formal puede abarcar un terreno más flexible, teniendo en cuenta la lógica con **medidas estadísticas**, y permitiendo una representación del conocimiento más fiable ante la incertidumbre.

Base completa: combinación de Stem y Luxenburger

Muchas veces, trabajar solo con implicaciones exactas (como las de la base Stem) o solo con reglas frecuentes (como las de Luxenburger) no es suficiente. Por eso, una buena idea es combinar ambas y construir lo que se llama la **base completa de reglas de asociación**. Esta base mezcla tanto conocimiento determinista como aproximado, y nos da una visión más rica del contexto.

Por un lado, tenemos las reglas de asociación estrictas de la **base Stem**, que son reglas que se cumplen siempre. Nos sirven para deducir información con total seguridad y para reducir atributos de forma lógica, sin perder nada. Por otro lado, la **base de Luxenburger** nos da las reglas de asociación aproximadas que, aunque no se cumplan siempre, se cumplen en la mayoría de los casos. Esto es muy útil cuando los datos están incompletos o son algo ruidosos, porque podemos seguir obteniendo patrones fiables.

Juntar ambas bases nos permite trabajar con una representación del conocimiento mucho más **completa y robusta**. Podemos deducir reglas seguras, pero también

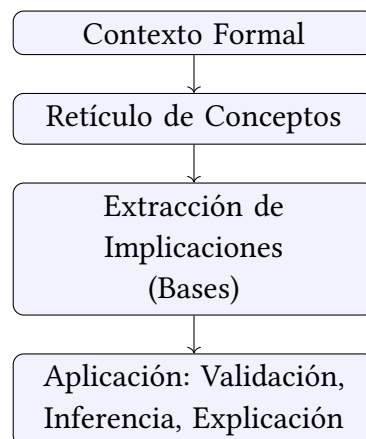


Figura 3.2: Proceso de generación y uso del conocimiento en AFC.

tener en cuenta tendencias generales que ocurren con bastante frecuencia. Esto es especialmente útil si queremos hacer tareas como:

- Entender mejor los datos y ver qué relaciones aparecen casi siempre.
- Tomar decisiones o hacer predicciones con cierto margen de flexibilidad.
- Identificar reglas frecuentes y reglas útiles, incluso si no son perfectas.

Esta combinación de la base Stem y la de Luxenburger nos ayuda a sacarle más partido al contexto y a adaptarnos mejor a diferentes tipos de datos.

Extracción y uso del conocimiento

Una de las mejores utilidades del Análisis Formal de Conceptos (AFC) es su capacidad para derivar implicaciones lógicas válidas entre atributos, que reflejan patrones presentes en los datos que se analizan. Este conocimiento vendrá expresado en forma de implicaciones que constituyen una base sólida para la interpretación, predicción y toma de decisiones en sistemas inteligentes.

Una vez obtenido el contexto formal y construido el retículo de conceptos, se extraen implicaciones mediante las bases mencionadas anteriormente de las cuales ya tratamos en el *Capítulo 2* su obtención. Las bases de implicaciones permiten sintetizar de manera compacta la estructura lógica del dominio representado, Figura 3.2.

Este conocimiento es valioso por diferentes razones:

1. **Validación de datos:** Las implicaciones son capaces de detectar inconsistencias. Si una implicación válida no se cumple en algún caso del conjunto, puede indicar un error en los datos.
2. **Reducción de redundancia:** Si un atributo puede deducirse a partir de otros quiere decir que es redundante, por lo que podríamos eliminarlo en nuestro estudio reduciendo la dimensión de nuestro espacio de implicaciones.
3. **Soporte a la toma de decisiones:** Las implicaciones actúan como reglas lógicas que ayudan a predecir información faltante mediante el cálculo de **cierres de implicaciones**, que permite deducir todos los atributos que deben estar presentes si se conocen algunos de ellos como se observa en el ejemplo del Cuadro 3.1. El conocimiento generado por lo tanto es explotable, comprensible y modular, lo que facilita su aplicación en tareas prácticas como clasificación, diagnóstico o recomendación.

Atributos Iniciales	{Mujer, Clase = 1 ^a }
Cierre Implicacional	{Mujer, Clase = 1 ^a , Clase = 3 ^a , Superviviente}

Cuadro 3.1: Ejemplo de deducción mediante cierre implicacional.

4. **Explicabilidad:** Dado que las implicaciones se derivan directamente de los datos y con un cierto umbral de confianza y soporte, ofrecen una justificación transparente de las inferencias, lo cual es esencial en el contexto de la inteligencia artificial explicable (XAI). Unas justificaciones podrían ser las vistas en el Cuadro 3.2.

Premisa	Conclusión	Explicación
{Edad $\leq$ 12, Mujer}	{Superviviente}	Las niñas suelen sobrevivir.
{Clase = 1 ^a , Mujer}	{Superviviente}	Las mujeres de primera clase tienen alta probabilidad de supervivencia.
{Hombre, Clase = 3 ^a }	{¬Superviviente}	Hombres de tercera clase tienden a no sobrevivir.

Cuadro 3.2: Explicaciones de posibles implicaciones del contexto del Titanic.

Sobre todo nos aprovecharemos de las aplicaciones 3 y 4. Que desarrollaremos en los *Capítulos 4 y 5* para la 3 y en el *Capítulo 6* para la 4.

3.2 Integración de implicaciones en modelos de Machine Learning

Una vez extraído el conocimiento en forma de bases de implicaciones mediante el Análisis Formal de Conceptos, el siguiente paso consiste en integrarlo dentro de un modelo de Machine Learning para aprovechar sus ventajas. Esta integración puede realizarse de diferentes maneras, dependiendo del tipo de problema a tratar y del tipo de modelo utilizado.

Uso como preprocesamiento de datos

Las implicaciones pueden emplearse como herramienta de preprocesamiento para mejorar la calidad de los datos antes de aplicar cualquier algoritmo de aprendizaje. Algunas aplicaciones concretas son:

- **Eliminación de atributos redundantes:** Si una implicación del tipo $A \rightarrow B$ es válida en todo el conjunto, y A ya está presente, entonces B no aporta información adicional y puede eliminarse.
- **Rellenado de datos faltantes:** En registros incompletos, las implicaciones permiten inferir valores faltantes si se conocen suficientes atributos en el antecedente.
- **Verificación de consistencia:** Se puede comprobar que cada registro del conjunto de datos cumple las implicaciones. Si no es así, podría ser erróneo o un caso particular (más frecuente en conjuntos de datos de menor tamaño).

Generación de nuevas características

Otra estrategia consiste en generar atributos adicionales a partir del conocimiento implícito en las implicaciones. Por ejemplo, si se detecta que $A \rightarrow B$ tiene una confianza alta, se puede crear una nueva variable binaria que indique si un registro cumple dicha regla. Estas nuevas características pueden mejorar el rendimiento del modelo, ya que aportan información derivada estructurada. Aunque no usaremos esta estrategia en nuestro ejemplo práctico.

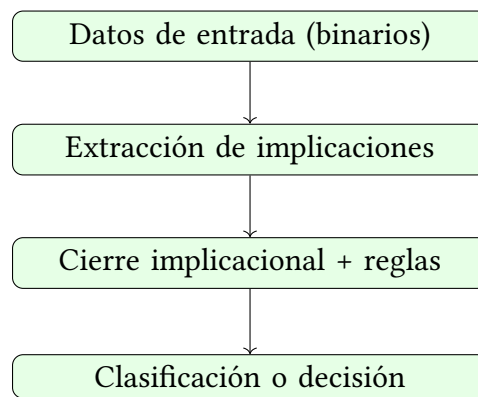


Figura 3.3: Aplicación de implicaciones para tareas de clasificación.

Explicabilidad del modelo

Uno de los desafíos actuales en aprendizaje automático es la interpretabilidad. Las implicaciones extraídas del contexto formal tienen una definición clara y son fácilmente comprensibles para nosotros. Por tanto, pueden utilizarse para:

- Generar explicaciones post-hoc de las decisiones del modelo.
- Validar reglas aprendidas por modelos de caja negra.
- Diseñar modelos basados en reglas, donde las implicaciones forman directamente el núcleo del razonamiento.

Tareas de clasificación

En tareas supervisadas como la clasificación, las implicaciones pueden utilizarse como un modelo base. Dado un nuevo objeto (registro) con ciertos atributos, se puede aplicar el **cierre implicacional** para predecir atributos faltantes o la pertenencia a una clase si esta forma parte de las implicaciones.

Esta es la **idea clave** detrás de nuestro modelo de IA. Crearemos las clases para la construcción de un contexto a partir de un conjunto de datos binario, que contendrá los algoritmos para la construcción de las bases de implicaciones necesarias. Con estas bases podremos dado un nuevo objeto hacer cierre implicacional para determinar si un atributo desconocido está presente en dicho cierre o no. Si está presente la decisión será positiva, por el contrario, si no está presente nuestra decisión será negativa. Estos pasos se resumen en el diagrama de flujo de la Figura 3.3.

Ventajas del enfoque

Integrar AFC en Machine Learning nos aporta una serie de ventajas:

- **Modelo interpretable:** Todas las decisiones pueden justificarse con implicaciones lógicas.
- **Reducción de complejidad:** Al eliminar redundancias y representar conocimiento estructurado.
- **Adaptabilidad:** Se pueden usar tanto implicaciones deterministas (base Stem) como aproximadas (base de Luxenburger).

3.3 Limitaciones y desafíos del enfoque con AFC

El enfoque del Análisis Formal de Conceptos (AFC) aplicado a Machine Learning presenta un conjunto de ventajas, como la reducción de la complejidad y la interpretabilidad, pero también surgen diversas limitaciones y desafíos que deben ser considerados en su implementación práctica.

A continuación veremos uno por uno los que han sido los más relevantes a tener en cuenta para nuestro trabajo práctico. Forman un conjunto de puntos a mejorar por medio del desarrollo de nuevas estrategias, intentos fallidos o aproximaciones que puedan aprovechar las fortalezas del AFC minimizando sus debilidades. En el siguiente capítulo, presentaremos un caso práctico utilizando el conjunto de datos del Titanic, donde analizaremos cómo superar algunos de estos retos y cómo podemos integrar el AFC en un modelo de Machine Learning efectivo.

Transformación de un conjunto de datos a binario

El primer desafío al que nos enfrentamos al aplicar aplicar AFC en Machine Learning es transformar un conjunto de datos a un formato adecuado para la generación de implicaciones. El formato más adecuado en el que tener los datos es en formato binario, ya que de esta manera si en una columna aparece un 1 (o 0) es que el atributo asociado es poseído (o no) por el objeto de la fila correspondiente. Básicamente queríamos hacer que nuestra relación binaria en forma de tabla de la definición de *Contexto formal* 1.3 fuese nuestro propio conjunto de datos.

Color original	Color_Rojo	Color_Verde	Color_Azul
Rojo	1	0	0
Azul	0	0	1
Verde	0	1	0

Cuadro 3.3: Codificación one-hot de un atributo categórico con tres valores.

En un conjunto de datos los atributos pueden ser continuos (por ejemplo, edad, tarifa) o discretos, pero todos deben ser convertidos de una forma binaria para que el AFC pueda extraer reglas válidas.

Decidir cómo realizar esta transformación en general es una cuestión crítica:

- Para columnas que toman un **reducido número de valores discretos** (por ejemplo 3, 4, 5, ...), es fácil, ya que bastaría con usar la técnica común de **codificación one-hot**.

El método de **codificación one-hot** consiste en convertir una columna con valores discretos (como Rojo, Verde, Azul) en varias columnas binarias, una por cada categoría posible. En cada fila, se coloca un 1 únicamente en la columna que corresponde a la categoría presente, y 0 en el resto. Por ejemplo el Cuadro 3.3.

De este modo, se evita asignar valores numéricos arbitrarios que podrían inducir un orden o relación inexistente entre las categorías originales.

Al usar esta técnica se mantiene toda la información presente en el conjunto de datos y es un **método determinista**, por lo que no es necesaria realizar ninguna decisión externa a la hora de ejecutar la codificación.

- Para columnas de **valores continuos** o con un **alto número de valores discretos** la codificación one-hot representaría un aumento bastante considerable del número de atributos. Además, que la mayoría de estos estarían presentes en escasos objetos, lo que haría que las implicaciones no aportasen ningún valor en nuestra predicción. Por eso hay que buscar una mejor manera de realizar el escalado.

El previo uso de rangos (por ejemplo, convertir la edad en grupos como "niño", "adulto", "anciano") o umbrales (como convertir la tarifa en categorías de alta o baja) será nuestra solución a este problema. Sin embargo, la elección del método y los umbrales adecuados **no es única** y puede tener un impacto significativo en la calidad de las implicaciones derivadas, ya que una mala discretización puede llevar a reglas poco útiles o incluso erróneas.

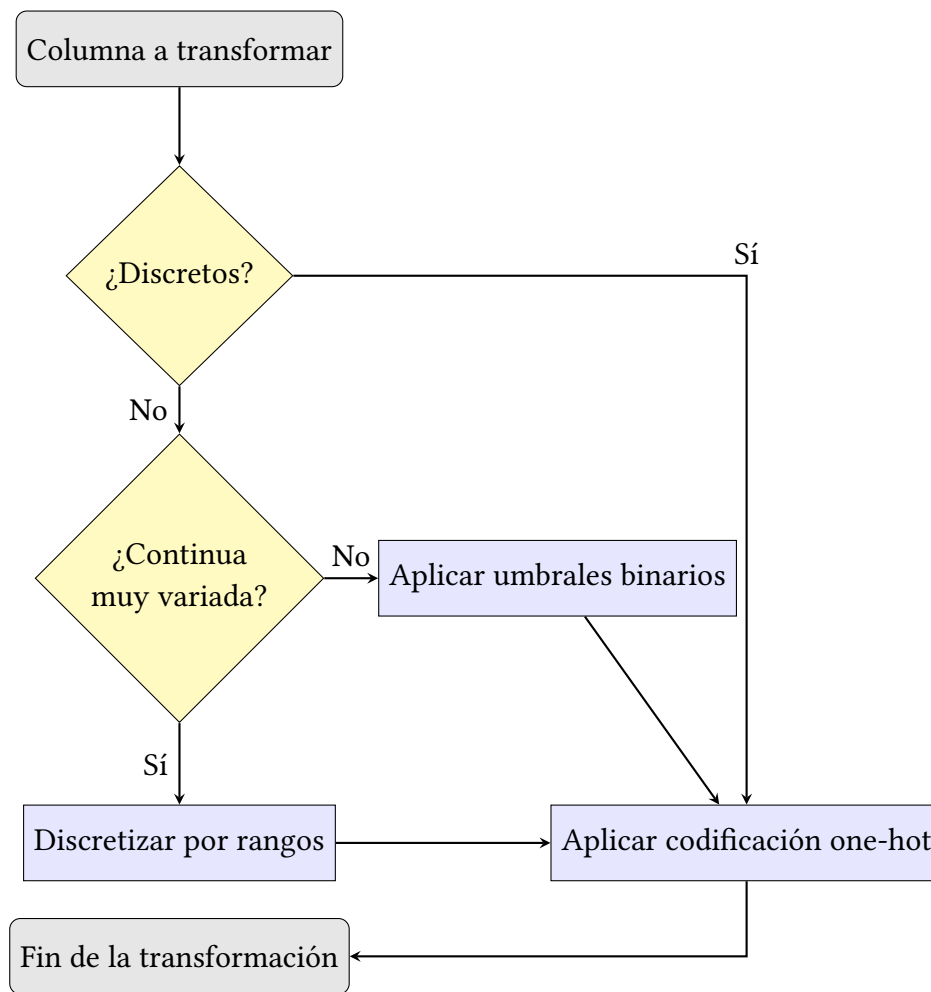


Figura 3.4: Diagrama de decisión para transformar una columna a binario.

En la Figura 3.4 se describe un diagrama de decisión que resume los pasos a seguir, descritos de forma detallada anteriormente, para la transformación de columnas a binarias.

Impacto del Escalado y Transformación

La decisión sobre cómo transformar y escalar un conjunto de datos no solo tiene implicaciones en términos de calidad de las implicaciones, sino que también en la eficiencia computacional del proceso. Si la transformación binaria y el escalado no se manejan de manera adecuada, el número de reglas generadas puede crecer como

Atributos	Implicaciones	Atributos	Implicaciones
25	638	35	1134
26	722	36	1196
27	793	37	1217
28	818	38	1252
29	874	39	1268
30	922	40	1338
31	977	41	1355
32	1040	42	1356
33	1095	43	1387
34	1068	44	1464

Cuadro 3.4: Número total de la base de implicaciones completa (confianza 0.95, soporte 0.005) por número de atributos.

Atributos	Tiempo medio (s)	Atributos	Tiempo medio (s)
25	92.29	35	115.89
26	104.20	36	120.01
27	104.72	37	119.41
28	104.90	38	118.73
29	107.55	39	119.58
30	109.28	40	121.63
31	111.75	41	120.88
32	113.98	42	119.13
33	114.74	43	120.61
34	115.14	44	122.61

Cuadro 3.5: Tiempo medio de ejecución por número de atributos.

se observa en el Cuadro 3.4, lo que puede hacer que el proceso sea extremadamente costoso en términos de tiempo y recursos. Además, una mala transformación puede dar lugar a reglas menos precisas o redundantes, lo que afecta negativamente a la capacidad del AFC para extraer conocimiento útil.

El Cuadro 3.5 contiene unos cálculos realizados sobre el conjunto de datos del Titanic haciendo aumentar el número de rangos de la columna edades. El número de **atributos** es igual al número total de columnas después de su transformación a binario y el **tiempo medio** se ha calculado como la media de tres ejecuciones haciendo variar el subconjunto de entrenamiento (siempre manteniendo 594 objetos). Se puede

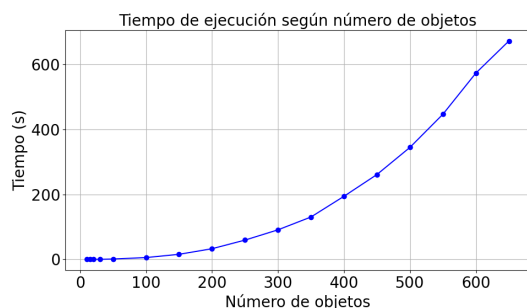


Figura 3.5: Tiempo de ejecución según número de objetos.

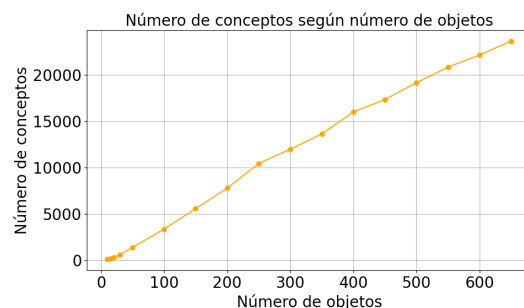


Figura 3.6: Número de conceptos según número de objetos.

apreciar el aumento del tiempo de ejecución con respecto al aumento del número total de atributos.

3.4 Impacto del Número de Objetos

El número de objetos en el conjunto de entrenamiento tiene un impacto considerable sobre el comportamiento del cálculo de las bases y en especial del retículo de conceptos. Las siguientes gráficas muestran cómo varían el tiempo de ejecución, la cantidad de conceptos formales y el número de implicaciones a medida que se incrementa el tamaño muestral para el conjunto de datos de *Churn Modelling* con una confianza del 80 % y soporte de 0.0224. Este conjunto de datos se estudiará más detalladamente en la última sección del *Capítulo 5*.

La gráfica de objetos vs. tiempo [3.5](#) muestra un crecimiento casi exponencial del tiempo necesario para completar el proceso. Si bien al principio el aumento es moderado (con 10 a 50 objetos el tiempo apenas supera el segundo), a partir de los 100 objetos el crecimiento se acelera, alcanzando casi 700 segundos con 650 objetos. No se observa estabilización, lo que sugiere que el tiempo de ejecución escala de forma no lineal y que el algoritmo es poco eficiente frente a grandes volúmenes de datos.

La relación entre el número de objetos y el número de conceptos, [Figura 3.6](#), también sigue una tendencia creciente, aunque con un ritmo más constante. A partir de los 250 objetos, el incremento se mantiene en torno a 1.000–2.000 conceptos adicionales cada 50 objetos, lo que indica una cierta estabilización en la tasa de crecimiento.

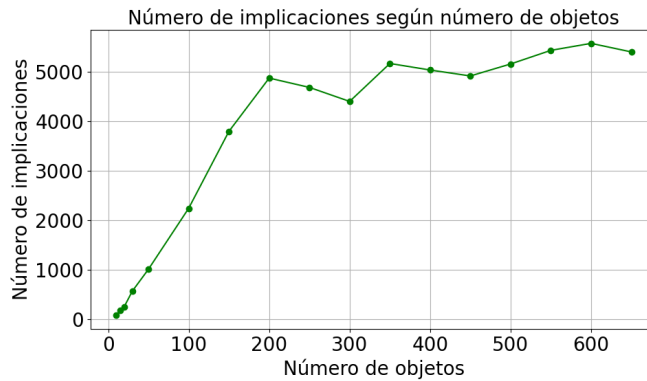


Figura 3.7: Número de implicaciones según número de objetos.

No obstante, el número total sigue aumentando de forma significativa, lo que refleja cómo la estructura del reticulado conceptual se vuelve más compleja con cada nuevo conjunto de ejemplos.

En la gráfica objetos vs. implicaciones 3.7 se observa una curva más irregular. Aunque en general el número de implicaciones aumenta con los objetos, a partir de los 250 objetos la tendencia parece estabilizarse e incluso decrecer ligeramente en ciertos tramos (por ejemplo, entre 300 y 400 objetos). Esto podría deberse a que, al crecer el conjunto de objetos, algunas reglas pierden validez por falta de soporte o se fusionan en reglas más generales. En todo caso, el comportamiento sugiere un efecto de saturación: añadir más datos no necesariamente implica generar más conocimiento útil.

Sensibilidad a la calidad de los datos

El AFC depende de la calidad de los datos disponibles para generar implicaciones. En escenarios donde los datos no contienen todos sus valores, ruidos o inconsistencias, las reglas derivadas pueden ser imprecisas o incluso incorrectas. Aunque existen métodos para manejar datos faltantes o ruidosos, como la base de Luxenburger, que introduce una aproximación probabilística, estas soluciones no son perfectas y pueden en muchas ocasiones perder precisión. El trabajo previo de limpieza y preprocesamiento de los datos se vuelve crucial para asegurar la calidad y utilidad de las implicaciones extraídas por las bases.

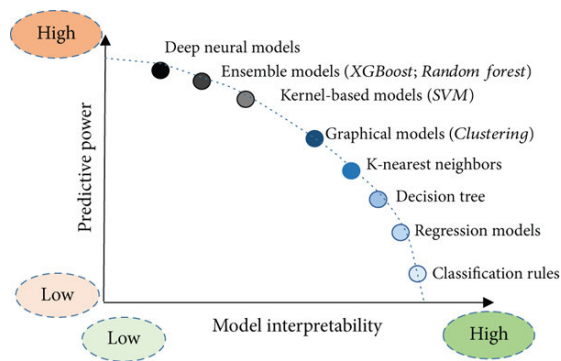


Figura 3.8: Poder predictivo VS Interpretabilidad.

La interpretabilidad frente a la precisión

El AFC facilita la interpretabilidad al ofrecer reglas lógicas claras. Aunque esto puede generar una limitación en términos de precisión. Los modelos basados en reglas pueden ser menos flexibles que otros más complejos, como las redes neuronales, que son capaces de modelar relaciones no lineales complejas. Esto puede ser un desafío cuando el objetivo es obtener una precisión máxima en tareas de predicción o clasificación, donde los modelos basados en reglas pueden no captar toda la riqueza de los datos.

Por tanto, comparándolo con otros modelos de Machine Learning en la gráfica de "Poder predictivo VS Interpretabilidad" 3.8 extraída de [7], este tipo de modelo se ubicaría cerca del extremo derecho (alta interpretabilidad) y parte inferior (bajo poder predictivo).

Manejo de relaciones no deterministas

El Análisis Formal de Conceptos tradicionalmente se ha enfocado en relaciones deterministas, donde una implicación se cumple de manera estricta (o sea, si $A \rightarrow B$ es válido, siempre que se cumpla A , también se debe cumplir B). Sin embargo, muchos problemas reales en Machine Learning no son puramente deterministas ya que hay una cierta incertidumbre, como se verá en nuestro caso de estudio del Titanic, donde existen factores que afectan la supervivencia de los pasajeros pero no garantizan resultados definitivos. La base de Luxenburger intenta abordar este desafío al permitir la incorporación de reglas probabilísticas, pero el manejo de la incertidumbre sigue

siendo un tema complejo y abierto, especialmente cuando se combinan varias fuentes de conocimiento con diferentes niveles de confianza.

Limitaciones en la detección de interacciones complejas

En lo que se basa el AFC es en identificar dependencias directas entre atributos, pero puede tener dificultades para capturar interacciones complejas entre múltiples atributos. En escenarios como el conjunto de datos del Titanic, donde factores como la clase social, el género, la edad y otros pueden afectar de maneras no triviales. El modelo basado en implicaciones podría no ser tan eficaz para capturar estas relaciones complejas como modelos más avanzados, como los árboles de decisión o las redes neuronales. Aunque el cierre implicacional puede ayudar a deducir algunas de estas interacciones, no siempre es suficiente para describir comportamientos complejos, especialmente si hay relaciones no lineales entre los atributos.

Limitaciones en la generalización

Por último, una de las principales limitaciones de los enfoques basados en reglas es su limitada capacidad de generalización. Si bien las implicaciones derivadas del AFC proporcionan una comprensión bastante clara de las relaciones en los datos, pueden ser muy específicas del conjunto de datos utilizado para la extracción (entrenamiento). Esto puede dificultar la capacidad del modelo para generalizar a nuevos conjuntos de datos o contextos. En tareas de clasificación o predicción, especialmente cuando se enfrenta a datos nunca antes vistos, a los modelos basados en AFC le pueden surgir dificultades para adaptarse rápidamente, especialmente si las implicaciones extraídas son demasiado rígidas o específicas del contexto.

4 | Caso de estudio: Titanic

4.1 Descripción del conjunto de datos

El conjunto de datos a usar como parte práctica va a ser el del **Titanic** en Kaggle [6] que es un recurso clásico utilizado para aprender sobre clasificación binaria y análisis de datos. Está basado en el trágico naufragio del RMS Titanic en 1912 y contiene información sobre los pasajeros, como su edad, clase social, sexo, tarifa del billete y si sobrevivieron o no a dicho desastre.

Contexto Histórico

El RMS Titanic era un conocido transatlántico británico que se hundió en su viaje inaugural, resultando en la muerte de más de 1.500 personas. Este desastre sucedió en parte a factores como el impacto con un iceberg y la insuficiencia de botes salvavidas.

El Titanic, en su época, era considerado un ejemplo de modernidad y lujo. Los pasajeros se dividían en tres clases cuya supervivencia estuvo influenciada por varios factores, como el género, la edad, la clase social y las decisiones de los oficiales durante el desastre.

El conjunto de datos

El conjunto de datos en **Kaggle** está dividido en tres partes:

- **train.csv**: contiene 891 instancias con datos sobre los pasajeros, incluyendo la columna objetivo **Survived**, que indica si un pasajero sobrevivió (1) o no (0).

Columna	Tipo	Descripción
PassengerId	Entero	Identificador único de cada pasajero.
Survived	Binario	Variable objetivo: 1 si sobrevivió, 0 en caso contrario.
Pclass	Categorico	Clase del billete (1ª, 2ª o 3ª). Refleja el nivel socio-económico.
Name	Texto	Nombre completo del pasajero. Incluye títulos sociales (Mr., Mrs., etc.).
Sex	Categorico	Sexo del pasajero (male, female).
Age	Numérico	Edad del pasajero en años. Contiene valores ausentes.
SibSp	Entero	Número de hermanos/esposos a bordo.
Parch	Entero	Número de padres/hijos a bordo.
Ticket	Texto	Código del billete. En algunos casos compartido por varios pasajeros.
Fare	Numérico	Tarifa pagada por el billete.
Cabin	Texto	Código de la cabina asignada. Contiene numerosos valores nulos.
Embarked	Categorico	Puerto de embarque: C (Cherbourg), Q (Queens-town), S (Southampton).

Cuadro 4.1: Descripción de las columnas del conjunto de datos del Titanic.

- **test.csv**: incluye 418 instancias sin la columna **Survived**, utilizadas para validar predicciones.
- **gender_submission.csv**: ofrece una solución de referencia basada en una regla sencilla (todas las mujeres sobreviven, los hombres no).

El objetivo principal de dicha competencia es predecir si un pasajero sobrevivió o no al desastre usando estos datos.

Nosotros solo usaremos el archivo **train.csv**, ya que como nuestro modelo es de aprendizaje supervisado, entonces necesitamos saber si la persona asociada a cada fila sobrevivió o no. El archivo **test.csv** lo usaremos para evaluar nuestra mejor versión del modelo en Kaggle en el *Capítulo 5*.

En el cuadro 4.1 se describen las columnas del conjunto de datos.

ID	Surv	Class	Name	Sex	Age
1	0	3	Braund, Mr. Owen Harris	male	22.0
2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0
3	1	3	Heikkinen, Miss. Laina	female	26.0
4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0
5	0	3	Allen, Mr. William Henry	male	35.0

Cuadro 4.2: Datos personales de los primeros cinco pasajeros del dataset.

SibSp	Parch	Ticket	Fare	Cabin	Emb
1	0	A/5 21171	7.25	–	S
1	0	PC 17599	71.2833	C85	C
0	0	STON/O2. 3101282	7.925	–	S
1	0	113803	53.1	C123	S
0	0	373450	8.05	–	S

Cuadro 4.3: Información del viaje para los primeros cinco pasajeros del dataset.

Los cuadros 4.2 y 4.3 muestran los valores de los 5 primeros pasajeros que aparecen en el conjunto de datos. Por ejemplo, ya se puede apreciar la falta de datos para la columna **Cabin**.

4.2 Preparación y construcción del contexto formal

Lo primero de todo es escoger el lenguaje de trabajo.

Motivación para el uso de Python

Se eligió el lenguaje de programación **Python** para el desarrollo de este trabajo debido a su amplio uso en el ámbito científico y en el análisis de datos. Python ofrece una gran cantidad de bibliotecas específicas que facilitan el procesamiento de datos, la transformación de atributos, la visualización y la implementación de algoritmos.

Entre las bibliotecas más utilizadas se encuentran:

- **pandas**: para la carga, limpieza y transformación del conjunto de datos.
- **numpy**: para operaciones numéricas eficientes.
- **matplotlib**: para la generación de gráficos exploratorios como histogramas.
- **concepts**: utilizada para representar el contexto formal y construir la jerarquía de conceptos.

Selección del subconjunto de datos

También llamaremos atributos a las columnas de nuestro conjunto de datos.

Para aplicar el Análisis Formal de Conceptos (AFC) al conjunto de datos del Titanic, se seleccionó un subconjunto de atributos que resultasen relevantes e influyentes a la hora de determinar el atributo objetivo **Survived** de un pasajero.

Los atributos seleccionados son los siguientes:

- **Sex**: permite distinguir entre hombres y mujeres, lo cual es especialmente relevante en el contexto del naufragio, dada la política de "mujeres y niños primero".
- **Pclass**: representa la clase del billete (1ª, 2ª o 3ª), lo que está bastante relacionado con el nivel socioeconómico del pasajero.
- **Age**: variable numérica que determina la edad del pasajero, que es relevante por la misma razón que el atributo **Sex**.
- **Fare**: indica la tarifa pagada por el billete, asociada también al nivel socioeconómico y al tipo de acomodación.
- **Embarked**: indica el puerto de embarque (S, C o Q), útil para estudiar agrupaciones geográficas o sociales.
- **SibSp** y **Parch**: indican si el pasajero viajaba con familiares, lo que puede influir en el comportamiento en el momento de la evacuación y por consiguiente a la supervivencia.

Además, se incluye el atributo **Survived** como referencia para posteriores análisis y validación del conocimiento extraído. Esta se incorpora directamente en la construcción del contexto formal.

Preprocesamiento

El archivo en formato CSV se ha procesado por medio de *pandas* y guardado en un objeto del tipo *DataFrame*.

Lo primero para empezar el preprocesamiento es limpiar los datos de errores, duplicados o valores nulos.

Para detectar posibles errores se han mostrado los valores únicos de todos los atributos que usaremos para identificar valores fuera de lo común. El resultado es que salvo en el atributo **Embarked** que había un NaN en los demás estaba todo correcto.

Duplicados en este caso no había ya que el **PassengerId** era diferente para todos.

Algunos valores nulos si que están presentes, así que para solucionarlo procederemos de la siguiente manera:

- Para **Sex**, **Pclass**, **Embarked**, **SibSp** y **Parch** sustituimos los valores nulos por la moda (el valor que más se repite).
- Para **Age** los sustituimos por su mediana (el valor que separa la mitad inferior y superior los datos en una columna).
- Para **Fare** los sustituimos por su media aritmética.

Por último eliminamos las columnas cuyos atributos no han sido seleccionados anteriormente. Estas son **Name**, **Ticket** y **Cabin**. En el caso de la columna **PassengerId** la dejamos ya que nos será útil tener un identificador único de cada objeto a la hora de construir nuestro contexto formal.

Aplicamos el escalado

Antes de construir el contexto formal, fue necesario aplicar un escalado al conjunto de datos para adaptar sus atributos al formato binario requerido por el Análisis Formal de Conceptos (AFC).

Al atributo **PassengerId** no hay que modificarlo al contener los índices de nuestros objetos. Igualmente **Survived** al tomar solo valores binarios no es necesario modificarlo.

Por el contrario para todos los demás atributos tendremos que aplicar directamente la *codificación one-hot* o un escalado previo a esta. Para **Pclass** y **Embarked**, como ambos son atributos discretos con 3 elementos cada uno, podemos aplicar directamente la *codificación one-hot*.

Sin embargo, para **Age**, **Fare**, **SibSp** y **Parch** tenemos que plantearnos el tipo de rangos en los que separarlos previamente. Esta no es una operación sencilla como ya vinimos introduciendo en los capítulos anteriores, por lo que no habrá unos rangos definidos para todo el estudio. Es más, en el *Capítulo 5* nos centraremos en la búsqueda de los rangos que dan un mejor desempeño, serán como hiperparámetros de nuestro modelo. Un ejemplo de rangos estándar para una primera implementación podrían ser:

- **Age**: [0, 13), [13, 20), [20, 30), [30, 40), [40, 65), [65, ∞) (80 es el máximo).
- **Fare**: [0, 12), [12, 25), [25, 50), [50, 100), [100, 200), [200, ∞) (512 es el máximo).
- **SibSp**: [0, 1), [1, 3), [3, 5), [5, 7), [7, ∞) (8 es el máximo).
- **Parch**: [0, 1), [1, 3), [3, 5), [5, ∞) (6 es el máximo).

Construcción del contexto formal

Una vez que tengamos nuestro conjunto de datos de tipo binario, salvo la primera columna de índices, con los atributos seleccionados podemos pasar a la construcción del contexto formal.

Para su construcción y gestión utilizaremos la librería `concepts` [12] de Python, la cual implementa la clase `Context(objects, properties, bools)`.

En particular, emplearemos su método de clase `fromfile`:

```
fromfile(filename, fformat='cxt', encoding=None, **kwargs)
```

Este método permite crear un contexto formal a partir de un archivo, como por ejemplo un fichero en formato CSV. En nuestro caso, dicho archivo debe tener la primera columna como identificador único de cada objeto, y las columnas restantes con valores "X" (marca de incidencia) o cualquier otro carácter distinto (ausencia de incidencia).

El resultado es un contexto formal donde:

- Las filas representan el conjunto de objetos.
- Las columnas, a partir de la segunda, representan los atributos.
- El `.csv` con las marcas "X" indican en forma de tabla la relación de incidencia entre objetos y atributos.

Por tanto, antes de construir el contexto, es necesario transformar el `.csv` para que todos los valores binarios (1 y 0) se conviertan en "X" y " " respectivamente, excepto en la columna de identificadores.

Una vez creado, el objeto de tipo `Context` proporciona diversas funcionalidades para el análisis formal, entre las que destacan:

- **Visualización del contexto:** por medio de `print(cxt)` se puede mostrar la tabla de incidencia completa.
- **Acceso a objetos y atributos:** a través de las propiedades `cxt.objects` y `cxt.properties`, que devuelven listas ordenadas.
- **Consulta de incidencias:** mediante `cxt[int][int]`, que indica si un objeto posee un atributo determinado.
- **Obtención de conceptos formales:** con `cxt.lattice` se puede crear la jerarquía de conceptos (retículo).
- **Exportación:** permite guardar el contexto en distintos formatos, como `.cxt`, para ser usado en herramientas externas como `ConExp`.

4.3 Extracción de implicaciones

Una vez que ya tenemos nuestro contexto creado el siguiente paso es la construcción de las bases de implicaciones que son el pilar fundamental de nuestro modelo. La construcción de dichas bases no está implementada en `concepts` por lo que es un trabajo que debemos hacer.

Crearemos una subclase de `Context`, con el mismo nombre, e implementaremos todos los métodos necesarios para la construcción de los algoritmos que calculen nuestras bases. A parte también serán necesarios otros métodos más típicos del Machine Learning. Los métodos añadidos son los siguientes:

- `__init__`: Constructor de la clase; inicializa el contexto formal y los parámetros de soporte y confianza.

- `desde_dataframe`: Crea una instancia de `Context` a partir de un `DataFrame` de `pandas`.
- `set_confianza`: Establece el valor de confianza.
- `get_confianza`: Devuelve el valor actual de confianza.
- `set_soporte`: Establece el valor de soporte.
- `get_soporte`: Devuelve el valor actual de soporte.
- `cerrar`: Cierra un conjunto de atributos según una base de implicaciones dada.
- `next_closure`: Calcula el siguiente cierre léxico de un conjunto de atributos.
- `doble_derivada`: Calcula la doble derivada de un conjunto de atributos.
- `calcular_base_stem`: Calcula la base Stem del contexto formal.
- `es_cerrado`: Verifica si un conjunto de atributos es cerrado.
- `soporte_atributos`: Calcula el soporte de un conjunto de atributos.
- `soporte_concepto`: Calcula el soporte de un concepto formal.
- `sigma_frecuente`: Verifica si un concepto es frecuente según un umbral σ .
- `es_inmediatamente_inferior`: Determina si un concepto es inmediatamente inferior a otro.
- `soporte_implicacion`: Calcula el soporte de una implicación.
- `confianza_implicacion`: Calcula la confianza de una implicación.
- `calcular_base_luxenburger`: Calcula la base de Luxenburger del contexto formal.
- `calcular_base_completa`: Calcula la base completa combinando la base Stem y la de Luxenburger.
- `cerrar_conjunto`: Cierra un conjunto de atributos usando la base completa.
- `entrenar`: Calcula todas las bases implicacionales del contexto.
- `predecir`: Predice etiquetas aplicando el cierre implicacional basado en la base completa.
- `precision`: Calcula la precisión de las predicciones respecto a las etiquetas reales.
- `falsos_positivos`: Cuenta los falsos positivos en la predicción.
- `falsos_negativos`: Cuenta los falsos negativos en la predicción.
- `calcula_resultados`: Resume la precisión, falsos positivos y falsos negativos.

Esto es un breve resumen de todos los métodos implementados. Ahora vamos a centrarnos en `calcular_base_stem`, `calcular_base_luxenburger` y `calcular_base_completa`. Trataremos sus implementaciones, detalles técnicos, peculiaridades y los diferentes problemas surgidos durante estas.

Listing 4.1: next_closure.

```

def next_closure(self, conjunto, conj_impls):
    """Calcula el siguiente cierre de un conjunto de atributos."""
    rev_set_prop=sorted(self.properties, reverse=True)
    for atributo in rev_set_prop:
        if atributo in conjunto:
            conjunto=conjunto - {atributo}
        else:
            sig_conj_cerrado=self.cerrar(conjunto | {atributo}, conj_impls
            )
            if all(x >= atributo for x in sig_conj_cerrado - conjunto):
                return sig_conj_cerrado
    return None

```

Método calcular_base_stem

La **primera aproximación** del método fue siguiendo el Algoritmo 2 que está basado en simplemente aplicar la definición de base Stem. Esta aproximación, aunque fue correcta, en la práctica el **tiempo de ejecución era excesivo** llegando a más de media hora de cálculo para un contexto con alrededor de 15 atributos.

Investigando para mejorarlo se encontró el algoritmo NextClosures, descrito en [13], sin éxito ya que estaba diseñado para computación paralela y me daba implicaciones correctas, pero no todas las pertenecientes a la base. Tras intentar solucionarlo y no avanzar deseché el algoritmo.

Así que después de buscar en más artículos y libros se dió con el algoritmo **Next Closure algorithm** que sirve de base para el algoritmo **Canonical Basis(M,")**. Ambos presentes en [9]. Sus pseudocódigos están descritos en los algoritmos 3 y 4 respectivamente. El segundo será el algoritmo definitivo para implementarlo como método de la clase. Tardando menos de un minuto para el cálculo de la base Stem en contextos con hasta 40 atributos lo que es una mejora muy notoria. Sus implementaciones se muestran en los listados 4.1 y 4.2.

Listing 4.2: calcular_base_stem

```
def calcular_base_stem(self):
    """Calcula la base Stem de un contexto como lista de tuplas."""
    conj_atrib=set()
    base=[]
    atributos=set(self.properties)
    while conj_atrib!=atributos and conj_atrib!= None:
        conj_atrib_pp=self.doble_derivada(conj_atrib)
        if conj_atrib != conj_atrib_pp:
            implicacion = (conj_atrib, conj_atrib_pp - conj_atrib)
            base.append((implicacion, 1., self.soporte_implicacion(
                implicacion)))
        conj_atrib = self.next_closure(conj_atrib, base)
    self.base_stem = base
    return base
```

Método calcular_base_luxenburger

Para la construcción de este método también se pasó por un intento anterior. En dicho intento lo que se hacía era por medio del método

Context.neighbors(objects, raw=False) (de concepts)

recorrer el retículo de conceptos desde el **menor concepto**, ya que neighbors devuelve los conceptos justamente superiores a este. Pero al recorrerlos del **mayor concepto** al menor se podría podar por la confianza mínima, ya que la confianza de una implicación de la base de Luxenburger es igual a la proporción de objetos del concepto, cuya intención es el consecuente, con respecto al total de objetos. Es decir, si al recorrer el retículo de conceptos de mayor a menor encontramos un concepto $C_1 = (X_1, Y_1)$, y uno de sus vecinos inferiores es $C_2 = (X_2, Y_2)$ tal que $\frac{|X_2|}{|O|}$ es menor que la confianza mínima, entonces podemos **podar** por C_2 . Esto se debe a que todos sus subconceptos tendrán una extensión de tamaño menor o igual, y por tanto no podrán generar ninguna implicación adicional que cumpla los umbrales mínimos para la base de Luxenburger.

Por estas razones el método que buscamos recorrerá el retículo partiendo del mayor concepto y pudiendo realizar la poda. El problema al que nos enfrentamos ahora es en cómo calcular los vecinos inferiores de un concepto. No existe en concepts un

método que lo calcule sin previamente tener que calcular todo el retículo de conceptos (por lo que la poda no se aprovecharía al máximo). Teniendo el retículo creado si que `concepts` ya dispone de un método que dado un concepto este calcula sus vecinos inferiores (`lower_neighbors`).

Así que tras investigar sin éxito la creación de un método que supliese nuestra necesidad, se pensó que se podría aprovechar la necesidad de la creación del retículo. Porque de esta forma calculando una vez la base de Luxenburger de una determinada confianza y soporte calculamos el retículo, ya este se quedaría guardado en la clase y para calcular otra base de Luxenburger con diferentes umbrales solo bastaría con recorrer el retículo haciendo el filtrado de las implicaciones por dicho umbral, pudiendo aprovechar la poda por confianza mínima, que es mucho menos costoso que el cálculo del retículo.

Con todas las anteriores consideraciones el código del método que calcula la base de Luxenburger quedaría de la forma que aparece en el Listado 4.3.

Listing 4.3: `calcular_base_luxenburger`.

```
def calcular_base_luxenburger(self):
    """Calcula la base de Luxenburger de un contexto formal como lista de
    tuplas de conjuntos de atributos."""
    base_luxenburger = []
    visitados = set()
    reticulo = self.lattice
    n = len(self.objects)
    def aux_base_luxenburger(concepto_actual):
        """Auxiliar para recorrer cada nodo del reticulo del contexto
        formal de arriba a abajo."""
        if concepto_actual in visitados: # Si el concepto actual ya fue
            visitado, no hacer nada
            return
        visitados.add(concepto_actual)
        for vecino in concepto_actual.lower_neighbors: # Recorrer los
            vecinos inferiores del concepto actual gracias al reticulo
            intencion_vecino = set(vecino.intent)
            intencion_actual = set(concepto_actual.intent)
            implicacion = (intencion_actual, intencion_vecino -
                intencion_actual) # Representa la impl Act -> Vec\Act
            num_objetos_actual = len(concepto_actual.extent)
            if num_objetos_actual != 0:
                conf_Implicacion = len(vecino.extent)/num_objetos_actual
```

```

else:
    conf_Implicacion = 1.0
    sop_Implicacion = len(vecino.extent)/n
    if sop_Implicacion >= self.soporte and conf_Implicacion >=
        self.confianza: # Si la implicacion es frecuente,
            agregarla a la base de Luxenburger
            base_luxenburger.append((implicacion, conf_Implicacion,
                sop_Implicacion))
    elif sop_Implicacion < self.soporte: # Realizar la poda si el
        soporte es menor al umbral
        return
    aux_base_luxenburger(vecino)
    aux_base_luxenburger(reticulo[len(reticulo)-1]) # Comenzar desde el
        concepto superior del reticulo
    self.base_luxenburger = base_luxenburger
    return base_luxenburger

```

Método calcular_base_completa

Este método es el más sencillo, simplemente hay que hacer la unión de las bases Stem y Luxenburger. El resultado es una lista de tuplas, donde cada tupla está formada por la implicación, su confianza y su soporte. La implicación a su vez es una tupla de dos elementos, donde el primero es el conjunto de atributos antecedente y el segundo el conjunto de atributos consecuente. Se muestra en el Listado 4.4.

Listing 4.4: calcular_base_completa.

```

def calcular_base_completa(self):
    """Calcula la base completa como la union de la base Stem y de
        Luxenburger."""
    if self.base_stem is None:
        raise Exception("Primero debe calcular la base Stem.")
    if self.base_luxenburger is None:
        raise Exception("Primero debe calcular la base de Luxenburger.")

    base_completa = self.base_stem.copy()
    base_completa.extend(self.base_luxenburger)
    self.base_completa = base_completa
    return base_completa

```

4.4 Primeros resultados y análisis

Una vez descritas todas las herramientas que usaremos para nuestro modelo vamos a ponerlo a prueba con el conjunto de datos del Titanic.

Para ello vamos a usar los rangos descritos a continuación para la creación de nuestros atributos:

- **Age:** [0, 13), [13, 20), [20, 30), [30, 40), [40, 65), [65, ∞) (80 es el máximo).
- **Fare:** [0, 12), [12, 25), [25, 50), [50, 100), [100, 200), [200, ∞) (512 es el máximo).
- **SibSp:** [0, 1), [1, 3), [3, 5), [5, 7), [7, ∞) (8 es el máximo).
- **Parch:** [0, 1), [1, 3), [3, 5), [5, ∞) (6 es el máximo).

El número total de atributos se quedaría en 30. Serían los siguientes:

- | | | |
|---------------------|------------------|------------------|
| ▪ Survived | ▪ AgeGroup_30-40 | ▪ SibSpGroup_1-3 |
| ▪ FareGroup_0-12 | ▪ AgeGroup_40-65 | ▪ SibSpGroup_3-5 |
| ▪ FareGroup_12-25 | ▪ AgeGroup_65+ | ▪ SibSpGroup_5-7 |
| ▪ FareGroup_25-50 | ▪ Embarked_C | ▪ SibSpGroup_7+ |
| ▪ FareGroup_50-100 | ▪ Embarked_Q | ▪ ParchGroup_0-1 |
| ▪ FareGroup_100-200 | ▪ Embarked_S | ▪ ParchGroup_1-3 |
| ▪ FareGroup_200+ | ▪ Pclass_1 | ▪ ParchGroup_3-5 |
| ▪ AgeGroup_0-13 | ▪ Pclass_2 | ▪ ParchGroup_5+ |
| ▪ AgeGroup_13-20 | ▪ Pclass_3 | ▪ Male |
| ▪ AgeGroup_20-30 | ▪ SibSpGroup_0-1 | ▪ Female |

Ya teniendo nuestro conjunto de datos binario vamos a separarlo aleatoriamente en un 80 % para el **entrenamiento** y un 20 % para la **prueba**. Dando un total de 712 y 179 filas (objetos) respectivamente. Como hay 712 objetos de entrenamiento las medidas de confianza mínima las tomaremos como fracciones de denominador 712, ya que la confianza representa la fracción respecto al total de objetos que cumplen el consecuente.

Pasemos a cómo realizar el **entrenamiento**. Primero creamos el contexto con los datos de entrenamiento, después establecemos una confianza mínima y un soporte mínimo mediante sus respectivos métodos. A continuación por medio del método entrenar del contexto se crean las bases anteriormente mencionadas. De esta manera

Antecedente	Consecuente	Soporte
{Male, SibSpGroup_0-1, AgeGroup_20-30, FareGroup_0-12}	{ParchGroup_0-1}	0.191
{SibSpGroup_1-3, AgeGroup_0-13}	{ParchGroup_1-3}	0.031
{ParchGroup_1-3, Embarked_C, FareGroup_12-25}	{Pclass_3}	0.015
{ParchGroup_0-1, AgeGroup_13-20, Female, Survived, FareGroup_12-25}	{Embarked_C}	0.003

Cuadro 4.4: Ejemplos de implicaciones de la base Stem.

ya estaría todo listo para hacer las predicciones, testeos, pruebas y todo lo que sea necesario.

Una vez ya realizado el entrenamiento la **prueba** consiste en eliminar la columna de **Survived** y para cada objeto del conjunto hacer el cierre implicacional de sus atributos por medio de la base completa (método `cerrar_conjunto`). Si en dicho cierre está contenido el atributo **Survived** entonces se marcará como resultado positivo, si no está entonces el resultado se marcará como negativo. Y así es como nuestro modelo realizará las respectivas predicciones.

Se utilizaron diversas **métricas de evaluación** para analizar el rendimiento del modelo, como la precisión, la tasa de falsos positivos y falsos negativos. Además también podremos medir la cantidad de implicaciones calculadas por cada base, el tiempo de ejecución de cada una y más.

Para nuestro conjunto de prueba elegido tengamos en cuenta que el número de objetos (filas) con **Survived** igual a "X" (1) es 69 y el número de iguales a " " (0) es 110. Sumando 179 que es el total de objetos del conjunto de prueba.

Resultados solo base Stem

Tardando escasos 4,3 segundos en la creación de la base Stem esta cuenta con 927 implicaciones. Unos ejemplos de las implicaciones creadas se pueden ver en el Cuadro 4.4, donde se precisa el soporte pero no la confianza al ser esta siempre igual a 1 para todas las implicaciones de la base.

Las implicaciones separadas por soporte se pueden visualizar por medio del his-

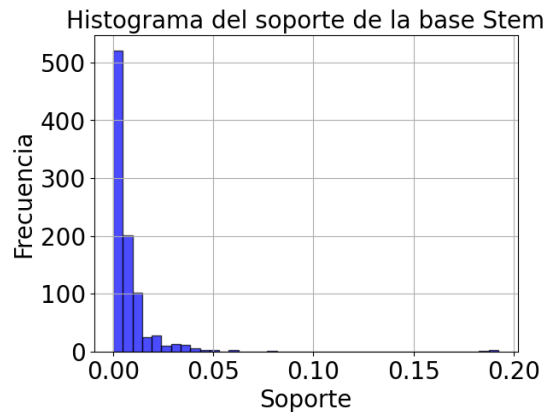


Figura 4.1: Histograma del soporte de las implicaciones de la base Stem.

tograma de la Figura 4.1. Los soportes se acumulan en torno a valores bajos, ya que por la variabilidad de objetos es poco frecuente que una implicación se satisfaga por varios de estos.

El número de implicaciones que contienen el atributo **Survived** en el consecuente es de 203, en torno al 22 % de las implicaciones. Estas son las que permiten generar predicciones positivas en nuestro modelo.

Mientras que las implicaciones que contienen el atributo **Survived** en el antecedente son 276, aproximadamente el 30 %. Estas implicaciones son inútiles para nuestro modelo, ya que para que estas sean disparadas haciendo el cierre implicacional debería de estar contenido el atributo **Survived** en el conjunto, por lo que ya la predicción sería seguro positiva a pesar de que faltasen estas por ser disparadas. Así que podríamos deshacernos de ellas con el modelo actual.

El **ratio de acierto** usando solo esta base para el cierre implicacional es de 0,73. Con 26 **falsos positivos** y 23 **falsos negativos**.

El hecho de que se generen 26 falsos positivos representa una limitación importante, tratada en el *Capítulo 5*. Al no trabajar con atributos negados, cualquier conjunto adicional de implicaciones derivadas de la base de Luxenburger solo puede mantener o aumentar los falsos positivos, ya que las reglas que los causan siguen presentes. Esto fija un máximo de acierto del 85,5 % que no resulta suficientemente alto para cualquier conjunto de implicaciones que contenga la base Stem.

Antecedente	Consecuente	Confianza	Soporte
{AgeGroup_30-40, Male, ParchGroup_0-1, Pclass_1, Embarked_S}	{Survived}	0.50	0.007
{FareGroup_12-25, SibSpGroup_1-3, Pclass_2}	{Male}	0.60	0.008
{FareGroup_12-25, ParchGroup_0-1, Survived, Female, Embarked_S}	{Pclass_3}	0.30	0.010
{Pclass_3, FareGroup_0-12}	{AgeGroup_40-65}	0.09	0.034

Cuadro 4.5: Ejemplos de implicaciones de la base Luxenburger.

Resultados solo base Luxenburger

Pasemos al caso de solo la base de Luxenburger. Al tener que crear el retículo de conceptos en la primera ejecución del método `calcular_base_luxenburger` esto hace que sea la más costosa llegando a los 3 minutos con 30 segundos aproximadamente. Ya una vez el retículo esté calculado tardará en una segunda ejecución como máximo 1,6 segundos, que es una mejora significativa cuando tengamos que trabajar y comparar diferentes bases de Luxenburger del mismo contexto para diferentes umbrales. Tomando la confianza y soporte mínimos a 0 la base contiene 18173 implicaciones.

En este caso el número de todas las posibles implicaciones de la base de Luxenburger que contienen el atributo **Survived** en el consecuente es 1695, el 9 %. Mientras que las que lo contienen en el antecedente son 6564, el 36 %. Podríamos deshacernos de estas últimas como comentamos anteriormente. Unos ejemplos de las implicaciones contenidas en la base se pueden ver en el Cuadro 4.5.

La **confianza** de las implicaciones de la base de Luxenburger con umbrales a 0, mostrada en el histograma de la Figura 4.2, presenta picos en valores como 0.25, 0.33, 0.5, 0.66 y 0.75. Esto se debe a que la confianza se calcula como el cociente entre el número de objetos que cumplen el consecuente y el número que cumplen el antecedente. Al trabajar con conceptos con pocos objetos, estas proporciones suelen tomar valores fraccionarios simples como $1/2$, $2/4$, $3/4$, etc., lo que genera una distribución discreta en el histograma.

En el histograma del **soporte** de la base de Luxenburger con umbrales a 0 (Figura 4.3), observamos que la mayoría de implicaciones se agrupan en valores cercanos a

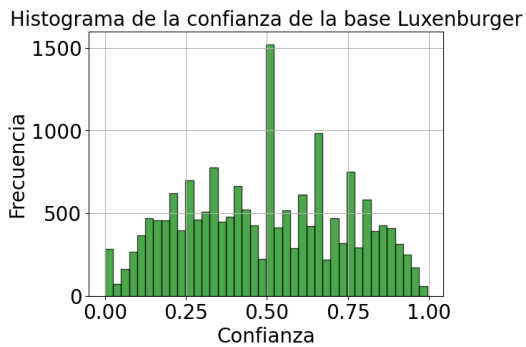


Figura 4.2: Histograma de la confianza de la base Luxenburger con umbrales a 0.



Figura 4.3: Histograma del soporte de la base Luxenburger con umbrales a 0.

0. Esto indica que afectan a pocos objetos del contexto. Es decir, se trata de reglas poco generales, lo que concuerda con el hecho de trabajar con el conjunto de conceptos del retículo.

Cuadro 4.6: Resultados de la evaluación de reglas de asociación aproximadas en función de distintos valores de confianza y soporte.

Confianza	Soporte	R. acierto	FP	FN	LHS	RHS	Total
0.000	0.000	0.385	110	0	6564	1695	18173
0.000	0.001	0.385	110	0	6409	1591	17914
0.000	0.003	0.385	110	0	5973	1488	17015
0.000	0.011	0.385	110	0	2988	845	9755
0.000	0.045	0.385	110	0	375	157	2104
0.000	0.180	0.385	110	0	6	6	343
0.200	0.000	0.385	110	0	5910	1467	15638
0.200	0.001	0.385	110	0	5910	1467	15638
0.200	0.003	0.385	110	0	5524	1368	14861
0.200	0.011	0.385	110	0	2848	772	8675
0.200	0.045	0.385	110	0	365	139	1847
0.200	0.180	0.385	110	0	6	6	341
0.500	0.000	0.486	88	4	3470	986	9411

Continúa en la siguiente página

Confianza	Soporte	R. acierto	FP	FN	LHS	RHS	Total
0.500	0.001	0.486	88	4	3470	986	9411
0.500	0.003	0.486	88	4	3252	913	8942
0.500	0.011	0.492	87	4	1697	467	5328
0.500	0.045	0.536	79	4	240	71	1346
0.500	0.180	0.816	14	19	6	1	315
0.800	0.000	0.709	40	12	844	268	2603
0.800	0.001	0.709	40	12	844	268	2603
0.800	0.003	0.709	40	12	844	268	2603
0.800	0.011	0.721	38	12	561	163	1948
0.800	0.045	0.827	6	25	63	13	489
0.800	0.180	0.615	0	69	0	0	144
0.900	0.000	0.782	16	23	194	74	778
0.900	0.001	0.782	16	23	194	74	778
0.900	0.003	0.782	16	23	194	74	778
0.900	0.011	0.782	16	23	194	74	778
0.900	0.045	0.810	3	31	29	10	227
0.900	0.180	0.615	0	69	0	0	73
0.950	0.000	0.782	3	36	20	14	199
0.950	0.001	0.782	3	36	20	14	199
0.950	0.003	0.782	3	36	20	14	199
0.950	0.011	0.782	3	36	20	14	199
0.950	0.045	0.737	0	47	7	4	86
0.950	0.180	0.615	0	69	0	0	35

En el Cuadro 4.6 **FP** y **FN** son falso positivo y falso negativo. **LHS** y **RHS** representan el número de implicaciones que contienen **Survived** en el antecedente o consecuente respectivamente.

Se observa que, a medida que se reducen los umbrales de confianza y soporte, aumenta el número de implicaciones cuyo **RHS** (right-hand side) contiene el atributo objetivo **Survived**, lo que implica que se clasifican más objetos como positivos. Esto hace que el número de **falsos positivos** aumente hasta el máximo de 110.

En cambio, mientras más altos sean los umbrales pasa exactamente lo contrario. Disminuye el número de implicaciones cuyo **RHS** contiene **Survived**, por lo que los objetos se clasificarán en su mayoría como negativos. Generando así el aumento de

Antecedente	Consecuente	Confianza	Soporte
{ParchGroup_1-3, Embarked_S}	{Pclass_1}	0.22	0.037
{FareGroup_12-25, SibSpGroup_0-1, ParchGroup_0-1, Female, Embarked_S}	{Survived}	0.94	0.024
{Female, Embarked_C}	{SibSpGroup_1-3}	0.43	0.035
{ParchGroup_1-3, Pclass_3, Survived, Embarked_S, Male}	{AgeGroup_0-13}	1.00	0.010

Cuadro 4.7: Ejemplos de implicaciones de la base completa.

los **falsos negativos** hasta su máximo de 69.

El mejor **ratio de acierto** ha sido **0.827** para la confianza de 80 % y soporte 0.045 que equivale a 32/712, osea conceptos como poco que contengan 8 objetos. Con 6 **falsos positivos** y 25 **falsos negativos**.

Lo que podríamos tratar es bajar levemente los umbrales cerca de 0.80 y 0.045 para así tratar de minimizar los 25 falsos negativos y aumentar ese 0.82 de ratio de acierto. Pero tras hacer las pruebas no se ha podido aumentar.

También es interesante explorar cerca de los umbrales 0.90 y 0.045 con 0.81 de ratio de acierto. Después de hacer los calculos vemos que para confianza 0.93 y soporte 0.056 se obtiene 0.82 de ratio de acierto que es un resultado también muy cercano a nuestro máximo. Lo bueno es que al tener una confianza mínima de 0.93 nos estamos asegurando que las reglas usadas para el cierre implicacional sean altamente probables en nuestro contexto.

Resultados base completa

Ahora pasemos a unir ambas bases en la llamada **base completa** y estudiar lo que ocurre. El tiempo de ejecución en este caso va marcado por la complejidad del cálculo del retículo de conceptos para la base de Luxenburger, en total en torno a 3 minutos con 35 segundos. Tenemos las mismas propiedades que con el caso de solamente la base de Luxenburger respecto al cálculo con diferentes umbrales mínimos. Tomando la confianza y soporte mínimos la base completa contiene 19100 reglas de asociación (implicaciones) tanto escritas como aproximadas. En el Cuadro 4.7 se muestran ejemplos de implicaciones de la base completa.

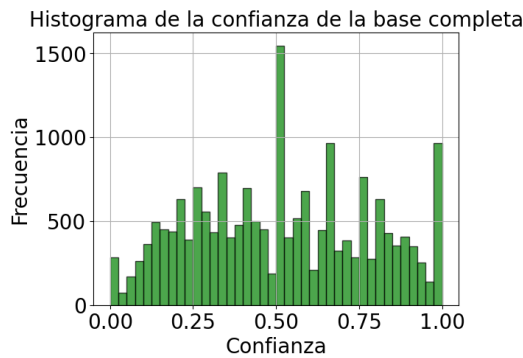


Figura 4.4: Histograma de la confianza de la base completa con umbrales mínimos.

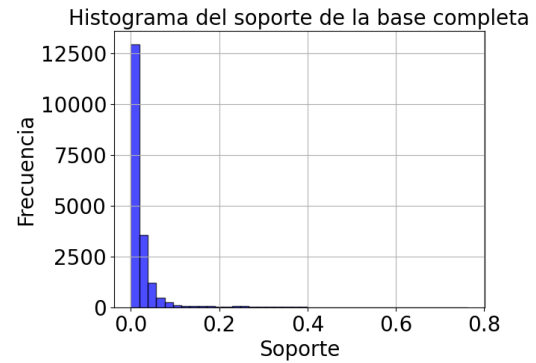


Figura 4.5: Histograma del soporte de la base completa con umbrales mínimos.

Con respecto a la **confianza** de las implicaciones, vista en el Histograma 4.4, esta es igual al de la base Luxenburger salvo por una diferencia. Como se ha añadido la base Stem y todas las implicaciones de esta tienen la confianza igual a 1 provoca que el último rango del histograma se despiunte hacia algo más del número total de implicaciones pertenecientes a dicha base.

El **soporte**, visto en el Histograma 4.5, no tiene mucha variación salvo que incrementa mucho más el número de implicaciones con un bajo soporte al ser esta la gran mayoría en la base Stem.

Cuadro 4.8: Resultados de la evaluación de la base completa en función de distintos valores de confianza y soporte de la base Luxenburger.

Confianza	Soporte	R. acierto	FP	FN	LHS	RHS	Total
0.000	0.000	0.385	110	0	6840	1898	19100
0.000	0.001	0.385	110	0	6685	1794	18841
0.000	0.003	0.385	110	0	6249	1691	17942
0.000	0.011	0.385	110	0	3264	1048	10682
0.000	0.045	0.385	110	0	651	360	3031
0.000	0.180	0.385	110	0	282	209	1270
0.200	0.000	0.385	110	0	6186	1670	16565

Continúa en la siguiente página

Confianza	Soporte	R. acierto	FP	FN	LHS	RHS	Total
0.200	0.001	0.385	110	0	6186	1670	16565
0.200	0.003	0.385	110	0	5800	1571	15788
0.200	0.011	0.385	110	0	3124	975	9602
0.200	0.045	0.385	110	0	641	342	2774
0.200	0.180	0.385	110	0	282	209	1268
0.500	0.000	0.458	95	2	3746	1189	10338
0.500	0.001	0.458	95	2	3746	1189	10338
0.500	0.003	0.458	95	2	3528	1116	9869
0.500	0.011	0.458	95	2	1973	670	6255
0.500	0.045	0.458	95	2	516	274	2273
0.500	0.180	0.458	95	2	282	204	1242
0.800	0.000	0.609	61	9	1120	471	3530
0.800	0.001	0.609	61	9	1120	471	3530
0.800	0.003	0.609	61	9	1120	471	3530
0.800	0.011	0.609	61	9	837	366	2875
0.800	0.045	0.609	61	9	339	216	1416
0.800	0.180	0.637	55	10	276	203	1071
0.900	0.000	0.654	46	16	470	277	1705
0.900	0.001	0.654	46	16	470	277	1705
0.900	0.003	0.654	46	16	470	277	1705
0.900	0.011	0.654	46	16	470	277	1705
0.900	0.045	0.665	43	17	305	213	1154
0.900	0.180	0.659	41	20	276	203	1000
0.950	0.000	0.698	33	21	296	217	1126
0.950	0.001	0.698	33	21	296	217	1126
0.950	0.003	0.698	33	21	296	217	1126
0.950	0.011	0.698	33	21	296	217	1126
0.950	0.045	0.698	32	22	283	207	1013
0.950	0.180	0.698	32	22	276	203	962

El comportamiento de los resultados del Cuadro 4.8 también es similar al de la base de Luxenburger solamente. Así que remarcaremos las diferencias que presenta con respecto a esta.

Lo primero es recordar que como la base Stem por si sola ya generaba 26 falsos positivos estos se mantendrán siempre en la base completa. Esto genera una cota

inferior en la columna de los falsos positivos haciendo que no lleguen a 0 como para la base de Luxenburger. Además vimos que por ejemplo para el último caso ya no aparecía el atributo **Survived** ni en el lado izquierdo ni derecho de las implicaciones para la base Luxenburger y esto podría hacernos pensar que entonces los resultados deberían de ser idénticos a los de la base Stem por si sola, pero por la presencia de 35 implicaciones en la base Luxenburger esto hace que en el cierre implicacional estas se disparen y encadenen otras implicaciones de la base Stem, con el RHS conteniendo **Survived**, que alteren el número de falsos positivos y negativos como es el caso.

El mejor **resultado** en este caso es el último de todos ya que es el que contiene menos implicaciones, con mayor confianza y soporte mínimos. Teniendo un **ratio de acierto** de 0.70, con 32 **falsos positivos** y 22 **falsos negativos**.

Para ser la combinación de ambas bases uno se esperaría que diera los mejores resultados, pero no es así. En el siguiente capítulo nos centraremos en mejorarlo, tratando de solucionar los principales problemas vistos con este análisis preliminar de los resultados.

5 | Ajustes y mejoras en la extracción de conocimiento

5.1 Modificaciones en las bases de implicaciones

Esta sección presenta una serie de mejoras en la fase de construcción de las bases de implicaciones con el objetivo de aumentar la relevancia del conocimiento extraído y mejorar la eficiencia y resultados del proceso.

Eliminación de implicaciones que nunca se verifican

Por lo visto en la última sección del *Capítulo 4* las implicaciones de la base a usar cuyo antecedente contenga el atributo **Survived** son irrelevantes a la hora de realizar el cierre implicacional. Ya que necesitan que en el conjunto esté presente el atributo objetivo, y si está ya presente entonces inevitablemente el objeto será marcado como resultado positivo por nuestro modelo.

Por ejemplo, para la base Stem vimos que este tipo de implicaciones comprendían en torno al 30 % de las implicaciones totales de la base. Así que es una reducción interesante del tamaño de las bases utilizadas.

Realizando su eliminación no se afectará a ningún resultado más. El ratio de acierto, los falsos positivos y los falsos negativos se mantendrán invariantes.

Filtrado de la base Stem por soporte mínimo

Nuestro objetivo es que en nuestro modelo se use la base completa para el cierre implicacional y hemos comprobado que los resultados son peores que los de la base Luxenburger. Por lo que trataremos de hacer modificaciones a la base Stem y así tratar de mejorar los resultados conjuntos.

Una primera idea sería aplicar los umbrales de la base Luxenburger a la base Stem. El umbral mínimo de confianza no tendría ningún efecto aplicárselo ya que todas las reglas de asociación estrictas (pertenecientes a la base Stem) tienen confianza igual a 1. Así que solo podríamos aplicarle el umbral mínimo de soporte para su filtrado.

La idea detrás de aplicar el filtrado por soporte mínimo a la base Stem radica en que buscamos minimizar el número de falsos positivos generados por esta, y la principal manera es aplicar algún tipo de filtro a dicha base. Asegurando que se cumpla dicho umbral de soporte estamos también obligando a trabajar con implicaciones que sean satisfechas por un gran número de objetos y así poder evitar las implicaciones generadas por casos particulares.

Cuadro 5.1: Resultados de la base completa tras la eliminación de las implicaciones LHS y habiendo aplicado el filtrado de la Stem por soporte mínimo.

Confianza	Soporte	R. acierto	FP	FN	RHS	Total
0.000	0.000	0.385	110	0	1898	12260
0.000	0.001	0.385	110	0	1730	12092
0.000	0.003	0.385	110	0	1587	11488
0.000	0.011	0.385	110	0	875	6895
0.000	0.045	0.385	110	0	158	1741
0.000	0.180	0.385	110	0	6	340
0.200	0.000	0.385	110	0	1670	10379
0.200	0.001	0.385	110	0	1606	10315
0.200	0.003	0.385	110	0	1467	9783
0.200	0.011	0.385	110	0	802	5955
0.200	0.045	0.385	110	0	140	1494
0.200	0.180	0.385	110	0	6	338

Continúa en la siguiente página

Confianza	Soporte	R. acierto	FP	FN	RHS	Total
0.500	0.000	0.458	95	2	1189	6592
0.500	0.001	0.486	88	4	1125	6528
0.500	0.003	0.486	88	4	1012	6136
0.500	0.011	0.492	87	4	497	3759
0.500	0.045	0.536	79	4	72	1118
0.500	0.180	0.816	14	19	1	312
0.800	0.000	0.609	61	9	471	2410
0.800	0.001	0.659	50	11	407	2346
0.800	0.003	0.665	49	11	367	2205
0.800	0.011	0.721	38	12	193	1515
0.800	0.045	0.827	6	25	14	438
0.800	0.180	0.615	0	69	0	147
0.900	0.000	0.654	46	16	277	1235
0.900	0.001	0.715	33	18	213	1171
0.900	0.003	0.732	30	18	173	1030
0.900	0.011	0.782	16	23	104	712
0.900	0.045	0.810	3	31	11	210
0.900	0.180	0.615	0	69	0	76
0.950	0.000	0.698	33	21	217	830
0.950	0.001	0.726	28	21	153	766
0.950	0.003	0.743	25	21	113	625
0.950	0.011	0.793	5	32	44	307
0.950	0.045	0.737	0	47	5	91
0.950	0.180	0.615	0	69	0	38

Analizando los resultados (Cuadro 5.1) se nota una **mejoría** considerable con respecto a los de la base Stem, y **la base completa**. Mientras que se asemejan mucho a los calculados simplemente con la base Luxenburger con ciertas diferencias.

Centrándonos en la base completa, se han mejorado los resultados que era el objetivo principal.

Reducción del cierre implicacional

En este ajuste se propone una modificación al proceso de cierre implicacional con el objetivo de reducir la introducción de atributos incorrectos y, con ello, minimizar la aparición de falsos positivos.

La idea que hay detrás consiste en iniciar el cierre implicacional a partir del conjunto de atributos que se cumplen exactamente para el objeto en cuestión, excluyendo en todo momento el atributo objetivo **Survived**. Es decir, solo se consideran los atributos verdaderos conocidos del objeto, y no aquellos que no se cumplen, ni tampoco el atributo objetivo.

Tomando este enfoque, si durante el cierre se aplica una implicación cuyo consecuente no contiene el atributo **Survived**, se estaría introduciendo un nuevo atributo que, en realidad, no está garantizado que se cumpla. Esto puede llevar a errores de falsos positivos en la clasificación final del objeto. Por ello, se propone filtrar la base de implicaciones para mantener únicamente aquellas cuyo consecuente contenga **Survived**. De este modo, se evita añadir atributos irrelevantes o incorrectos, y además se limita el proceso a una única ronda de inferencia, simplificando la ejecución del cierre implicacional.

Al final este ajuste pretende centrarse únicamente en las inferencias que aportan información directa sobre la predicción del atributo objetivo, mejorando así la precisión del modelo y reduciendo el número de inferencias innecesarias.

Cuadro 5.2: Resultados de la base completa tras los cambios anteriores y la simplificación a solo las reglas cuyo **RHS** contenga **Survived**

Confianza	Soporte	R. acierto	FP	FN	Total
0.000	0.000	0.385	110	0	1898
0.000	0.001	0.385	110	0	1730
0.000	0.003	0.385	110	0	1587
0.000	0.011	0.385	110	0	875
0.000	0.045	0.385	110	0	158
0.000	0.180	0.385	110	0	6
0.200	0.000	0.385	110	0	1670
<i>Continúa en la siguiente página</i>					

Confianza	Soporte	R. acierto	FP	FN	Total
0.200	0.001	0.385	110	0	1606
0.200	0.003	0.385	110	0	1467
0.200	0.011	0.385	110	0	802
0.200	0.045	0.385	110	0	140
0.200	0.180	0.385	110	0	6
0.500	0.000	0.581	71	4	1189
0.500	0.001	0.581	71	4	1125
0.500	0.003	0.581	71	4	1012
0.500	0.011	0.581	71	4	497
0.500	0.045	0.598	68	4	72
0.500	0.180	0.816	14	19	1
0.800	0.000	0.793	23	14	471
0.800	0.001	0.799	22	14	407
0.800	0.003	0.810	20	14	367
0.800	0.011	0.832	13	17	193
0.800	0.045	0.827	4	27	14
0.800	0.180	0.615	0	69	0
0.900	0.000	0.788	17	21	277
0.900	0.001	0.793	16	21	213
0.900	0.003	0.810	13	21	173
0.900	0.011	0.816	7	26	104
0.900	0.045	0.810	3	31	11
0.900	0.180	0.615	0	69	0
0.950	0.000	0.782	14	25	217
0.950	0.001	0.788	13	25	153
0.950	0.003	0.810	9	25	113
0.950	0.011	0.799	2	34	44
0.950	0.045	0.737	0	47	5
0.950	0.180	0.615	0	69	0

Los resultados de la prueba (Cuadro 5.2) han sido realmente positivos, habiendo incrementado del ratio de acierto en la mayoría de casos comparándolos con los anteriores. Además se ha llegado a un nuevo máximo del ratio de acierto a 0.832 con 13 falsos positivos y 17 falsos negativos para la confianza 0.80 y el soporte 0.011.

Si se explora en un entorno del máximo anterior aparece la base de 0.80 de con-

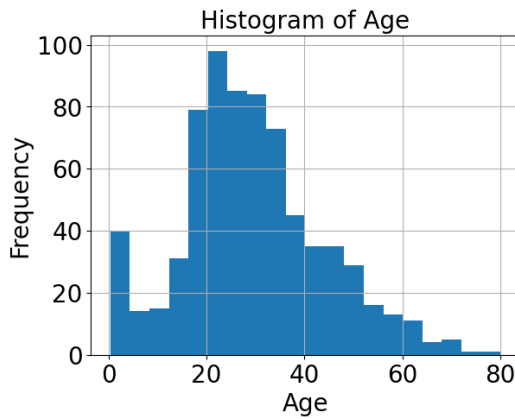


Figura 5.1: Histograma de Age del dataset Titanic.

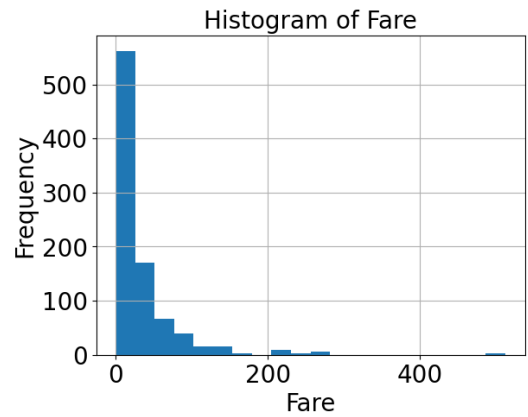


Figura 5.2: Histograma de Fare del dataset Titanic.

fianza y 0.018 de soporte para la cual el **ratio de acierto** es **0.844**, con 11 y 17 falsos positivos y negativos. Este es por ahora el resultado más elevado que se ha obtenido.

5.2 Ajuste de hiperparámetros: confianza, soporte y discretización

En esta sección se abordará el ajuste de los hiperparámetros del modelo. Hasta ahora, los únicos hiperparámetros considerados eran la confianza y el soporte, pero no son los únicos que pueden influir en el comportamiento del modelo.

Al aplicar un proceso de discretización, se toma una decisión sobre el número de rangos y su distribución en los atributos continuos. Así como se han utilizado ciertos rangos en las pruebas anteriores, podrían haberse elegido otros intervalos distintos, lo que afectaría directamente a los resultados obtenidos. Los histogramas 5.1 y 5.2 muestran la distribución de los datos de edades y tarifas, según decidamos los rangos estos contendrán más o menos objetos y podrían alterar las implicaciones obtenidas.

Por ello, se realizarán modificaciones al modelo para incorporar, además de la confianza y el soporte, el número de rangos de edad y de tarifa como hiperparámetros adicionales. Estos actuarán de la siguiente manera:

- **rangos_age**: define el número de rangos, todos del mismo tamaño, en los que se dividirá el intervalo de edades de 0 a 81 (máximo valor observado).
- **rangos_fare**: define el número de rangos, también de tamaño uniforme, en los que se dividirá el intervalo de tarifas de 0 a 513 (máximo valor observado).

En cuanto a **SibSp** y **Parch**, también podrían tratarse mediante hiperparámetros similares. Sin embargo, debido a la escasa variabilidad en sus valores, se ha decidido no modificarlos. Las variaciones en los rangos de edad y tarifa nos permitirán observar el impacto en el comportamiento del modelo, siendo este efecto representativo para otros atributos discretizados.

Cabe recordar que, en los análisis anteriores, el número de rangos utilizados tanto para la edad como para la tarifa fue de 6 pero con ciertas diferencias, porque los rangos no tenían todos el mismo tamaño.

Con esto en mente, se procede a definir el espacio de búsqueda para el ajuste de los hiperparámetros. Los valores considerados serán:

- **rangos_age y rangos_fare**: [4, 6, 8, 10, 12, 14, 16, 18]
- **confianza**: [0.5, 0.8, 0.85, 0.9, 0.95]
- **soporte**: [0/712, 1/712, 2/712, 4/712, 8/712, 16/712, 32/712, 128/712]

El conjunto de datos con el que se trabajará será el original que se separará en un 80 % para entrenamiento y un 20 % para las pruebas, como anteriormente. Esta separación será la misma para todas las combinaciones, osea que los resultados se harán teniendo en cuenta los mismos objetos para el entrenamiento e igualmente para la prueba.

Para cada combinación de valores, se calculará el ratio de acierto utilizando la base Stem sin modificaciones, la base Stem, la base Luxenburger y la base completa las tres con los filtrados. El filtrado se realizará según las condiciones de soporte, LHS y RHS definidas previamente.

Una vez obtenidos todos los resultados, se analizará cuál de las combinaciones ofrece el mejor ratio de acierto, si este mejora respecto al actual, cómo afecta la variación de rangos a cada base y qué valores de confianza y soporte resultan más efectivos...

De entre todos los resultados, el **mejor ratio de acierto** obtenido es de **0.844**, correspondiente tanto a la base Luxenburger como a la base completa. Este valor se

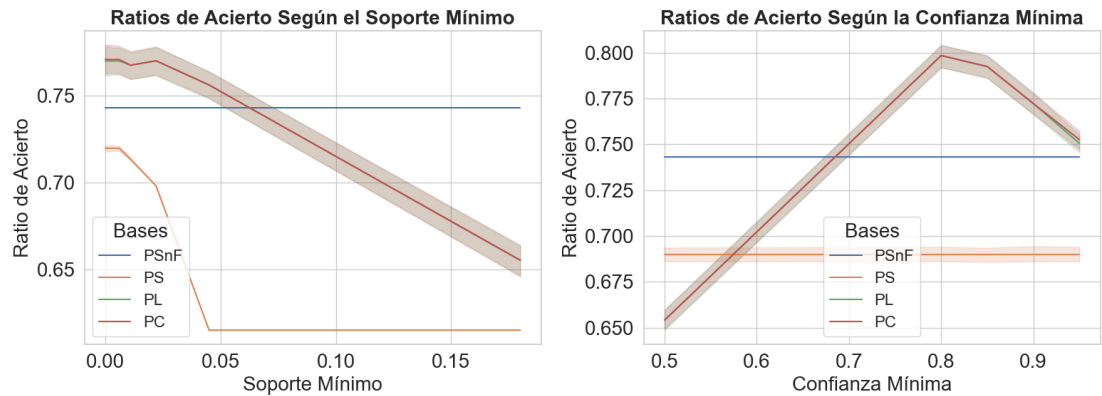


Figura 5.3: Variación los ratios de acierto de diferentes bases en función del soporte y la confianza.

alcanza con una confianza del 80 %, un soporte de 16/712, y utilizando a partir de 4 rangos para la edad y 8 para la tarifa.

El anterior dato muestra que aun variando los rangos de edades y de tarifas no se ha conseguido mejorar los resultados que se tenían hasta ahora, denota que puede que este sea el **ratio de acierto máximo posible** por el modelo para este conjunto de datos.

Sobre el conjunto resultante del ajuste se presentan una serie de gráficos que recogen todos los resultados obtenidos para así sacar conclusiones. Las leyendas son:

- **PSnF**: predicción base Stem sin filtro.
- **PS**: predicción base Stem con filtros de la sección anterior.
- **PL**: predicción base Luxenburger.
- **PC**: predicción base completa.

Los primeros son los gráficos de la Figura 5.3. El referido al **soporte** muestra que mientras menor sea el mínimo en general, mejores resultados de acierto para todas las bases. Esto se debe a que si aumenta en exceso se filtrarán demasiadas implicaciones y todo se marcará como resultados negativos, maximizando los falsos negativos. Para el referido a la **confianza** se ve que se alcanza el máximo para la confianza 80 % y las bases Luxenburger y completa, que hasta ahora nos han dado los mejores resultados para el caso del Titanic. Las otras dos bases se mantienen constantes al no depender de la confianza mínima.

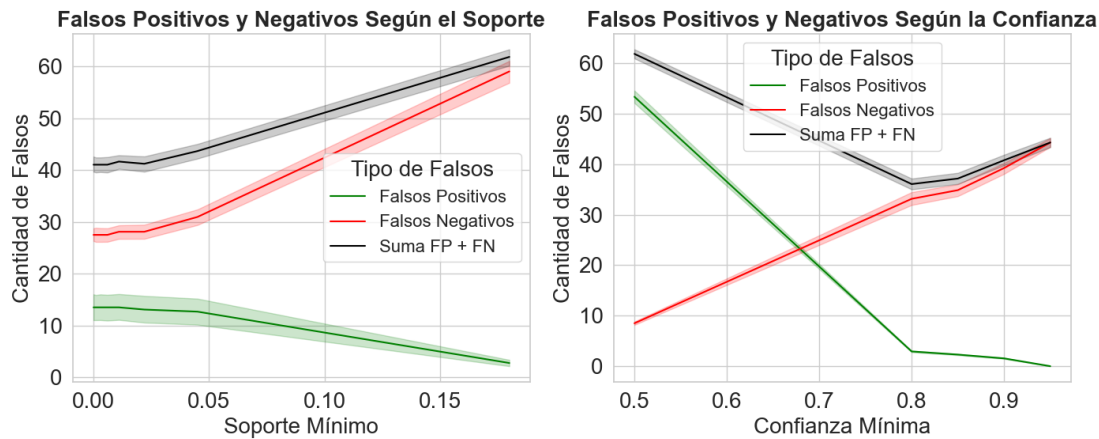


Figura 5.4: Falsos positivos (FP), falsos negativos (FN) y su suma en función del soporte mínimo y la confianza mínima para la base completa.

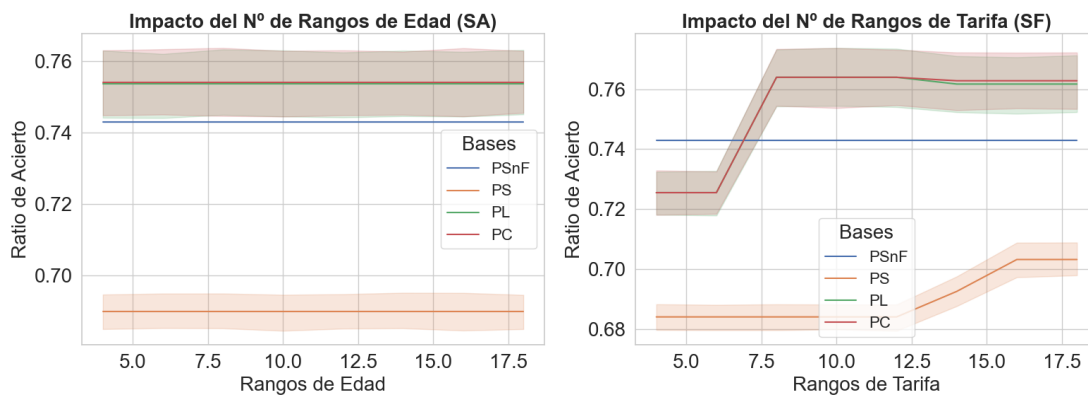


Figura 5.5: Impacto de los rangos en el ratio de acierto de cada base.

Por la Figura 5.4 se observa que aún variando los rangos de edad y tarifa se sigue manteniendo la disminución de los falsos positivos y el aumento de los falsos negativos cuando la confianza o el soporte aumentan. Esto es debido a la disminución de las implicaciones haciendo que se pase de marcar más resultados positivos a menos. Nuestro modelo debe minimizar la suma de ambas, que se obtiene para la confianza sobre el 80 % y el soporte cercano a 0.

Viendo las gráficas de la Figura 5.5 y el mapa de correlaciones de la Figura 5.6, salvo para la tarifa que aumenta de media en el paso de los 6 a los 8 rangos seguramente por

Mapa de Correlaciones de los Hiperparámetros

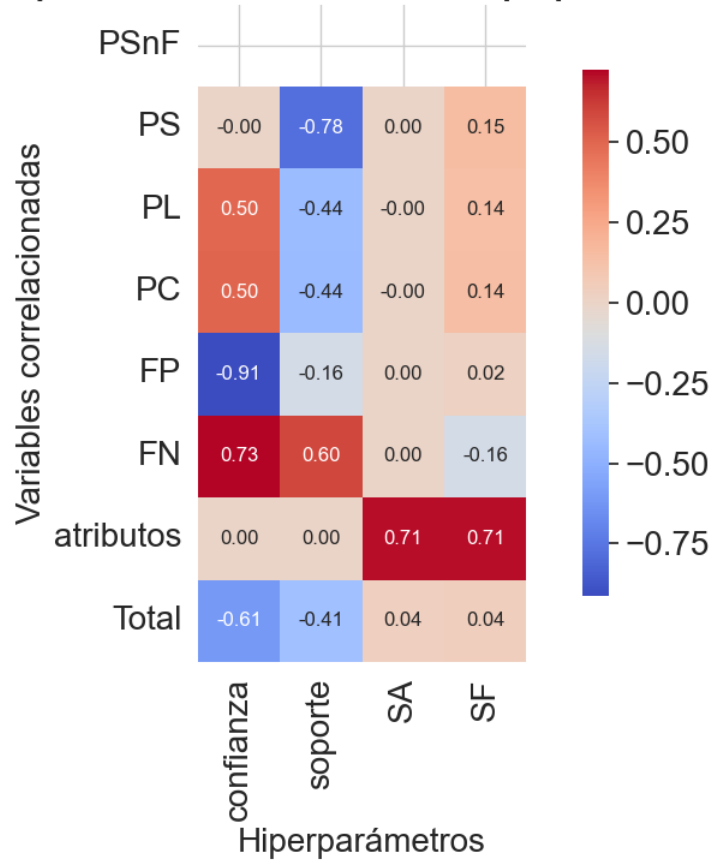


Figura 5.6: Correlaciones entre los hiperparámetros y los resultados.

la variación de alguna implicación que afecte a algún objeto a predecir, se puede llegar rápidamente a la conclusión de que la **variación de los rangos equidistantes** de edades y tarifas tienen un **efecto mínimo** en los resultados del modelo. Contrastando con la importancia de la elección de un buen soporte y confianza mínimos. Por lo que los **verdaderos hiperparámetros** de nuestro modelo serán únicamente la **confianza** y el **soporte** mínimos.

5.3 Validación externa del modelo

Una vez ajustado el modelo y detectado su punto óptimo en términos de hiperparámetros, con un valor de **confianza del 80 %** y un **soporte de 16/712**, así como

Model	Accuracy	Balanced Accuracy	ROC AUC	F1 Score
RandomForestClassifier	0.86	0.82	0.82	0.86
XGBClassifier	0.85	0.82	0.82	0.85
LGBMClassifier	0.85	0.82	0.82	0.85
LogisticRegression	0.83	0.80	0.80	0.83
GaussianNB	0.82	0.80	0.80	0.82

Cuadro 5.3: Comparativa de rendimiento entre modelos de clasificación en el conjunto Titanic

discretización en **4 rangos para la edad y 8 para la tarifa**. Se procedió a evaluar su capacidad generalizadora mediante el envío del conjunto de resultados a la plataforma **Kaggle**, específicamente en la competición *Titanic: Machine Learning from Disaster*.

El **ratio de acierto alcanzado fue de 0.75598**, lo que representa una ligera caída respecto al valor de **0.844** obtenido internamente en el conjunto de prueba del 20 %. Esta disminución es razonable y esperada, dado que la evaluación oficial se realiza sobre un conjunto oculto, no accesible durante el entrenamiento ni en las pruebas anteriores.

Comparación con otros modelos de IA

El ratio alcanzado (0.75598) se encuentra dentro de una franja competitiva, especialmente si se tiene en cuenta la naturaleza simbólica y explicativa del modelo propuesto. A continuación se presentan ratios de acierto aproximados obtenidos por otros enfoques más complejos y comunes en la competencia, según discusiones y notebooks compartidos por la comunidad en Kaggle [21]. Estos se pueden observar en el Cuadro 5.3.

Por tanto, aunque el modelo FCA no alcanza los niveles máximos de precisión logrados por soluciones avanzadas basadas en aprendizaje profundo o técnicas de ensamble, su rendimiento tampoco es despreciable. Además, su **capacidad de explicabilidad, bajo coste computacional y transparencia lógica** le otorgan un valor diferencial frente a modelos más opacos.

Columna	Tipo	Descripción
RowNumber	Entero	Índice de la fila.
CustomerId	Entero	Identificador único de cada cliente.
Surname	Texto	Apellido del cliente.
CreditScore	Numérico	Puntuación crediticia del cliente, relacionada con su solvencia.
Geography	Categorico	País de residencia: France, Germany o Spain.
Gender	Categorico	Género del cliente: Male o Female.
Age	Numérico	Edad del cliente, expresada en años.
Tenure	Entero	Años que el cliente lleva con el banco.
Balance	Numérico	Saldo promedio de la cuenta bancaria del cliente.
NumOfProducts	Entero	Número de productos bancarios contratados.
HasCrCard	Binario	Indica si el cliente posee una tarjeta de crédito.
IsActiveMember	Binario	Indica si el cliente es considerado activo.
EstimatedSalary	Numérico	Estimación del salario anual del cliente.
Exited	Binario	Variable objetivo: 1 si el cliente ha abandonado el banco, 0 en caso contrario.

Cuadro 5.4: Descripción de las columnas del conjunto de datos Churn Modelling.

5.4 Caso de estudio adicional: *Churn Modelling*

El resultado de aplicar el modelo a un solo conjunto de datos como el del Titanic puede que se quede corto y haga sacar conclusiones precipitadas de comportamientos particulares de dicho conjunto.

Por esta razón en esta sección se aplicará el modelo a otro conjunto de datos con una temática totalmente diferente llamado **Churn Modelling**. De esta manera se busca evadir peculiaridades y generalizar algo más los resultados obtenidos.

Descripción de Churn Modelling

El conjunto de datos utilizado en esta sección proviene de la plataforma Kaggle y está disponible públicamente bajo el nombre **Churn Modelling Dataset** [16]. Este dataset simula información bancaria de 10,000 clientes y está diseñado para abordar el problema de predicción de abandono de clientes (churn), un problema clásico en el

ID	Score	Age	Name	Sex	Country
1	619	42	Hargrave	Female	France
2	608	41	Hill	Female	Spain
3	502	42	Onio	Female	France
4	699	39	Boni	Female	France
5	850	43	Mitchell	Female	Spain

Cuadro 5.5: Datos personales de los primeros cinco clientes del dataset.

Tenure	Balance	Products	Card	Active	Salary	Exited
2	0.00	1	1	1	101348.88	1
1	83807.86	1	0	1	112542.58	0
8	159660.80	3	1	0	113931.57	1
1	0.00	2	0	0	93826.63	0
2	125510.82	1	1	1	79084.10	0

Cuadro 5.6: Información financiera de los primeros cinco clientes del dataset.

ámbito de la minería de datos y la inteligencia de negocio.

El conjunto de datos está compuesto por 14 variables que aparecen en el Cuadro 5.4.

El conjunto de datos

El conjunto de datos en **Kaggle** consta de un solo archivo *Churn_Modelling.csv* con 10,000 instancias de clientes.

El objetivo principal consiste en predecir si un cliente abandonará la entidad bancaria, representado por la variable objetivo **Exited**, la cual toma el valor 1 si el cliente se ha marchado y 0 en caso contrario.

Los cuadros 5.5 y 5.6 muestran los valores de los 5 primeros clientes que aparecen en el conjunto de datos.

Selección del subconjunto de datos

También llamaremos atributos a las columnas de nuestro conjunto de datos.

Para aplicar el Análisis Formal de Conceptos (AFC) al conjunto de datos Churn Modelling, se seleccionó un subconjunto de atributos que resultasen relevantes e influyentes a la hora de determinar el atributo objetivo **Exited**, que indica si un cliente ha abandonado el servicio.

Los atributos seleccionados son los siguientes:

- **HasCrCard** e **IsActiveMember**: características propias del cliente que pueden influir directamente en su fidelidad.
- **CreditScoreGroup 300-600** y **CreditScoreGroup 600-900**: indican el grupo de puntuación crediticia al que pertenece el cliente.
- **Geography France**, **Geography Germany**, **Geography Spain** y **Gender Female**, **Gender Male**: permiten identificar la localización geográfica y el género del cliente.
- **AgeGroup 0-18**, **18-35**, **35-55** y **55-100**: reflejan en qué intervalo de edad se encuentra el cliente.
- **TenureGroup 0-1**, **1-5**, **5-10**: representa la antigüedad del cliente en el banco, agrupada por rangos.
- **BalanceGroup 0-50000**, **50000-150000**, **150000-300000**: indica el nivel de saldo que mantiene el cliente en su cuenta.
- **NumOfProducts 1**, **2**, **3**, **4**: indica cuántos productos tiene contratados con el banco.
- **EstimatedSalaryGroup 0-50000**, **50000-100000**, **100000-200000**: agrupa a los clientes según su salario estimado.

Además, se incluye el atributo **Exited** como referencia para posteriores análisis y validación del conocimiento extraído. Esta se incorpora directamente en la construcción del contexto formal.

Preprocesamiento

El archivo en formato CSV fue leído mediante *pandas* y se almacenó en un objeto del tipo *DataFrame*.

La limpieza de datos fue bastante directa: no había valores nulos ni duplicados, ya que el conjunto original se encuentra limpio. Aun así, se comprobó que todos los atributos seleccionados tenían valores válidos dentro de los rangos esperados.

Los atributos numéricos como **CreditScore**, **Age**, **Balance**, **Tenure** y **EstimatedSalary** se transformaron en variables categóricas mediante discretización, y posteriormente se aplicó la *codificación one-hot*. Esto permitió generar atributos como **CreditScoreGroup** 300-600, **AgeGroup** 35-55 o **BalanceGroup** 0-50000.

Aplicamos el escalado

Como paso previo a la construcción del contexto formal, fue necesario transformar los atributos numéricos en atributos categóricos agrupados por rangos (escalado) y aplicar posteriormente una codificación binaria (*one-hot encoding*).

Al atributo **RowNumber**, que actúa como identificador de cada cliente, no se le aplica ningún cambio. Los atributos **HasCrCard**, **IsActiveMember** y **Exited** ya están en binario y no necesitan transformación.

Para el resto, se aplicaron los siguientes rangos dependientes de los mínimos y máximos de cada columna:

- **CreditScore**: [300, 600), [600, 900).
- **Age**: [0, 18), [18, 35), [35, 55), [55, 100).
- **Tenure**: [0, 1), [1, 5), [5, 10).
- **Balance**: [0, 50 000), [50 000, 150 000), [150 000, 300 000).
- **EstimatedSalary**: [0, 50 000), [50 000, 100 000), [100 000, 200 000).

Los atributos categóricos originales como **Geography**, **Gender** y **NumOfProducts** se codificaron directamente mediante *one-hot encoding*.

Construcción del contexto formal e implicaciones

Una vez obtenida la representación binaria del conjunto de datos, se procedió a intentar construir el contexto formal de la misma manera que para el dataset del **Titanic**, utilizando la librería `concepts` [12] de Python, que permite construir objetos del tipo `Context(objects, properties, bools)`.

Pero a la hora de crear el retículo de conceptos necesario para el cálculo de las implicaciones de la base de Luxenburger este tardaba demasiado, en torno a 1-2 días.

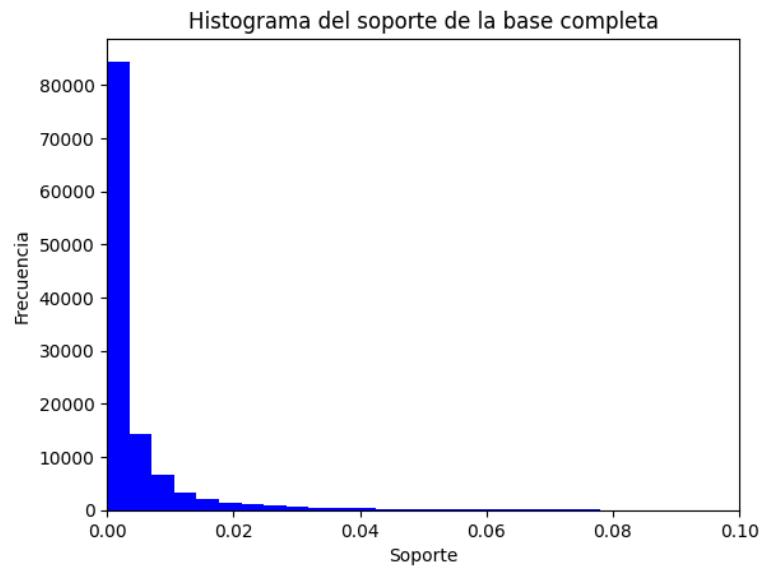


Figura 5.7: Histograma del soporte de las implicaciones de la base Completa.

Todo esto a raíz del número tan elevado de objetos de este conjunto de datos. Así que usaremos la herramienta de **ConExp**¹ para calcular la base completa con umbrales mínimos que guardaremos para trabajar con nuestra clase de python. Por lo que en nuestro contexto solo estarán definidas las bases de implicaciones, que nos bastarán para realizar los análisis del modelo.

Resultados y análisis

Una vez calculadas las implicaciones con ConExp realizamos los filtrados para separarlas en las implicaciones respectivas de la base Stem y Luxenburger. La base Stem cuenta con 7,825 implicaciones, la base Luxenburger con 110,095 y la base completa (unión de las anteriores) con 117,920.

Para las tres bases se ha calculado el ratio de acierto, los falsos positivos y los falsos negativos, con y sin los filtrados y propuestos en las secciones anteriores. En este caso los filtrados han dado mejores resultados, como pasaba con el Titanic. Así que solo se expondrán los resultados tras aplicar los filtros. Además para decidir en qué soportes (Histograma de los soportes 5.7) realizar las pruebas se usarán los sextiles del soporte (dividir el soporte en 6 rangos teniendo cada uno el mismo número de implicaciones).

¹<https://conexp.sourceforge.net/>

Algunas peculiaridades a tener en cuenta son que el soporte máximo de todas las implicaciones es 0.7088, el número máximo de falsos positivos es 1,593 y el de falsos negativos es 407. Por lo que si por ejemplo decidimos que todos los clientes se quedan en el banco (**Exited**=0) esto daría un ratio del 79.65 %, ya que el conjunto de datos de prueba consta de 2,000 objetos. Esto muestra una desigualdad entre los resultados del atributo objetivo, pero que no tendría que suponer un problema para el modelo.

Pasemos a calcular los resultados y analizarlos para las diferentes bases:

- **Base Stem:** por si sola da mucho mejores resultados comparándola con la base Stem del Titanic. Esto se debe principalmente al aumento en más de 10 veces del número de objetos a estudiar, por lo que las reglas de asociación exactas tienen una mayor exactitud y dependen menos de casos aislados. El mejor resultado sería para el soporte de 0.0005 con un ratio de acierto de 0.832 (véase el Cuadro 5.7).

Cuadro 5.7: Resultados de la evaluación de reglas de asociación estrictas con distintos valores de soporte.

Confianza	Soporte	R. acierto	FP	FN	Total
1.0	0.0	0.806	69	320	366
1.0	0.0005	0.823	15	339	118
1.0	0.0009	0.822	4	351	54
1.0	0.0015	0.820	4	356	33
1.0	0.0029	0.811	1	377	7
1.0	0.0069	0.796	0	407	0
1.0	0.7088	0.796	0	407	0

- **Base Luxenburger:** esta base produce unos resultados casi idénticos aunque en ocasiones peores que los de la base completa, por eso pasaremos a estudiar directamente los resultados de esta última.
- **Base Completa:** los mejores resultados los produce esta base con un ratio de acierto de 0.830 hasta en tres ocasiones. Aunque todos resultados en general están muy parejos y hay pocas desviaciones, por lo que casi que cualquier configuración de los hiperparámetros confianza y soporte darán un ratio de acierto cercano al 80 % (véase el Cuadro 5.8).

Cuadro 5.8: Resultados de la evaluación de la base completa de reglas de asociación.

Confianza	Soporte	R. acierto	FP	FN	Total
0.0	0.0	0.788	247	176	2713
0.0	0.0005	0.819	169	193	1747
0.0	0.0009	0.826	144	203	1210
0.0	0.0015	0.828	127	218	733
0.0	0.0029	0.830	102	237	282
0.0	0.0069	0.826	79	270	42
0.0	0.7088	0.796	0	407	0
0.2	0.0	0.788	247	176	2713
0.2	0.0005	0.819	169	193	1747
0.2	0.0009	0.826	144	203	1210
0.2	0.0015	0.828	127	218	733
0.2	0.0029	0.830	102	237	282
0.2	0.0069	0.826	79	270	42
0.2	0.7088	0.796	0	407	0
0.5	0.0	0.788	247	176	2713
0.5	0.0005	0.819	169	193	1747
0.5	0.0009	0.826	144	203	1210
0.5	0.0015	0.828	127	218	733
0.5	0.0029	0.830	102	237	282
0.5	0.0069	0.826	79	270	42
0.5	0.7088	0.796	0	407	0
0.8	0.0	0.804	92	301	1508
0.8	0.0005	0.816	53	314	1260
0.8	0.0009	0.826	25	323	867
0.8	0.0015	0.823	16	338	544
0.8	0.0029	0.822	9	347	209
0.8	0.0069	0.817	9	357	29
0.8	0.7088	0.796	0	407	0
0.9	0.0	0.806	72	316	732
0.9	0.0005	0.824	19	334	484
0.9	0.0009	0.824	12	339	420

Continúa en la siguiente página

Confianza	Soporte	R. acierto	FP	FN	Total
0.9	0.0015	0.820	8	351	293
0.9	0.0029	0.820	7	352	110
0.9	0.0069	0.814	4	369	9
0.9	0.7088	0.796	0	407	0
0.95	0.0	0.805	70	320	454
0.95	0.0005	0.822	16	339	206
0.95	0.0009	0.822	5	351	142
0.95	0.0015	0.822	5	352	121
0.95	0.0029	0.818	3	362	55
0.95	0.0069	0.810	2	377	4
0.95	0.7088	0.796	0	407	0

Tras aplicar el modelo a otro conjunto de datos mucho más rico en objetos y comparar sus resultados con los del Titanic podemos sacar varias conclusiones:

1. El **tiempo de ejecución** como comentamos en el *Capítulo 3* crece considerablemente con respecto al número de objetos con nuestro algoritmo. Llegando al punto de necesitar utilizar ConExp para su cálculo.
2. Los resultados de los **ratios de acierto son muy similares** con 0.844 para el Titanic y 0.830 para el Churn Modelling. Lo que puede dar indicios de que el modelo siempre suela dar esos resultados, ya que los conjuntos de datos con completamente diferentes en todos los ámbitos.
3. Las variaciones del ratio de acierto de los resultados de Churn Modelling se mueven en rangos mucho más acotados que los del Titanic, lo que puede sugerir que mientras **mayor** sea el **número de objetos** en el entrenamiento **menor** será la **varianza** de los mismos.
4. Lo anterior también implica que mientras **mayor** sea el **número de objetos** **menos importancia** tendrá la elección de los **hiperparámetros** de confianza y soporte. Ya que aunque los falsos positivos y negativos varíen más el ratio de acierto se mantiene cercano a su media

6 | FCA como Herramienta para la Explicabilidad en IA

En los últimos años, la necesidad de que los modelos de Inteligencia Artificial sean comprensibles e interpretables ha crecido drásticamente. El término de XAI (eXplainable Artificial Intelligence) hace referencia al conjunto de técnicas cuyo objetivo es ofrecer explicaciones comprensible sobre las predicciones realizadas por modelos complejos, como redes neuronales o árboles de decisión profundos [17].

El Análisis Formal de Conceptos presenta una nueva alternativa dentro de este ámbito [3, 19]. Su capacidad para extraer implicaciones entre atributos permite representar el conocimiento de manera estructurada y comprensible, y esto es especialmente útil para analizar el comportamiento de modelos de clasificación. Gracias a la extracción de implicaciones válidas o parciales desde los datos o desde las predicciones de un modelo, es posible identificar patrones relevantes, inconsistencias, y ofrecer justificaciones lógicas para aciertos o errores en la clasificación.

En este capítulo nos centraremos en el manejo de las implicaciones extraídas mediante AFC como herramienta explicativa, analizando tanto verdaderos positivos como falsos positivos y, en un análisis complementario, extendiéndolo a verdaderos y falsos negativos. Para ello, nos apoyaremos principalmente en el caso de estudio del conjunto de datos del Titanic.

6.1 Análisis Explicable de Resultados: VP y FP

Uno de los enfoques más directos de la explicabilidad en clasificación binaria consiste en analizar los errores y aciertos del modelo. En este apartado, nos centraremos en:

- **Verdaderos Positivos (VP):** predicciones correctas de supervivientes del Titanic.
- **Falsos Positivos (FP):** pasajeros clasificados como supervivientes que realmente no lo eran.

Trabajaremos sobre el contexto del Titanic con el mayor ratio de acierto calculado, el 0.844 obtenido en el ajuste de hiperparámetros con el filtrado en el *Capítulo 5*. Este se creó a partir de la siguiente discretización de los atributos para la confianza del 80 % y el soporte de 16/712:

- **Age:** [0, 20), [20, 40), [40, 60), [60, 81) (80 es el máximo).
- **Fare:** [0, 64), [64, 128), [128, 192), [192, 256), [256, 320), [320, 384), [384, 448), [448, 513) (512 es el máximo).
- **SibSp:** [0, 1), [1, 3), [3, 5), [5, 7), [7, ∞) (8 es el máximo).
- **Parch:** [0, 1), [1, 3), [3, 5), [5, ∞) (6 es el máximo).

Este contexto se ha calculado con el 80 % del conjunto de datos original. Para la prueba se tomó el 20 % (179 objetos) resultante que contenía 69 supervivientes y 110 no supervivientes. Tras la ejecución del modelo se obtuvieron 13 falsos positivos y 15 falsos negativos.

Verdaderos Positivos

Primero analizaremos un subconjunto de los verdaderos positivos, que son 54 en total. Recordemos que, gracias a los filtrados propuestos en el anterior capítulo, el cierre implicacional constará siempre de una sola ronda de disparos de las reglas de la base completa. Esto se debe a que las filtramos para solo quedarnos con las que contengan el atributo **Survived** en el consecuente. Así que lo que haremos será almacenar todas las implicaciones que se podrían disparar para cada objeto y quedarnos con la de mayor confianza. Esta implicación nos estará aportando la información más segura para considerar nuestro objeto como superviviente.

A continuación se muestran los atributos de 5 de los 54 verdaderos positivos seleccionados al azar, junto a su respectiva implicación con máxima confianza que realiza la decisión correcta de clasificarlo como superviviente.

- 1.

- FareGroup_0-64
- AgeGroup_20-40
- Embarked_S
- Pclass_1
- SibSpGroup_1-3
- ParchGroup_0-1
- Male

Imp.: {AgeGroup_20-40, SibSpGroup_1-3, Pclass_1} → Survived (conf. = 0.81)

2.
 - FareGroup_0-64
 - AgeGroup_0-20
 - Embarked_S
 - Pclass_3
 - SibSpGroup_0-1
 - ParchGroup_1-3
 - Female

Imp.: {Embarked_S, Female, ParchGroup_1-3, SibSpGroup_0-1} → Survived (conf. = 0.88)

3.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_2
 - SibSpGroup_1-3
 - ParchGroup_0-1
 - Female

Imp.: {AgeGroup_20-40, Pclass_2, Female, ParchGroup_0-1, FareGroup_0-64} → Survived (conf. = 0.96)

4.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_2
 - SibSpGroup_0-1
 - ParchGroup_1-3
 - Female

Imp.: {AgeGroup_20-40, Pclass_2, Female, FareGroup_0-64, SibSpGroup_0-1} → Survived (conf. = 0.96)

5.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_C
 - Pclass_1
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Female

Imp.: {AgeGroup_20-40, SibSpGroup_0-1, Female, Pclass_1} → Survived (conf. = 1.00)

Como se puede ver, las implicaciones que se activan en los verdaderos positivos tienen bastante sentido y encajan con lo que ya sabemos sobre el Titanic: muchas

mujeres, personas jóvenes o con pocos acompañantes, y sobre todo los de primera clase, tenían más probabilidades de sobrevivir. Gracias a este enfoque podemos ver claramente por qué el modelo ha acertado en esos casos y qué información ha utilizado para justificar su decisión.

Además, al quedarnos con la implicación de mayor confianza, nos aseguramos de que la explicación sea la más sólida posible. Es decir, la que mejor refleja el patrón aprendido por el modelo.

Falsos Positivos

En esta parte vamos a hacer lo mismo pero con falsos positivos, para ver qué reglas han llevado al modelo a equivocarse clasificando como supervivientes a personas que en realidad no lo fueron. Esto nos ayudará a entender mejor en qué situaciones el modelo falla y si hay algún tipo de error común.

El número total de falsos positivos en este caso es de 13, escogeremos 5 al azar para ver cual ha sido la implicación de mayor confianza que ha originado la decisión incorrecta en cada caso.

1.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_2
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Female

Imp.: {AgeGroup_20-40, Pclass_2, Female, ParchGroup_0-1, FareGroup_0-64}
→ Survived (conf. = 0.96)

2.
 - FareGroup_64-128
 - AgeGroup_0-20
 - Embarked_C
 - Pclass_1
 - SibSpGroup_1-3
 - ParchGroup_0-1
 - Male

Imp.: {ParchGroup_0-1, SibSpGroup_1-3, Pclass_1, Embarked_C} → Survived
(conf. = 0.89)

- 3.

- FareGroup_0-64
- AgeGroup_20-40
- Embarked_S
- Pclass_2
- SibSpGroup_1-3
- ParchGroup_0-1
- Female

Imp.: {AgeGroup_20-40, Pclass_2, Female, ParchGroup_0-1, FareGroup_0-64}
→ Survived (conf. = 0.96)

4.
 - FareGroup_64-128
 - AgeGroup_40-60
 - Embarked_S
 - Pclass_1
 - SibSpGroup_1-3
 - ParchGroup_0-1
 - Male

Imp.: {ParchGroup_0-1, FareGroup_64-128, Pclass_1} → Survived (conf. = 0.82)

5.
 - FareGroup_0-64
 - AgeGroup_0-20
 - Embarked_S
 - Pclass_3
 - SibSpGroup_1-3
 - ParchGroup_1-3
 - Female

Imp.: {Embarked_S, ParchGroup_1-3, AgeGroup_0-20, FareGroup_0-64, SibSpGroup_1-3} → Survived (conf. = 0.80)

Como se puede ver, muchas de las reglas que se disparan en los falsos positivos son similares o incluso idénticas a las que aparecían en verdaderos positivos. Esto es hasta cierto punto esperable ya que algunas características que suelen estar asociadas a la supervivencia pueden no ser suficientes por sí solas para garantizar un acierto, especialmente si se trata de casos que se apartan ligeramente del patrón general o que representan excepciones.

Por ejemplo, la implicación con confianza 0.96 en el primer falso positivo también aparece en varios verdaderos positivos. Esto indica que el modelo ha aprendido que ese conjunto de atributos normalmente implica la supervivencia, aunque no siempre sea así. En estos casos, el análisis con AFC permite detectar estas limitaciones y matices del modelo, revelando reglas que funcionan bien en general pero que pueden llevar a errores puntuales.

Además, al poder identificar qué implicación concreta ha motivado la predicción, podemos plantearnos acciones como refinar las reglas, añadir más atributos para contextos ambiguos, o incluso revisar los datos de entrenamiento para buscar sesgos. Este

tipo de análisis no solo nos hace confiar más en el modelo, sino que también facilita ajustes más precisos y nos aporta explicaciones más claras para los usuarios.

6.2 Análisis Explicable de Resultados: VN y FN

Siguiendo el mismo procedimiento utilizado para los verdaderos y falsos positivos, en esta sección abordaremos el análisis explicativo de:

- **Verdaderos Negativos (VN):** predicciones correctas de pasajeros que no sobrevivieron.
- **Falsos Negativos (FN):** pasajeros clasificados como no supervivientes que realmente si lo eran.

Para ello, trabajaremos sobre el mismo contexto de la sección anterior. Pero incluiremos en el contexto el atributo **notSurvived**, que representa la clase negativa, y filtraremos las implicaciones de la base completa para conservar únicamente aquellas cuyo consecuente contenga dicho atributo o el atributo **Survived**. De esta forma, podremos estudiar las reglas que llevan al modelo a decidir que un individuo no sobrevivió, igual que hicimos anteriormente con las implicaciones hacia **Survived**. Añadir este atributo no cambia los resultados del modelo, ya que las implicaciones que anteriormente daban como superviviente o no a un pasajero seguirán estando presentes en la base completa.

Verdaderos Negativos

Para este grupo, analizaremos 5 de los 97 verdaderos negativos. Como en las secciones anteriores, seleccionamos para cada objeto la implicación con mayor confianza que haya sido activada durante la clasificación y que justifique la decisión de etiquetarlo como no superviviente. Esta implicación debe tener como consecuente el atributo **NotSurvived**.

1.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_1
 - SibSpGroup_0-1
 - ParchGroup_0-1

- Male

Imp.: {Male, Embarked_S, AgeGroup_20-40} → NotSurvived (conf. = 0.85)

2.
 - FareGroup_192-256
 - AgeGroup_20-40
 - Embarked_C
 - Pclass_1
 - SibSpGroup_0-1
 - ParchGroup_1-3
 - Male

Imp.: {Male, ParchGroup_1-3, AgeGroup_20-40} → NotSurvived (conf. = 0.86)

3.
 - FareGroup_0-64
 - AgeGroup_0-20
 - Embarked_S
 - Pclass_3
 - SibSpGroup_3-5
 - ParchGroup_1-3
 - Female

Imp.: {Pclass_3, SibSpGroup_3-5, ParchGroup_1-3, FareGroup_0-64} → NotSurvived (conf. = 0.86)

4.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_3
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Male

Imp.: {Pclass_3, Embarked_S, AgeGroup_20-40, Male} → NotSurvived (conf. = 0.89)

5.
 - FareGroup_0-64
 - AgeGroup_0-20
 - Embarked_Q
 - Pclass_3
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Male

Imp.: {ParchGroup_0-1, Embarked_Q, SibSpGroup_0-1, FareGroup_0-64, Male} → NotSurvived (conf. = 0.91)

Las reglas que explican estos verdaderos negativos muestran patrones fáciles de reconocer, como hombres adultos de tercera clase con pasajes baratos y pocos acompañantes. Son perfiles que, en general, tenían pocas posibilidades de sobrevivir, y el modelo ha aprendido a identificarlos bien. Estas implicaciones no solo tienen lógica, sino que también ayudan a confiar más en el sistema, porque muestran que las decisiones correctas se basan en patrones claros y conocidos.

Falsos Negativos

En este apartado nos centraremos en los 15 pasajeros que fueron clasificados erróneamente como no supervivientes, cuando en realidad sí sobrevivieron. Analizar estos casos nos permite identificar patrones que el modelo no ha captado correctamente o contextos en los que las reglas extraídas inducen a error.

Como antes, seleccionamos 5 de estos falsos negativos al azar y extraemos la implicación de mayor confianza que fue activada durante la clasificación. Todas estas implicaciones tienen como consecuente el atributo **NotSurvived**, lo cual explica por qué el modelo tomó la decisión equivocada.

1.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_1
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Male

Imp.: {Male, Embarked_S, AgeGroup_20-40} → NotSurvived (conf. = 0.85)

2.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_3
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Male

Imp.: {Pclass_3, Embarked_S, AgeGroup_20-40, Male} → NotSurvived (conf. = 0.89)

3.
 - FareGroup_448-513
 - AgeGroup_20-40
 - Embarked_C
 - Pclass_1
 - SibSpGroup_0-1
 - ParchGroup_1-3
 - Male

Imp.: {Male, ParchGroup_1-3, AgeGroup_20-40} → NotSurvived (conf. = 0.86)

4.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_3
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Male

Imp.: {Pclass_3, Embarked_S, AgeGroup_20-40, Male} → NotSurvived (conf. = 0.89)

5.
 - FareGroup_0-64
 - AgeGroup_20-40
 - Embarked_S
 - Pclass_1
 - SibSpGroup_0-1
 - ParchGroup_0-1
 - Male

Imp.: {Male, Embarked_S, AgeGroup_20-40} $\rightarrow$ NotSurvived (conf. = 0.85)

Estos falsos negativos muestran que el modelo tiende a clasificar como no supervivientes a pasajeros que presentan características más parecidas a perfiles con bajas probabilidades de sobrevivir según el patrón general aprendido, aunque en la realidad sí lograron sobrevivir. Este suceso puede indicar limitaciones en el conjunto de atributos usados, o que ciertas decisiones difíciles de predecir estén presentes en los datos.

Usar las implicaciones nos ayuda a entender mejor qué reglas hicieron que el modelo fallara, dándonos pistas claras para mejorar, como añadir más atributos o tener en cuenta contextos más complejos.

7 | Conclusiones y Trabajo Futuro

En este documento se ha diseñado y presentado un modelo explicativo fundamentado en el Análisis Formal de Conceptos (AFC) con el propósito de ofrecer una mayor interpretabilidad a los resultados obtenidos de modelos de aprendizaje automático. La aplicación de implicaciones formales, que se derivan de contextos construidos a partir de datos reales, ha demostrado ser una estrategia eficaz para simbolizar y hacer accesible el conocimiento, lo que a su vez aumenta la transparencia en los sistemas y fomenta la confianza del usuario en las predicciones obtenidas.

Los resultados obtenidos de los conjuntos de datos *Titanic* y *Churn Modelling* indican que las bases de implicaciones, en particular la Stem y Luxenburger, son útiles para detectar patrones coherentes entre diferentes atributos. Estos pueden ser aplicados en tareas explicativas así como en etapas iniciales de clasificación. No obstante, se ha observado que la naturaleza y calidad del conocimiento recuperado están influenciadas en gran medida por factores como la discretización, el filtrado de reglas, el número de objetos analizados y el manejo de los valores faltantes. Además, se ha notado que la ausencia del atributo en el lado derecho (RHS) de las implicaciones puede restringir en gran parte el potencia explicativo de las reglas extraídas porque se puede llegar a incoherencias de atributos.

Aunque se han hecho progresos significativos, el estudio presenta múltiples oportunidades para seguir investigación que podrían ser:

- **Tratamiento de valores ausentes:** se plantea como futuro trabajo estudiar el efecto de no rellenar ningún dato faltante en el conjunto de entrada, ni durante el entrenamiento ni en la evaluación. Esto permitirá evaluar cómo el AFC se comporta en presencia de datos incompletos, preservando así información potencialmente relevante y evitando sesgos introducidos artificialmente. En este escenario, se propone habilitar todas las implicaciones de la base correspon-

diente, incluyendo aquellas que no contienen el atributo objetivo en el RHS, de esta manera las implicaciones también podrían predecir estos atributos faltantes mejorar los resultados.

- **Exploración de nuevos tipos de rangos en la discretización:** hasta el momento se han utilizado principalmente rangos equidistantes para convertir atributos numéricos en booleanos. Sin embargo, se propone estudiar la incorporación de rangos más expresivos como "menor que", "mayor que", o combinaciones complejas, mediante escalas ordinales o biordinales, que puedan capturar relaciones más ajustadas a la semántica del dominio y generar implicaciones más precisas.
- **Análisis de la robustez ante perturbaciones:** se propone realizar un estudio sistemático sobre la robustez del modelo frente a cambios en los objetos de entrada. Se parte de la hipótesis de que el enfoque basado en implicaciones es **poco robusto** a ligeras modificaciones en los datos, dado que pequeños cambios pueden alterar significativamente el conjunto de implicaciones válidas. Evaluar esta sensibilidad permitirá proponer mecanismos de estabilización o normalización del conocimiento extraído. Se podrían aplicar las ideas de [2] a nuestro caso.
- **Estudio profundo del uso del atributo objetivo negado:** otra línea prometedora es analizar el efecto de trabajar con la negación del atributo objetivo. Por ejemplo, de manera similar a lo que se hizo en el *Capítulo 6*, en vez de extraer implicaciones que concluyan con "Survived", estudiar también las que concluyen con "¬Survived" y si un objeto tras el cierre de implicaciones obtiene ambas, entonces adjudicarle el atributo objetivo que maximice la confianza. Esta manera puede revelar otros patrones más claros o complementarios, y mejorar la cobertura explicativa del modelo sobre casos negativos o dudosos.

En conjunto, este trabajo busca profundizar en el uso de AFC como herramienta para la *Inteligencia Artificial Explicable*, no solo mejorando su rendimiento práctico, sino también su integración en contextos reales, donde los datos pueden estar incompletos, ruidosos o mal distribuidos. El objetivo es llegar a conseguir sistemas de IA que no solo sean eficaces, sino también comprensibles, justos y fiables.

Bibliografía

- [1] ALBANO, A. *Polynomial growth of concept lattices, canonical bases and generators: extremal set theory in Formal Concept Analysis*. Ph.d. thesis, Technische Universität Dresden, 2017. Qucosa Dresden.
- [2] ALVAREZ-MELIS, D., AND JAAKKOLA, T. S. On the robustness of interpretability methods. In *Proceedings of the International Conference on Machine Learning* (2018), pp. 1–7.
- [3] BHASKARAN, V., AND BARUAH, P. K. A novel approach to explainable ai using formal concept lattice. *International Journal of Innovative Technology and Exploring Engineering* 11, 7 (2022), 1–17.
- [4] BORREGO DÍAZ, J., CORDÓN FRANCO, A., AND RAMOS GONZÁLEZ, V. Tema 4: Conceptualización y organización del conocimiento. sistemas inteligentes. análisis formal de conceptos, 2023.
- [5] BORREGO DÍAZ, J., CORDÓN FRANCO, A., AND RAMOS GONZÁLEZ, V. Tema 5: Razonamiento en análisis formal de conceptos, 2023.
- [6] CUKIERSKI, W. Titanic: Machine learning from disaster. <https://www.kaggle.com/competitions/titanic>, 2012. Último acceso: 27 de mayo de 2025.
- [7] DIWALI, A., SAEEDI, K., DASHTIPOUR, K., GOGATE, M., CAMBRIA, E., AND HUSSAIN, A. Sentiment analysis meets explainable artificial intelligence: A survey on explainable sentiment analysis. *IEEE Transactions on Affective Computing* 15, 3 (2024), 837–846.
- [8] DUQUENNE, V., AND GUIGUES, J.-L. Famille minimale d’implications informatives résultant d’un tableau de données binaires. *Mathématiques et Sciences Humaines* 95, 24 (1986), 5–18.

- [9] GANTER, B., AND WILLE, R. *Formal Concept Analysis: Mathematical Foundations*. Springer, Heidelberg, 1999.
- [10] GRUBER, T. R. A translation approach to portable ontology specifications. *Knowledge Acquisition* 5, 2 (1993), 199–220.
- [11] KAYTOUE, M., NAPOLI, A., AND KUZNETSOV, S. O. Formal concept analysis for knowledge discovery and data mining. In *Formal Concept Analysis. ICFCA 2017. Lecture Notes in Computer Science* (2017), G. Stumme and L. Kwuida, Eds., vol. 10308 of *Lecture Notes in Computer Science*, Springer, pp. 255–267.
- [12] KOPP, P. concepts: Formal concept analysis for python, 2021. Versión consultada en 2025.
- [13] KRIEGEL, F., AND BORCHMANN, D. Nextclosures: Parallel computation of the canonical base. In *Proceedings of the 14th International Conference on Formal Concept Analysis (ICFCA 2017)* (Dresden, Germany, 2017), vol. 10308 of *Lecture Notes in Computer Science*, Springer, pp. 125–141.
- [14] KUZNETSOV, S. On computing the size of a lattice and related decision problems. *Order* 18 (2001), 313–321.
- [15] LUXENBURGER, M. *Implikationen, Abhängigkeiten und Galois-Abbildungen: Beiträge zur formalen Begriffsanalyse*. PhD thesis, Technische Hochschule Darmstadt, Aachen, 1993. Teil I: Partielle Implikationen.
- [16] MECHLEARN, S. Churn modelling dataset. <https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling>, 2020. Último acceso: 25 de mayo de 2025.
- [17] MOLNAR, C. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Lulu.com, 2020.
- [18] POELMANS, J., IGNATOV, D. I., KUZNETSOV, S. O., AND DEDENE, G. Formal concept analysis in knowledge processing: A survey on applications. *Expert Systems with Applications* 40, 16 (2013), 6538–6560.
- [19] SANGROYA, A., ANANTARAM, C., RAWAT, M., AND RASTOGI, M. Using formal concept analysis to explain black box deep learning classification models. In *Proceedings of International Workshop on Machine Learning* (2019), pp. 19–26.
- [20] SCHÜTT, D. Abschätzungen für die anzahl der begriffe von kontexten. Diplomarbeit, Technische Hochschule Darmstadt, 1987.

- [21] SKIKRI, H. Titanic classification kaggle competition. <https://github.com/SkikriHassane01/Titanic-classification-kaggle-competition>, 2024. Último acceso: 28 de mayo de 2025.
- [22] STUMME, G., TAOUIL, R., BASTIDE, Y., PASQUIER, N., AND LAKHAL, L. Intelligent structuring and reducing of association rules with formal concept analysis. In *KI 2001: Advances in Artificial Intelligence* (Berlin, Heidelberg, 2001), F. Baader, G. Brewka, and T. Eiter, Eds., vol. 2174 of *Lecture Notes in Artificial Intelligence*, Springer, pp. 335–350.